

目录

快排	3rd
归并排序	5th
高精度加法	7th
高精度减法	9th
高精度乘低精度	11th
高精度除低精度	13th
前缀和	15th
二维前缀和	17th
差分	21st
差分数组（二维）	24th
双指针	28th
位运算	30th
离散化	39th
单链表	49th
双链表	53rd
栈	57th
单调栈	60th
队列	62nd
KMP 算法	65th
Trie 树	67th
查并集	71st
查并集例题	76th
堆	79th
堆排序	81st
堆的 5 个操作（带映射）	83rd
哈希表-开放寻址法	88th
哈希表-拉链法	92nd
深度优先搜索（八皇后）	96th
深度优先搜索（全排列）	104th
广度优先搜索	106th
数和图的存储	109th
数和图的遍历（宽度优先遍历）	114th
数和图的遍历（深度优先遍历）	117th
最短路	120th
最短路-朴素 Dijkstra 算法	122nd
最短路-堆优化版的 Dijkstra 算法	126th
最短路-Bellman-Ford 算法	130th
最短路-SPFA 算法	134th
最短路-Floyd 算法	139th
最小生成树-朴素 Prim 算法	142nd
最小生成树-堆优化 Prim 算法（略）	145th
最小生成树-Kruskal 算法	146th
二分图-染色法	149th



加油喵！QwQ！

二分图-匈牙利算法 152nd

优化代码效率 155th

快速判断素数 155th



快排

```

1
2
3 // 快排的思路：
4 // 1. 先找到一个基准值，一般是第一个值
5 // 2. 然后定义两个指针，一个指向第一个值，一个指向最后一个值
6 // 3. 然后左指针向右移动，右指针向左移动，直到左指针指向的值大于基准值，右指针指
7 向的值小于基准值
8 // 4. 然后交换左指针和右指针指向的值
9
10 // 有一个反转其排序顺序的非常作弊的方法：可以将传入的数组里的每个数处理一下变成
11 其负数，然后再进行快排，最后再将每个数处理一下变成其正数
12
13 #include<iostream>
14
15 using namespace std;
16 const int N = 100010;
17
18 int n;
19 int arr[N];
20
21 void quick_sort(int arr[], int l, int r)
22 {
23     if(l >= r) return; // 如果左指针大于等于右指针，说明此时 l~r 之间只有一个数或
24     没有数，直接返回
25
26     int x = arr[l]; // 记录当时的左指针指向的值
27     int i = l - 1; // 初始化左指针为第一位的前一位
28     int j = r + 1; // 初始化右指针为最后一位的后一位
29
30     while(i < j) // 作用是让左边的都小于记录值，右边的都大于记录值
31     {
32         do i++; while(arr[i] < x); // 左指针向右移动到下一位（然后判断当前左指针
33         指向的值和记录的值（起始位置的值的值）的大小），如果记录的值（左边的值）大于当前值
34         （右边的值），左指针就继续向右移动一位，直到找到一个大于等于记录值的值
35         do j--; while(arr[j] > x); // 右指针向左移动到上一位（然后判断当前右指针
36         指向的值和记录的值的大小），如果记录的值小于当前值，右指针就继续向左移动一位，
37         直到找到一个小于等于记录值的值
38
39         if(i < j) swap(arr[i], arr[j]); // 如果左指针在右指针的左边，交换左右指
40         针指向的值
41     }
42     quick_sort(arr, l, j); // 对大于等于（右边部分排序）
43     quick_sort(arr, j + 1, r); // 对小于等于（左边部分排序）

```



加油喵! QwQ !

```
44 }
45
46 int main()
47 {
48     scanf("%d", &n);
49     for(int i = 0; i < n; i++)
50     {
51         scanf("%d", &arr[i]);
52     }
53
54     quick_sort(arr, 0, n - 1); // 对 arr 数组的 0 到 n-1 使用快排
55
56     cout << "答案:" << endl;
57     for(int i = 0; i < n; i++)
58     {
59         printf("%d ", arr[i]); // 注意这里有一个空格" "
60     }
61
62     return 0;
63 }
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
```



归并排序

```

87
88
89 // 归并排序
90
91 #include<iostream>
92
93 using namespace std;
94
95 const int N = 10010;
96
97 int n;
98 int arr[N];
99 int tmp[N]; // 临时数组
100
101 void merge_sort(int arr[], int l, int r)
102 {
103
104     if(l >= r) return;
105
106     int mid = (l + r) >> 1; // 相当于除以 2
107     // 这里相较于快排先递归是因为我需要先找到每一小小块（只含 1 个元素），然后按
108     照规律合并各个小数组
109     merge_sort(arr, l, mid);
110     merge_sort(arr, mid + 1, r);
111
112     int k = 0; // temp 临时数组（指向第一个元素的位置，意思是此时 tmp 数组为空的）
113     int i = l; // 左边数组的指针起点是 l
114     int j = mid + 1; // 右边数组的起点是 mid + 1
115     while(i <= mid && j <= r) // 当左边的数组和右边的数组都还有数字时
116     {
117         if(arr[i] <= arr[j]) // 左右指针的数字哪个比较大，那个就赋值给 tmp 临时
118         数组
119         {
120             tmp[k] = arr[i];
121             k++;
122             i++;
123         }
124         else
125         {
126             tmp[k] = arr[j];
127             k++;
128             j++;
129         }

```



加油喵! QwQ !

```
130     }
131
132     while(i <= mid) // 看看左边的数组有没有剩余
133     {
134         tmp[k] = arr[i];
135         k++;
136         i++;
137     }
138     while(j <= r) // 再看看右边的数组有没有剩余 (这里是偷懒就没写判断直接从重复
139 写了, 反正没影响)
140     {
141         tmp[k] = arr[j];
142         k++;
143         j++;
144     }
145
146     for(int i = l, j = 0; i <= r; i++, j++) // 将排序好的 tmp 数组拷贝到原数组中
147     {
148         arr[i] = tmp[j];
149     }
150 }
151
152 int main()
153 {
154     scanf("%d", &n);
155
156     for(int i = 0; i < n; i++)
157     {
158         scanf("%d", &arr[i]);
159     }
160
161     merge_sort(arr, 0, n - 1);
162
163     for(int i = 0; i < n; i++)
164     {
165         printf("%d ", arr[i]);
166     }
167
168     return 0;
169 }
170
171
172
173
```



高精度加法

```

174
175
176 // 高精度加法
177
178 #include<iostream>
179 #include<vector>
180 #include<string>
181
182 using namespace std;
183
184 vector<int> add(vector<int> &A, vector<int> &B)
185 {
186     if(A.size() < B.size()) // 我们要保证让 A 是数位大的数字, B 是数位小的数字
187     {
188         return add(B, A);
189     }
190
191     vector<int> C; // C 储存答案
192     int t = 0;
193     for(int i = 0; i < static_cast<int>(A.size()); i++) // A 是数位大的数字
194     {
195         t += A[i];
196         if(i < static_cast<int>(B.size()))
197         {
198             t += B[i];
199         }
200         C.push_back(t % 10);
201         t /= 10;
202     }
203     if(t != 0)
204     {
205         C.push_back(t);
206     }
207
208     return C;
209 }
210
211 void print(vector<int> &num)
212 {
213     for(int i = static_cast<int>(num.size() - 1); i >= 0; i--) // 因为前面填入的
214     是个位, 所以个位在 num[0] (前面), 我们要从高位到低位打印
215     {
216         cout << num[i];

```



```

217     }
218     cout << endl;
219 }
220
221 int main()
222 {
223     string a, b;
224     cin >> a >> b;
225
226     vector<int> A, B;
227     for(int i = a.size() - 1; i >= 0; i--) // 因为个位在字符串的末尾（最右边），
228 我们需要先读入个位，也就是先读入最右边末尾的字符
229     {
230         A.push_back(a[i] - '0');
231     }
232     for(int i = b.size() - 1; i >= 0; i--)
233     {
234         B.push_back(b[i] - '0');
235     }
236
237     //测试代码
238     // for(int n : A)
239     // {
240     //     cout << n << " ";
241     // }
242     // cout << endl;
243     // for(int n : B)
244     // {
245     //     cout << n << " ";
246     // }
247     // cout << endl;
248
249     vector<int> answer = add(A, B);
250
251     print(answer);
252
253     return 0;
254 }
255
256
257
258
259

```



高精度减法

260

261

262 // 高精度减法

263

264 #include<iostream>

265 #include<vector>

266 #include<string>

267

268 using namespace std;

269

270 vector<int> sub(vector<int> &A, vector<int> &B)

271 {

272 // if(A.size() < B.size()) return sub(B, A); // 确保是数位多的减去数位少的

273

274 vector<int> C;

275 int t = 0; // t 表示的是计算中的退位，初始值设定为 0

276 for(int i = 0; i < static_cast<int>(A.size()); i++)

277 {

278 t = A[i] - t; // 可以理解成 A[i] 先减去 上一次计算中的退位

279 if(i < static_cast<int>(B.size()))

280 {

281 t -= B[i];

282 }

283 C.push_back((t + 10) % 10); // +10 是为了确保填入 C 中的数字是正数

284 if(t < 0) t = 1; // 如果 t 再减去 B[i] 后变成了负数，就将 t 退位成更高一位的 1;

285 else t = 0; // 意味着减去 退位 t 后的 A[i] 再减去 B[i] 也大于等于 0，下次计算中不需要额外的退位，将 退位 t 设为 0。

286 }

287

288 while(C.size() > 1 && C.back() == 0) C.pop_back(); // 需要弹出可能出现的最高位为 0。

289

290 return C;

291 }

292

293 void print(vector<int> &num)

294 {

295 for(int i = static_cast<int>(num.size() - 1); i >= 0; i--)

296 {

297 cout << num[i];

298 }

299 }



加油喵! QwQ !

```
302     cout << endl;
303 }
304
305 int main()
306 {
307     string a, b;
308     cin >> a >> b;
309
310     vector<int> A, B;
311     for(int i = a.size() - 1; i >= 0; i--) A.push_back(a[i] - '0');
312     for(int i = b.size() - 1; i >= 0; i--) B.push_back(b[i] - '0');
313
314     // 测试代码
315     // for(int n : A)
316     // {
317     //     cout << n << " ";
318     // }
319     // cout << endl;
320     // for(int n : B)
321     // {
322     //     cout << n << " ";
323     // }
324     // cout << endl;
325
326     vector<int> answer = sub(A, B);
327
328     print(answer);
329
330     return 0;
331 }
332
333
334
335
336
337
338
339
340
341
342
343
344
```



高精度乘低精度

```

345
346
347 // 高精度乘低精度
348
349 #include<iostream>
350 #include<vector>
351 #include<string>
352
353 using namespace std;
354
355 vector<int> mul(vector<int> &A, int b)
356 {
357     vector<int> C; // C 储存答案
358     int t; // t 储存进位 (通常都会出现大于 10 的情况)
359
360     // 原理是：让 大整数 A 的每一位 (从个位开始) 都乘一次 低精度数 b, 每次提取 t 的
361     各位
362     for(int i = 0; i < static_cast<int>(A.size()) || t; i++)
363     {
364         if(i < static_cast<int>(A.size())) // A 的每一位都乘一次 b
365         {
366             t += A[i] * b;
367         }
368         C.push_back(t % 10);
369         t /= 10;
370     }
371
372     while(C.size() > 1 && C.back() == 0) C.pop_back(); // 删去可能出现在计算结果
373     最高位上的 0 (当乘数为 0 的时候就会在最高位上出现 0 了)。
374
375     return C;
376 }
377
378 void print(vector<int> &num)
379 {
380     for(int i = static_cast<int>(num.size() - 1); i >= 0; i--)
381     {
382         cout << num[i];
383     }
384     cout << endl;
385 }
386

```



加油喵！QwQ！

```
387 int main()
388 {
389     string a;
390     int b;
391     cin >> a >> b;
392
393     vector<int> A;
394     for(int i = a.size() - 1; i >= 0; i--) A.push_back(a[i] - '0');
395
396     vector<int> answer = mul(A, b);
397
398     print(answer);
399
400     return 0;
401 }
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
```



高精度除低精度

```

431
432
433 // 高精除低精
434
435 #include<iostream>
436 #include<vector>
437 #include<string>
438 #include<algorithm>
439
440 using namespace std;
441
442 vector<int> div(vector<int> &A, int b, int &r) // 传入 A 和 r 的地址, 传入 A 的
443 地址是为了节省内存, 传入 r 的地址目的是使 r 的值不用返回也可以被改变
444 {
445     vector<int> C;
446
447     for(int i = A.size() - 1; i >= 0; i--) // 这里和 高精度加/减/乘 反过来, 因为
448 我们要先处理位于 A 数组 末尾的数位更高的数字
449     {
450         r = r * 10 + A[i]; // 先把前一位的余数*10 后, 加上被除数的当前位上的数字,
451 接着将计算出来的值赋给 t
452         C.push_back(r / b);
453         r %= b;
454     }
455     reverse(C.begin(), C.end()); // 这行代码的作用是将存储结果的向量 C 进行反转
456
457     while(C.size() > 1 && C.back() == 0) C.pop_back();
458
459     return C;
460 }
461
462 void print(vector<int> &num)
463 {
464     for(int i = static_cast<int>(num.size() - 1); i >= 0; i--) // 此时 num 数组 对
465 应的数字使 倒序 的
466     {
467         cout << num[i];
468     }
469     cout << endl;
470 }
471
472 int main()

```



```
473 {
474     string a;
475     int b;
476     cin >> a >> b;
477
478     vector<int> A;
479     for(int i = a.size() - 1; i >= 0; i--)
480     {
481         A.push_back(a[i] - '0');
482     }
483
484     int r = 0; // 定义一个余数
485     vector<int> answer = div(A, b, r);
486
487     print(answer);
488
489     cout << "余数是：" << r << endl;
490
491     return 0;
492 }
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
```



前缀和

516

517

518 // 前缀和

519

520 // 算法原理假设有一个序列 $a_1, a_2, a_3, \dots, a_n$ 521 // 现在对这个序列定义一个前缀和序列 $s_i = a_1 + a_2 + \dots + a_i$

522

523 // 如何求 s_i 前缀和的计算方式如下524 // $s[0] = 0$ // 边界值 `for(int i=1; i<=n; i++) s[i] = s[i-1] + a[i]` // 注意前缀和的下标必须

525 从 1 开始

526

527 // 前缀和的作用

528 // 前缀和可以快速求出序列中某一段序列的和例如我们需要计算 $a_1 + a_2 + \dots + a_r$ 的和529 // 如果使用传统方法扫描一遍时间复杂度是 $O(n)$ 530 // 但如果使用前缀和我们可以通过下面的公式计算 $a_1 + a_2 + \dots + a_r = s_r - s_{l-1}$ 531 // 其中 $s_r = a_1 + a_2 + \dots + a_r$ $s_{l-1} = a_1 + a_2 + \dots + a_{l-1}$ 532 // 通过前缀和公式计算某一段序列的和的时间复杂度仅仅为 $O(1)$

533

534 // 题目来源: AcWing 795. 前缀和

535

536 // 一、题目描述

537 // 输入一个长度为 n 的整数序列。538 // 接下来再输入 m 个询问, 每个询问输入一对 l, r 。539 // 对于每个询问, 输出原序列中从第 l 个数到第 r 个数的和。

540

541 // 输入格式

542 // 第一行包含两个整数 n 和 m 。543 // 第二行包含 n 个整数, 表示整数数列。544 // 接下来 m 行, 每行包含两个整数 l 和 r , 表示一个询问的区间范围。

545

546 // 输出格式

547 // 共 m 行, 每行输出一个询问的结果。

548

549 // 数据范围

550 // $1 \leq l \leq r \leq n$,551 // $1 \leq n, m \leq 100000$,552 // $-1000 \leq \text{数列中元素的值} \leq 1000$

553

554 // 输入样例:

555 // 5 3

556 // 2 1 3 6 4

557 // 1 2



```

558 // 1 3
559 // 2 4
560
561 // 输出样例 :
562 // 3
563 // 6
564 // 10
565
566 #include <iostream> // 包含输入输出流库
567
568 using namespace std; // 使用标准命名空间, 避免频繁使用 std::
569
570 const int N = 100010; // 定义前缀和数组的大小, 足够容纳较大的输入数据
571
572 int s[N], n, m; // 前缀和数组 s, 以及输入的数组大小 n 和查询次数 m
573
574 int main()
575 {
576     scanf("%d%d", &n, &m); // 读取输入的数组大小 n 和查询次数 m
577
578     // 计算前缀和数组
579     s[0] = 0;
580     for (int i = 1; i <= n; i++)
581     {
582         scanf("%d", &s[i]); // 读取数组元素并存入 s[i]
583         s[i] += s[i - 1]; // 计算前缀和: s[i] = s[i-1] + a[i]
584         // 此处假设输入的数组元素是 a[i], 通过前缀和数组 s 存储累积和
585     }
586
587     // 处理 m 次查询
588     while (m--)
589     {
590         int l, r; // 定义查询区间的左端点 l 和右端点 r
591         scanf("%d%d", &l, &r); // 读取查询的左右端点
592         printf("%d\n", s[r] - s[l - 1]); // 计算区间 [l, r] 的和并输出
593         // 前缀和的性质: s[r] - s[l-1] 表示从索引 l 到 r 的和
594     }
595
596     return 0; // 程序结束, 返回 0 表示正常退出
597 }
598
599
600

```



二维前缀和

601

602

603 // 二维前缀和

604

605 // $S[i, j]$ = 第 i 行 j 列格子左上部分所有元素的和606 // 以 $(x1, y1)$ 为左上角, $(x2, y2)$ 为右下角的子矩阵的和为:607 // $S[x2, y2] - S[x1 - 1, y2] - S[x2, y1 - 1] + S[x1 - 1, y1 - 1]$

608

609 // 本题题目:

610 // 输入一个 n 行 m 列的整数矩阵, 在输入 q 个询问, 每个询问包含 4 个整数, $x1, y1, x2,$ 611 $y2$, 表示一个子矩阵的左上角坐标和右下角坐标。

612 // 对于每个询问输出矩阵中所有数的和。

613

614 // 二维前缀和是一种高效计算矩阵中子矩阵和的方法, 通过预处理前缀和数组, 可以在 $O(1)$
615 的时间复杂度内回答任意子矩阵的和。

616

617 // **概念**:

618 // $S[i][j]$ 表示矩阵中从 $(1, 1)$ 到 (i, j) 这个矩形区域内的所有元素的和。619 // 例如, 矩阵中的某个位置 (i, j) , 其前缀和 $S[i][j]$ 等于它上方所有行、左边所有列以
620 及左上方所有元素的和。

621

622 // **求子矩阵和的公式**:

623 // 假设我们有一个子矩阵, 左上角是 $(x1, y1)$, 右下角是 $(x2, y2)$ 。这个子矩阵的和可以
624 通过前缀和数组计算:625 // $S[x2][y2] - S[x1 - 1][y2] - S[x2][y1 - 1] + S[x1 - 1][y1 - 1]$

626 // 这个公式的作用是从整个大矩形中减去多余的上下左右部分, 只剩下目标子矩阵的和。

627

628 // **题目要求**:

629 // 输入一个整数矩阵, 然后处理多个查询。每个查询给出一个子矩阵的左上角和右下角,
630 要求输出该子矩阵的和。

631

632 // AcWing 796. 子矩阵的和

633 // 1、题目 (来源于 AcWing):

634 // 输入一个 n 行 m 列的整数矩阵, 再输入 q 个询问, 每个询问包含四个整数 $x1, y1, x2, y2$,
635 表示一个子矩阵的左上角坐标和右下角坐标。

636 // 对于每个询问输出子矩阵中所有数的和。

637

638 // 输入格式

639 // 第一行包含三个整数 n, m, q 。640 // 接下来 n 行, 每行包含 m 个整数, 表示整数矩阵。641 // 接下来 q 行, 每行包含四个整数 $x1, y1, x2, y2$, 表示一组询问。

642



```

643 // 输出格式
644 // 共 q 行, 每行输出一个询问的结果。
645
646 // 数据范围
647 // 1   n, m   1000,
648 // 1   q   200000,
649 // 1   x1   x2   n,
650 // 1   y1   y2   m,
651 // -1000   矩阵内元素的值   1000
652
653 // 输入样例 :
654 // 3 4 3
655 // 1 7 2 4
656 // 3 6 2 8
657 // 2 1 2 3
658 // 1 1 2 2
659 // 2 1 3 4
660 // 1 3 3 4
661 // 输出样例 :
662 // 17
663 // 27
664 // 21
665
666 #include <iostream>
667 using namespace std;
668
669 const int N = 1010; // 定义数组的最大大小为 1010x1010 (考虑到题目中的输入可能不
670 会超过这个范围)
671
672 int n, m, q; // 矩阵的行数、列数, 以及查询的次数
673 int a[N][N]; // 原始矩阵数组
674 int s[N][N]; // 前缀和数组
675
676 int main()
677 {
678     // 读取输入 : 矩阵的行数、列数和查询次数
679     scanf("%d%d%d", &n, &m, &q);
680
681     // 输入矩阵中的元素
682     for (int i = 1; i <= n; i++)
683     {
684         for (int j = 1; j <= m; j++)
685         {
686             scanf("%d", &a[i][j]); // 读取矩阵的每个元素, 注意数组从 1 开始索引,

```



```

687 方便后续处理边界
688     }
689 }
690
691 // 计算二维前缀和数组 s
692 for (int i = 1; i <= n; i++)
693 {
694     for (int j = 1; j <= m; j++)
695     {
696         // 根据二维前缀和的递推公式计算 s[i][j]
697         s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + a[i][j];
698         // 该公式考虑了相邻行和列的和, 减去重复计算的左上角区域
699     }
700 }
701
702 /*
703 // 可选: 打印前缀和数组 (用于调试)
704 for (int i = 1; i <= n; i++)
705 {
706     for (int j = 1; j <= m; j++)
707     {
708         cout << s[i][j] << " ";
709     }
710     cout << endl;
711 }
712 */
713
714 // 处理每个查询
715 while (q--)
716 {
717     int x1, y1, x2, y2;
718     scanf("%d%d%d%d", &x1, &y1, &x2, &y2); // 读取子矩阵的左上角(x1, y1)和右
719     下角(x2, y2)
720
721     // 根据二维前缀和公式计算子矩阵的和
722     int ans = s[x2][y2] - s[x1 - 1][y2] - s[x2][y1 - 1] + s[x1 - 1][y1 - 1];
723     printf("%d\n", ans); // 输出结果
724 }
725
726 return 0;
727 }
728
729 // **关键点解释**:
730 // 1. **二维前缀和数组的构建**:

```



加油喵! QwQ !

```
731 //  通过将当前元素与上方、左方的元素累加，并减去重复计算的左上角区域，得到当前
732 元素的前缀和。
733 //
734 // 2. **子矩阵和的计算**：
735 //  利用容斥原理，从大矩形中减去多余的部分，得到目标子矩阵的和。
736 //
737 // 3. **边界处理**：
738 //  数组从 1 开始索引，方便处理边界情况（比如  $x1 - 1$  为 0 时，对应的区域视为 0）。
739
740 // 这种方法的时间复杂度为  $O(nm)$  预处理， $O(1)$  查询，非常适合处理大量查询的情况。
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
```



差分

```

774
775
776 // 差分
777
778 // 作用：可以使数组的某一个范围都加上一个数
779
780 // ps：平时使用的时候只需要考虑如何去更新的可以了
781 // ps：涉及到差分和前缀和，一定要想到 左边界为有一个虚拟起点 0！真正的起点是 1。
782
783 // AcWing.797 差分
784 // 输入一个长度为 n 的整数序列。
785 // 接下来输入 m 个操作，每个操作包含三个整数 l, r, c, 表示将序列中 [l, r] 之间的每个
786 数加上 c。
787 // 请你输出进行完所有操作后的序列。
788
789 // 输入格式
790 // 第一行包含两个整数 n 和 m。
791 // 第二行包含 n 个整数，表示整数序列。
792 // 接下来 m 行，每行包含三个整数 l, r, c 表示一个操作。
793
794 // 输出格式
795 // 共一行，包含 n 个整数，表示最终序列。
796
797 // 数据范围
798 // 1 ≤ n, m ≤ 1000001
799 // 1 ≤ l ≤ r ≤ n
800 // -1000 ≤ c ≤ 1000
801 // -1000 ≤ 整数序列中元素的值 ≤ 1000
802
803 // 输入样例：
804 // 6 3
805 // 1 2 2 1 2 1
806 // 1 3 1
807 // 3 5 1
808 // 1 6 1
809 // 输出样例：
810 // 3 4 5 3 4 2
811
812 // 解题思路：
813 // 给定原数组 a[1], a[2], ..., a[N], 构造差分数组 b[N], 使得 a[i] = b[1] + b[2] + ...
814 + b[i], 一般假定初始全部为 0, 用 insert(i, i, a[i]) 构造出 b[N]
815

```



```

816 // 核心操作: 将 a[L~R] 全部加上 c, 等价于: b[L] += c, b[R+1] -= c
817
818 #include <iostream>
819 using namespace std;
820
821 // 定义数组的最大长度
822 const int N = 100010;
823
824 // n 表示数组的长度, m 表示操作的次数
825 int n, m;
826 // a 数组用于存储原始的整数序列
827 // b 数组用于存储 a 数组的差分数组
828 int a[N], b[N];
829
830 // 插入函数, 用于对差分数组进行修改
831 // 这个函数的作用是将原数组中从索引 l 到索引 r 的元素都加上 c
832 // 原理是通过修改差分数组的两个端点值来实现对原数组指定区间的修改
833 void insert(int l, int r, int c)
834 {
835     // 将差分数组中索引为 l 的元素加上 c
836     // 这一步意味着从原数组的第 l 个元素开始, 后面的元素都会受到加上 c 的影响
837     b[l] += c;
838     // 将差分数组中索引为 r + 1 的元素减去 c
839     // 这一步是为了限制加上 c 的影响范围, 使得从原数组的第 r + 1 个元素开始不受加
840     上 c 的影响
841     b[r + 1] -= c;
842 }
843
844 int main()
845 {
846     // 从标准输入读取数组的长度 n 和操作的次数 m
847     scanf("%d%d", &n, &m);
848
849     // 输入原始的整数序列, 存储到 a 数组中
850     for (int i = 1; i <= n; i++)
851     {
852         // 从标准输入读取每个元素的值
853         scanf("%d", &a[i]);
854     }
855
856     // 得到原数组的差分数组
857     // 对于原数组的每个元素 a[i], 可以看作是在区间 [i, i] 上插入 a[i]
858     // 这样就可以通过 insert 函数来构建差分数组
859     for (int i = 1; i <= n; i++)

```



```

860     {
861         insert(i, i, a[i]);
862     }
863
864     // 通过修改差分数组特定范围的左右端点的值, 来达到改变原数组的目的
865     // 循环 m 次, 处理 m 个操作
866     while (m--)
867     {
868         // 定义三个变量, 用于存储每次操作的左端点 l、右端点 r 和要加上的值 c
869         int l, r, c;
870         // 从标准输入读取每次操作的具体参数
871         scanf("%d%d%d", &l, &r, &c);
872         // 调用 insert 函数, 对差分数组进行修改, 从而间接修改原数组
873         insert(l, r, c);
874     }
875
876     // 把 b 数组变成自己的前缀和
877     // 因为差分和前缀和是互逆的操作, 对差分数组求前缀和就可以得到经过修改后的原
878 数组
879     for (int i = 1; i <= n; i++)
880     {
881         // 通过累加前面的元素, 将差分数组转换为前缀和数组
882         b[i] += b[i - 1];
883     }
884
885     // 打印输出经过所有操作后的数组
886     for (int i = 1; i <= n; i++)
887     {
888         // 依次输出数组中的每个元素, 元素之间用空格分隔
889         printf("%d ", b[i]);
890     }
891
892     return 0;
893 }
894
895
896
897
898
899
900
901
902

```



差分数组 (二维)

903

904

905 // 差分矩阵 2 (二维)

906

907 // 作用: 可以使矩阵的某一个子矩阵范围都加上一个数

908 // ps: 平时使用的时候只需要考虑如何去更新的可以了

909

910 // AcWing. 798 差分矩阵

911

912 // 题目描述:

913 // 输入一个 n 行 m 列的整数矩阵, 再输入 q 个操作, 每个操作包含五个整数 $x1, y1, x2, y2,$ 914 $c,$ 915 // 其中 $(x1, y1)$ 和 $(x2, y2)$ 表示一个子矩阵的左上角坐标和右下角坐标。916 // 每个操作都要将选中的子矩阵中的每个元素的值加上 c 。

917 // 请你将进行完所有操作后的矩阵输出。

918

919 // 输入格式

920 // 第一行包含整数 n, m, q 。921 // 接下来 n 行, 每行包含 m 个整数, 表示整数矩阵。922 // 接下来 q 行, 每行包含 5 个整数 $x1, y1, x2, y2, c$, 表示一个操作。

923

924 // 输出格式

925 // 共 n 行, 每行 m 个整数, 表示所有操作进行完毕后的最终矩阵。

926

927 // 数据范围

928 // 1 n, m 1000929 // 1 q 100000930 // 1 $x1$ $x2$ n 931 // 1 $y1$ $y2$ m 932 // -1000 c 1000

933 // -1000 矩阵内元素的值 1000

934

935 // 输入样例:

936 // 3 4 3

937 // 1 2 2 1

938 // 3 2 2 1

939 // 1 1 1 1

940 // 1 1 2 2 1

941 // 1 3 2 3 2

942 // 3 1 3 4 1

943 // 输出样例:

944 // 2 3 4 1




```

945 // 4 3 4 1
946 // 2 2 2 2
947
948 #include <iostream>
949 using namespace std;
950
951 // 定义数组的最大长度, 这里假设矩阵的行数和列数最大为 1010
952 const int N = 1010;
953
954 // n 表示矩阵的行数, m 表示矩阵的列数, q 表示操作的次数
955 int n, m, q;
956 // a 数组用于存储原始的二维矩阵
957 // b 数组用于存储 a 数组的二维差分数组
958 int a[N][N], b[N][N];
959
960 // 插入函数, 用于对二维差分数组进行修改
961 // 这个函数的作用是将原矩阵中以 (x1, y1) 为左上角, (x2, y2) 为右下角的子矩阵中的
962 每个元素都加上 c
963 // 原理是通过修改差分数组的四个关键位置的值来实现对原矩阵指定子矩阵的修改
964 void insert(int x1, int y1, int x2, int y2, int c)
965 {
966     // 将差分数组中 (x1, y1) 位置的元素加上 c
967     // 这一步意味着从原矩阵的 (x1, y1) 位置开始, 后面的元素都会受到加上 c 的影响
968     b[x1][y1] += c;
969     // 将差分数组中 (x2 + 1, y1) 位置的元素减去 c
970     // 这一步是为了限制加上 c 的影响范围, 使得从原矩阵的第 x2 + 1 行开始, 第 y1 列
971 及以后的元素不受加上 c 的影响
972     b[x2 + 1][y1] -= c;
973     // 将差分数组中 (x1, y2 + 1) 位置的元素减去 c
974     // 这一步是为了限制加上 c 的影响范围, 使得从原矩阵的第 x1 行开始, 第 y2 + 1 列
975 及以后的元素不受加上 c 的影响
976     b[x1][y2 + 1] -= c;
977     // 将差分数组中 (x2 + 1, y2 + 1) 位置的元素加上 c
978     // 这一步是为了修正前面两次减法多减去的部分, 确保影响范围正确
979     b[x2 + 1][y2 + 1] += c;
980 }
981
982 int main()
983 {
984     // 从标准输入读取矩阵的行数 n、列数 m 和操作的次数 q
985     scanf("%d%d%d", &n, &m, &q);
986
987     // 输入原始的二维矩阵, 存储到 a 数组中
988     for (int i = 1; i <= n; i++)

```



```

989     {
990         for (int j = 1; j <= m; j++)
991         {
992             // 从标准输入读取矩阵中每个元素的值
993             scanf("%d", &a[i][j]);
994         }
995     }
996
997     // 得到原矩阵的二维差分数组
998     // 对于原矩阵的每个元素 a[i][j], 可以看作是在以 (i, j) 为左上角和右下角的子
999 矩阵上插入 a[i][j]
1000    // 这样就可以通过 insert 函数来构建二维差分数组
1001    for (int i = 1; i <= n; i++)
1002    {
1003        for (int j = 1; j <= m; j++)
1004        {
1005            insert(i, j, i, j, a[i][j]);
1006        }
1007    }
1008
1009    // 循环 q 次, 处理 q 个操作
1010    while (q--)
1011    {
1012        // 定义五个变量, 用于存储每次操作的左上角坐标 (x1, y1)、右下角坐标 (x2, y2)
1013        和要加上的值 c
1014        int x1, y1, x2, y2, c;
1015        // 从标准输入读取每次操作的具体参数
1016        cin >> x1 >> y1 >> x2 >> y2 >> c;
1017        // 调用 insert 函数, 对二维差分数组进行修改, 从而间接修改原矩阵
1018        insert(x1, y1, x2, y2, c);
1019    }
1020
1021    // 对二维差分数组求前缀和, 得到经过修改后的原矩阵
1022    // 因为差分和前缀和是互逆的操作, 对二维差分数组求前缀和就可以得到经过修改后
1023    的原矩阵
1024    for (int i = 1; i <= n; i++)
1025    {
1026        for (int j = 1; j <= m; j++)
1027        {
1028            // 通过容斥原理计算前缀和
1029            // b[i - 1][j] 表示上方子矩阵的和
1030            // b[i][j - 1] 表示左方子矩阵的和
1031            // b[i - 1][j - 1] 表示左上角子矩阵的和, 由于前面两个和都包含了它,
1032            所以要减去一次

```



加油喵! QwQ !

```
1033         b[i][j] += b[i - 1][j] + b[i][j - 1] - b[i - 1][j - 1];
1034     }
1035 }
1036
1037 // 打印输出经过所有操作后的矩阵
1038 for (int i = 1; i <= n; i++)
1039 {
1040     for (int j = 1; j <= m; j++)
1041     {
1042         // 依次输出矩阵中每个元素的值，元素之间用空格分隔
1043         printf("%d ", b[i][j]);
1044     }
1045     // 每输出完一行元素，换行
1046     puts("");
1047 }
1048
1049 return 0;
1050 }
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
```



双指针

1076

1077

1078 // 双指针模板

1079

1080 // for (int i = 0, j = 0; i < n; i ++)

1081 // {

1082 // while (j < i && check(i, j)) j ++ ;

1083

1084 // // 具体问题的逻辑

1085 // }

1086 // 常见问题分类：

1087 // (1) 对于一个序列，用两个指针维护一段区间

1088 // (2) 对于两个序列，维护某种次序，比如归并排序中合并两个有序序列的操作

1089

1090

1091 // AcWing 799. 最长连续不重复子序列

1092

1093 // 题目（来源于 AcWing）：

1094 // 给定一个长度为 n 的整数序列，请找出最长的不包含重复数字的连续区间，输出它的长
1095 度。

1096

1097 // 输入格式

1098 // 第一行包含整数 n 。1099 // 第二行包含 n 个整数（均在 $0 \sim 100000$ 范围内），表示整数序列。

1100

1101 // 输出格式

1102 // 共一行，包含一个整数，表示最长的不包含重复数字的连续子序列的长度。

1103

1104 // 数据范围

1105 // $1 \leq n \leq 100000$

1106

1107 // 输入样例：

1108 // 5

1109 // 1 2 2 3 5

1110 // 输出样例：

1111 // 3

1112

1113 // 2、基本思想：

1114 // 用双指针 i 和 j 。 i 从左向右遍历给定序列中的每一个数并表示满足条件子序列的右
1115 端点， j 在子序列无法满足条件时从左向右移动直到子序列中不再有重复数字并表示满足条
1116 件的子序列的左端点。与暴力算法相比将时间复杂度从 $O(n^2)$ 优化到了 $O(n)$ 。

1117 #include<iostream>



```

1118
1119 using namespace std;
1120
1121 // 定义常量 N 为 100010, 作为数组的最大长度
1122 const int N = 1e5 + 10;
1123 // 数组 a 用于存储输入的整数序列
1124 int a[N];
1125 // 数组 cnt 用于记录每个整数出现的次数
1126 int cnt[N];
1127 // 变量 n 表示输入序列的长度
1128 int n;
1129
1130 int main()
1131 {
1132     scanf("%d", &n);
1133     for(int i = 0; i < n; i++)
1134     {
1135         scanf("%d", &a[i]);
1136     }
1137
1138     int res = 0; // res 记录的是到当前为止的最长连续不重复子序列的长度, 初始化为
1139     0
1140     for(int i = 0, j = 0; j < n; j++) // i 是左指针, j 是右指针
1141     {
1142         cnt[a[j]]++; // 将 a[j] 这个数字出现的次数+1
1143         while(cnt[a[j]] > 1) // 如果 a[j] 这个数字出现的次数 >1 的话, 就移动左指
1144         针直到出现的次数 <1
1145         {
1146
1147             cnt[a[i]]--; // 左指针指向的这个数字出现的次数 -1
1148             i++; // 左指针向右移动一位
1149         }
1150
1151         // 计算当前不重复子序列的长度 (j - i + 1)
1152         res = max(res, j - i + 1); // 并更新 res 为当前长度和之前记录的最大长度中
1153         的较大值
1154     }
1155
1156     printf("%d\n", res);
1157
1158     return 0;
1159 }
1160

```



位运算

```

1161
1162
1163 // 位运算
1164
1165 //求 n 的第 k 位数字：n >> k & 1
1166
1167 #include<iostream>
1168 using namespace std;
1169
1170 //例：n 的二进制表中第 k 位是几 ps：最右边是第一位
1171 int main()
1172 {
1173     int n = 10;
1174     // cin >> n;
1175     for(int k = 3; k >= 0; k--) cout << (n >> k & 1);
1176
1177
1178     return 0;
1179 }
1180
1181 // 解释：
1182
1183 // 在这段代码中，以整数 10（二进制表示为 1010）为例，当 k 从 3 依次递减到 0 进行
1184 循环时，对应的操作及输出结果如下：
1185 //当 k = 3 时：
1186 // n >> 3：将 10 的二进制 1010 右移 3 位，得到 0001。
1187 // 0001 & 1（二进制 0001）：按位与操作后结果为 1，输出 1。
1188
1189 //当 k = 2 时：
1190 // n >> 2：将 10 的二进制 1010 右移 2 位，得到 0010。
1191 // 0010 & 1（二进制 0001）：按位与操作后结果为 0，输出 0。
1192
1193 //当 k = 1 时：
1194 // n >> 1：将 10 的二进制 1010 右移 1 位，得到 0101。
1195 // 0101 & 1（二进制 0001）：按位与操作后结果为 1，输出 1。
1196
1197 //当 k = 0 时：
1198 // n >> 0：相当于不移动，还是 1010。
1199 // 1010 & 1（二进制 0001）：按位与操作后结果为 0，输出 0。
1200
1201
1202

```



```

1203
1204 //位运算
1205
1206 // 返回 n 的最后一位 1: lowbit(x) = x & -x;
1207
1208 //例题：
1209 //返回一个数字二进制中有多少个 1
1210 #include<iostream>
1211 using namespace std;
1212
1213 int lowbit(int x)
1214 {
1215     return x & -x;
1216 }
1217
1218 int main()
1219 {
1220     int n; // n 组测试数据
1221     cin >> n;
1222     while(n--)
1223     {
1224         int x;
1225         cin >> x;
1226
1227         int res = 0;
1228         while(x) x -= lowbit(x), res++; // 每次减去 x 的最后一位，每减一次 res（计
1229 数器）就加 1
1230
1231         cout << res << endl;
1232     }
1233
1234     return 0;
1235 }
1236
1237
1238
1239
1240 // 基本思想：
1241
1242 // 我们知道计算机中的数在内存中都是以二进制形式进行存储的，而位运算就是
1243 直接对整数在内存中的二进制位进行操作，因此其执行效率非常高，在程序中使用位运算进
1244 行操作，会大大提高程序的性能。在有些 dp 等问题中，灵活运用数的二进制来表示某种状
1245 态可以大大降低代码的复杂度，位运算很多时候可以将我们的问题简单化，但是并不容易想
1246 到，需要一定的积累和对位运算知识的熟练掌握。

```



```

1247
1248 //          lowbit 原理：根据计算机负数表示的特点，如一个数字原码是 011111000，他
1249 的负数表示形式是补码，也就是反码+1，反码是 10000111，加一则是 10001000，二者按位
1250 与得到了 1000，就是我们想要的 lowbit 操作，即得到最后一位 1
1251
1252 C++ 位运算简介
1253
1254 一、常用位运算
1255
1256 1. 按位与  $a \& b$ 
1257 法则：两者相同位都为 1，则结果中该位为 1；否则结果中该位为 0
1258 例：
1259  $12 \& 6 = 4$ 
1260  $12: 1\ 1\ 0\ 0$ 
1261  $6: 0\ 1\ 1\ 0$ 
1262  $4: 0\ 1\ 0\ 0$ 
1263 // 常用于判断某一位是否为 1 或者清零某些位
1264
1265 2. 按位或  $a \mid b$ 
1266 法则：两者相同位中有一个为 1，则结果中该位为 1；否则结果中该位为 0
1267 例：
1268  $12 \mid 6 = 14$ 
1269  $12: 1\ 1\ 0\ 0$ 
1270  $6: 0\ 1\ 1\ 0$ 
1271  $14: 1\ 1\ 1\ 0$ 
1272 // 常用于设置某一位为 1
1273
1274 3. 按位异或  $a \wedge b$ 
1275 法则：两者相同位的值若不同，则结果中该位为 1；否则结果中该位为 0
1276 例：
1277  $12 \wedge 6 = 10$ 
1278  $12: 1\ 1\ 0\ 0$ 
1279  $6: 0\ 1\ 1\ 0$ 
1280  $10: 1\ 0\ 1\ 0$ 
1281 // 常用于交换两个变量的值或者求两个数的差异
1282
1283 4. 按位取反  $\sim a$ 
1284 法则：该数中 0 的位置变为 1；1 的位置变为 0（涉及二进制补码，先不例）
1285 // 常用于获取一个数的补码，注意负数的补码表示
1286
1287 5. 按位左移  $a \ll b$ 
1288 法则：将该数  $a$  左移  $b$  位，正数左移后为正数，负数左移后为负数
1289 例：
1290  $3 \ll 2 = 12$ 

```




```

1291 3:0 0 1 1
1292 12:1 1 0 0
1293  $a \ll b = a \times 2^b$ 
1294 // 常用于快速乘以 2 的幂次方
1295
1296 6. 按位右移  $a \gg b$ 
1297 法则: 将该数  $a$  右移  $b$  位, 正数右移后为正数, 负数右移后为负数。不论正负, 都下取整
1298 例:
1299  $13 \gg 2 = 3$ 
1300 13:1 1 0 1
1301 3:0 0 1 1
1302  $a \gg b = a / 2^b$ 
1303 // 常用于快速除以 2 的幂次方
1304
1305 7. 非  $!a$ 
1306 法则: 该数是 0 则为 1; 否则为 0
1307 例:
1308  $!0 = 1$ 
1309  $!1 = 0$ 
1310  $!2 = 0$ 
1311 // 常用于逻辑判断
1312
1313 二、运算符优先级
1314  $-, !, \sim >$  // 一元运算符优先级最高
1315  $*, /, \% >$  // 乘除运算优先级次高
1316  $\gg, \ll >$  // 位移运算优先级
1317  $>, <, >=, <= >$  // 关系运算优先级
1318  $==, != >$  // 等值运算优先级
1319  $\& >$  // 按位与运算优先级
1320  $\wedge >$  // 按位异或运算优先级
1321  $| >$  // 按位或运算优先级
1322  $\&\& >$  // 逻辑与运算优先级
1323  $|| >$  // 逻辑或运算优先级
1324 问号表达式  $>$  // 三元运算符优先级
1325 赋值语句 // 赋值运算优先级最低
1326
1327 三、负数表示
1328 用补码表示负数:
1329  $-1 = 0 - 1$ 
1330  $0 = 000...00$ 
1331  $1 = 000...01$ 
1332 所以  $-1 = 111...11$ 
1333 同理:
1334  $-2 = 111...110$ 

```



```

1335 -3 = 111...101
1336 -4 = 111...100
1337 ...
1338  $-x = \sim x + 1$ 
1339 // 负数在计算机内部以补码形式存储, 补码制能够方便地进行加减运算

```

```

1340

```

1341 四、八进制与十六进制

```

1342

```

1343 1. 八进制数

```

1344 每一位用 0~7 表示, 每一位占二进制中的 3 位
1345 在 C++ 中, 前面加 0 表示八进制数, 例:
1346 printf("%d\n", 015); // 输出: 13
1347 // 八进制数方便与二进制数相互转换, 适用于某些特定的系统设置或权限管理

```

```

1348

```

1349 2. 十六进制数

```

1350 每一位用 0~f 表示 (a~f 分别表示 10~15), 每一位占二进制中的 4 位
1351 在 C++ 中, 前面加 0x 表示十六进制数, 例:
1352 printf("%d\n", 0xd); // 输出: 13
1353 // 十六进制数在编程中广泛用于颜色表示、内存地址等

```

```

1354

```

1355 五、字节

```

1356 一个字节为 8 位二进制数, 也是 2 位十六进制数, 即 00000000~11111111
1357 一个 int 占 4 个字节, 即 32 位二进制数
1358 一个 long long 占 8 个字节, 即 64 位二进制数
1359 memset 按字节赋值, 故当 f 为 int 型数组时, memset(f, 0x3f, sizeof f) 是将 f 赋
1360 值为 0x3f3f3f3f
1361 // 字节是计算机中数据存储的基本单位, 不同的数据类型占用的字节数不同

```

```

1362

```

1363 六、位运算常用技巧 (多用于状压 DP)

```

1364

```

1365 1. 将 x 第 i 位取反: $x \oplus= 1 \ll i$

```

1366 // 通过异或操作实现特定位的取反

```

```

1367

```

1368 2. 将 x 第 i 位制成 1: $x \mid= 1 \ll i$

```

1369 // 通过或操作实现特定位的设置

```

```

1370

```

1371 3. 将 x 第 i 位制成 0: $x \&= -1 \wedge (1 \ll i)$ 或 $x \&= \sim(1 \ll i)$

```

1372 // 通过与操作和取反操作实现特定位的清零

```

```

1373

```

1374 4. 取 x 对 2 取模的结果: $x \& 1$

```

1375 // 利用二进制最低位判断奇偶性

```

```

1376

```

1377 5. 取 x 的第 i 位是否为 1: $x \& (1 \ll i)$ 或 $(x \gg i) \& 1$

```

1378 // 通过位移和与操作判断特定位的状态

```



```

1379
1380 6. 取 x 的最后一位:  $x \& -x$ 
1381 // 利用负数的补码特性获取最低位的 1
1382
1383 7. 取 x 的绝对值:  $(x \wedge (x >> 31)) - (x >> 31)$  (int 型)
1384 // 通过位运算实现绝对值的快速计算
1385
1386 8. 判断 x 是否不为 2 的整次方幂:  $x \& (x - 1)$ 
1387 // 利用二进制中 2 的幂次方数的特性
1388
1389 9. 判断 a 是否不等于 b:  $a \neq b$ 、 $a - b$ 、 $a \wedge b$ 
1390 // 位运算实现简单的比较操作
1391
1392 10. 判断 x 是否不等于 -1:  $x \neq -1$ 、 $x \wedge -1$ 、 $x + 1$ 、 $\sim x$ 
1393 // 多种方式判断数值是否为 -1
1394
1395 七、应用
1396
1397 1. 枚举子集
1398 枚举一个集合 S 的子集  $0(2^{|S|})$  ( $|S|$  表示 S 中的元素个数)
1399 // S 是一个二进制数, 表示一个集合, i 枚举 S 的所有子集
1400 for (int i = S; i; i = S & (i - 1))
1401     // blablabla
1402 // 通过位运算高效地枚举子集, 适用于组合问题
1403
1404 枚举集合  $0, 1, \dots, n-1$  的所有子集的子集  $0(3^n)$ 
1405 for (int S = 0; S < (1 << n); S++) // 枚举集合  $\{0, \dots, n-1\}$  的所有子集
1406     for (int i = S; i; i = S & (i - 1)) // 枚举子集 S 的子集
1407         // blablabla
1408 // 更高层次的子集枚举, 适用于复杂的组合问题
1409
1410 2. 交换两个数字
1411 void swap(int &a, int &b)
1412 {
1413     a ^= b;
1414     b ^= a;
1415     a ^= b;
1416 }
1417 // 利用异或操作实现无中间变量的交换
1418
1419 3. 求二进制中数字 1 的个数
1420
1421 (1) 暴力  $O(\log x)$ 
1422 int count(int x)

```



```

1423 {
1424     int res = 0;
1425     while (x) res += x & 1, x >>= 1;
1426     return res;
1427 }
1428 // 逐位检查, 适用于简单的场景
1429
1430 (2) 针对 1 较稀疏的情况的优化  $O(\log x)$ 
1431 int count(int x)
1432 {
1433     int res = 0;
1434     while (x) x &= x - 1, res ++ ;
1435     return res;
1436 }
1437 // 利用  $x \& (x-1)$  快速消除最低位的 1, 适用于 1 较稀疏的情况
1438
1439 (3) 位运算  $O(1)$ 
1440 int count(int x)
1441 {
1442     x = (x >> 1 & 0x55555555) + (x & 0x55555555);
1443     x = (x >> 2 & 0x33333333) + (x & 0x33333333);
1444     x = (x >> 4 & 0x0f0f0f0f) + (x & 0x0f0f0f0f);
1445     return x % 255;
1446 }
1447 // 利用位运算并行统计每一位的 1, 实现常数时间的统计
1448
1449 4. 求  $\log_2 x$ 
1450
1451 (1) 暴力  $O(\log x)$ 
1452 int log_2(int x)
1453 {
1454     int res = 0;
1455     while (x >> 1) res ++ , x >>= 1;
1456     return res;
1457 }
1458 // 通过不断右移计算最高位的位置
1459
1460 (2) 二分  $O(\log \log x)$ 
1461 int log_2(int x)
1462 {
1463     int l = 0, r = 31, mid;
1464     while (l < r)
1465     {
1466         mid = l + r + 1 >> 1;

```



```

1467         if (x >> mid) l = mid;
1468         else     r = mid - 1;
1469     }
1470     return r;
1471 }
1472 // 利用二分法快速定位最高位
1473
1474 (3) 位运算 (二分展开) O(1)
1475 int log_2(int x)
1476 {
1477     int res = 0;
1478     if (x & 0xffff0000) res += 16, x >>= 16;
1479     if (x & 0xff00) res += 8, x >>= 8;
1480     if (x & 0xf0) res += 4, x >>= 4;
1481     if (x & 0xc) res += 2, x >>= 2;
1482     if (x & 2) res ++ ;
1483     return res;
1484 }
1485 // 利用位运算分段查找, 实现常数时间的计算
1486
1487 (4) 预处理 1~n 中所有数 log 后的结果 O(n)
1488 方法一:
1489 int log_2[N]; // 存 log2(i) 的结果
1490 void init(int n)
1491 {
1492     for (int i = 0; (1 << i) <= n; i ++ )
1493         log_2[1 << i] = i;
1494     for (int i = 1; i <= n; i ++ )
1495         if (!log_2[i])
1496             log_2[i] = log_2[i - 1];
1497 }
1498 // 预处理 log 值, 适用于需要多次查询的场景
1499
1500 方法二:
1501 int log_2[N]; // 存 log2(i) 的结果
1502 void init(int n)
1503 {
1504     for (int i = 1; i <= n; i ++ )
1505         log_2[i] = log_2[i - 1] + ((1 << log_2[i - 1]) == i);
1506     for (int i = 1; i <= n; i ++ )
1507         log_2[i] -- ;
1508 }
1509 // 另一种预处理方式, 适用于不同的编程习惯
1510

```



```
1511 5. 快读优化
1512 inline int read()
1513 {
1514     int x = 0; bool f = false; char ch = getchar();
1515     while (ch < 48 || ch > 57) {f |= (ch == 45); ch = getchar();}
1516     while (ch > 47 && ch < 58) x = (x << 1) + (x << 3) + (ch ^ 48), ch = getchar();
1517     return f ? ~x + 1 : x;
1518 }
1519 // 利用位运算和异或操作实现快速读取整数, 适用于需要高效输入的场景
1520
1521 6. 龟速加
1522 int add(int a, int b)
1523 {
1524     while (b)
1525     {
1526         int x = a ^ b;
1527         b = (a & b) << 1;
1528         a = x;
1529     }
1530     return a;
1531 }
1532 // 利用位运算模拟加法, 适用于理解计算机底层运算原理
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
```



离散化

```

1554
1555
1556 // 离散化
1557
1558 // 用来处理非常分散的数据(值域跨度很大, 但非常稀疏)
1559
1560 // 离散化
1561
1562 // 题目描述 :
1563 // 假定有一个无限长的数轴, 数轴上每个坐标上的数都是 0。
1564 // 现在, 我们首先进行  $n$  次操作, 每次操作将某一位置  $x$  上的数加  $c$ 。
1565 // 接下来, 进行  $m$  次询问, 每个询问包含两个整数  $l$  和  $r$ , 你需要求出在区间  $[l, r]$  之间
1566 // 的所有数的和。
1567
1568 // 输入格式
1569 // 第一行包含两个整数  $n$  和  $m$ 。
1570 // 接下来  $n$  行, 每行包含两个整数  $x$  和  $c$ 。
1571 // 再接下里  $m$  行, 每行包含两个整数  $l$  和  $r$ 。
1572
1573 // 输出格式
1574 // 共  $m$  行, 每行输出一个询问中所求的区间内数字和。
1575
1576 // 数据范围
1577 //  $-10^9 \leq x \leq 10^9$ ,
1578 //  $1 \leq n, m \leq 10^5$ ,
1579 //  $-10^9 \leq l \leq r \leq 10^9$ ,
1580 //  $-10000 \leq c \leq 10000$ 
1581
1582 // 输入样例 :
1583 // 3 3
1584 // 1 2
1585 // 3 6
1586 // 7 5
1587 // 1 3
1588 // 4 6
1589 // 7 8
1590 // 输出样例 :
1591 // 8
1592 // 0
1593 // 5
1594
1595 // 分析 :

```



```

1596 // 本题操作的数可能在 $-10^9$ 到 $10^9$ 之间, 如果开数组的话便过于庞大了, 但是操作的次
1597 数  $n$  在十万以内, 意味着最终出现的非 0 的数字会不超过十万, 考虑采取离散化, 将每次操
1598 作的数离散化为一个较小的数。
1599 // 第一步: 将待离散化的一组数存入向量;
1600 // 第二步: 去重, 对向量进行排序, 之后用 unique 函数去重, unique 函数是将不重复的
1601 元素都移动到向量前面, 向量后面的元素维持不变, 然后返回不重复序列的后一个位置, 所
1602 以可以用 alls.erase(unique(alls.begin(), alls.end()), alls.end()); 语句来实现对向
1603 量 alls 去重的目的;
1604 // 第三步: 离散化, 利用二分将向量中的元素映射为其在向量中的位置, 这里第一个位置
1605 从 1 开始, 为了后续前缀和的使用方便。
1606 // 离散化完数组后直接用前缀和即可求得指定区间内数的和了。这里离散化过程中为什么
1607 要每次都二分去查找元素映射的值, 而不是一劳永逸的用哈希表存储下映射关系呢? 因为元
1608 素过大, 使用哈希表的话效率很低, 而二分查找一个的速度为  $\log 100000$ , 最多十几次二分
1609 操作即可得到元素映射的位置, 速度非常快。
1610
1611 #include <iostream>
1612 #include <vector>
1613 #include <algorithm>
1614 using namespace std;
1615
1616 typedef pair<int, int> PII; // 创建一个二元组 PII
1617
1618 const int N = 300010;
1619
1620 int n, m;
1621 int a[N], s[N]; // a 数组是离散化后的数组 (意思就是 a 数组储存的是原函数的坐标),
1622 s 数组用来储存 a 数组的前缀和
1623
1624 vector<int> alls; // alls 数组用来收集所有在操作和询问过程中出现过的坐标值,
1625 后续会基于它进行离散化处理, 构建起原始坐标与离散化后坐标的映射关系。
1626 vector<PII> add, query; // add 数组用于存储操作信息 (坐标与增加值), query 数组用
1627 于储存询问区间的左右端点
1628
1629 int find(int x) // 二分查找 (找到原函数坐标所对应的离散化后的数组的位置)
1630 {
1631     int l = 0;
1632     int r = alls.size() - 1;
1633     while (l < r)
1634     {
1635         int mid = (l + r) >> 1;
1636         if (alls[mid] >= x)
1637             r = mid;
1638         else
1639             l = mid + 1;

```




```

1640     }
1641     return r + 1; // 返回 1 的原因是方便后续计算前缀和：计算前缀和数组 s 和使用前
1642     缀和计算区间和的部分，数组下标是从 1 开始使用的。
1643 }
1644
1645 int main()
1646 {
1647     cin >> n >> m; // 先进行 n 次操作，再进行 m 次询问
1648     for (int i = 0; i < n; i++)
1649     {
1650         int x, c;
1651         cin >> x >> c;
1652         add.push_back({x, c}); // 记录 add 操作详情，包含坐标与增加值的二元组
1653         alls.push_back(x); // 记录操作涉及的坐标 x
1654     }
1655
1656     for (int i = 0; i < m; i++)
1657     {
1658         int l, r;
1659         cin >> l >> r;
1660         query.push_back({l, r}); // 将区间端点组成的二元组存入 query 向量，保存
1661         询问的区间范围
1662
1663         alls.push_back(l);
1664         alls.push_back(r);
1665     }
1666
1667     // 离散化操作
1668     // 去重
1669     sort(alls.begin(), alls.end());
1670     alls.erase(unique(alls.begin(), alls.end()), alls.end());
1671
1672     for (auto item : add) // 每个 item 是一个 PII 类型，包含操作坐标 x 和增加值 c
1673     {
1674         int x = find(item.first);
1675         a[x] += item.second;
1676     }
1677
1678     // 预处理前缀和
1679     for (int i = 1; i <= static_cast<int>(alls.size()); i++)
1680         s[i] = s[i - 1] + a[i];
1681
1682     // 处理询问
1683     for (auto item : query)

```



```

1684     {
1685         int l = find(item.first), r = find(item.second);
1686         cout << s[r] - s[l - 1] << endl;
1687     }
1688     return 0;
1689 }
1690
1691 // ai の详细注释版
1692
1693 // #include <iostream>
1694 // #include <vector>
1695 // #include <algorithm>
1696 // using namespace std;
1697
1698 // // 创建一个二元组 PII, 方便后续用来清晰地表示由两个整数组成的有序对, 这里用于
1699 // 存储操作信息 (坐标与增加值) 以及询问区间的左右端点。
1700 // typedef pair<int, int> PII;
1701
1702 // const int N = 300010;
1703 // // 定义一个相对较大的常量 N, 用于限制数组 a 和 s 的容量, 防止后续操作中可能出现
1704 // 的数组越界情况, 其具体值可根据预估的数据量规模适当调整。
1705
1706 // int n, m;
1707 // // 分别用于接收从标准输入读取的操作次数和询问次数, 确定后续要进行的操作以及询
1708 // 问的轮数。
1709
1710 // int a[N], s[N];
1711 // // a 数组用于存储离散化后各坐标位置经过相应操作后的数值情况, 例如 a[i] 表示离散
1712 // 化后坐标为 i 的位置上经过之前操作累加后的数值。
1713 // // s 数组用于存储前缀和, s[i] 表示从离散化后的第一个坐标位置到坐标 i 处所有 a 数
1714 // 组对应元素的总和, 方便后续快速计算区间和。
1715
1716 // vector<int> alls;
1717 // // 因为题目中坐标值域跨度大但实际涉及的坐标较稀疏, 所以通过收集所有相关坐标值,
1718 // 经过排序、去重等离散化操作后, 能够建立起原始坐标与离散化后相对紧凑的坐标范围之间
1719 // 的对应关系, 便于后续处理。
1720
1721 // vector<PII> add, query;
1722 // // add 向量中每个元素是一个 PII 类型, 其中 first 成员表示操作对应的坐标 x, second
1723 // 成员表示在该坐标上要增加的值 c, 以此记录每次操作的具体情况。
1724 // // query 向量里每个元素同样是 PII 类型, first 和 second 分别对应询问区间的左右端
1725 // 点 l 和 r, 用于保存每次询问的区间范围信息。
1726
1727 // // 二分查找, 找到原函数坐标所对应的离散化后的数组的位置

```



```

1728 // int find(int x)
1729 // {
1730 //     int l = 0;
1731 //     int r = alls.size() - 1;
1732 //     while (l < r)
1733 //     {
1734 //         int mid = (l + r) >> 1;
1735 //         if (alls[mid] >= x) r = mid;
1736 //         else l = mid + 1;
1737 //     }
1738 //     // 由于后续在处理如前缀和数组 s 的赋值以及利用 s 计算区间和时, 代码逻辑中是
1739 // 基于下标从 1 开始操作的, 返回 r + 1 能确保这里找到的坐标 x 在 alls 数组中对应的离散
1740 // 化后的位置索引与后续使用的数组下标准确对应,
1741 //     // 避免在计算如 s[r] - s[l - 1] (计算区间和) 等操作时因下标不一致而出错,
1742 // 使得整个离散化后的坐标体系与基于前缀和的区间计算逻辑相契合。
1743 //     return r + 1;
1744 // }
1745
1746 // int main()
1747 // {
1748 //     cin >> n >> m;
1749 //     // 通过标准输入读取两个整数, 分别赋值给 n 和 m, 后续会基于这两个值分别使用
1750 // 循环来读取 n 次操作的具体信息以及 m 次询问的区间信息, 以完成整个问题的处理流程。
1751
1752 //     for (int i = 0; i < n; i++)
1753 //     {
1754 //         int x, c;
1755 //         cin >> x >> c;
1756 //         add.push_back({x, c});
1757 //         // 这样每次操作对应的坐标 x 和增加值 c 都被准确记录下来, 其中坐标 x 添加
1758 // 到 alls 数组中, 是为了后续在离散化过程中将其纳入所有相关坐标值集合, 确保离散化涵
1759 // 盖所有操作涉及的坐标;
1760 //         // 而将{x, c} 添加到 add 向量, 则完整保存了每次操作的详细内容, 便于后续
1761 // 根据离散化后的坐标位置来更新相应的数值情况。
1762 //         alls.push_back(x);
1763 //     }
1764
1765 //     for (int i = 0; i < m; i++)
1766 //     {
1767 //         int l, r;
1768 //         cin >> l >> r;
1769 //         query.push_back({l, r});
1770 //         // 把询问区间的左右端点 l 和 r 组成的二元组存入 query 向量, 方便后续遍历
1771 // 所有询问区间进行处理;

```



```

1772
1773 //      alls.push_back(l);
1774 //      alls.push_back(r);
1775 //      // 同时将 l 和 r 也添加到 alls 数组中, 是因为离散化需要综合考虑操作和询
1776 问过程中涉及的所有坐标值, 这样能保证在去重、建立映射等离散化操作时, 所有相关坐标
1777 都被正确处理, 进而准确计算出各区间的和。
1778 //      }
1779
1780 //      // 去重
1781 //      sort(alls.begin(), alls.end());
1782 //      // 首先使用 sort 函数对 alls 数组中的所有坐标值按照从小到大的顺序进行排序,
1783 这是离散化的基础步骤, 使得后续能准确地去除重复元素以及建立正确的坐标映射关系。
1784 //      alls.erase(unique(alls.begin(), alls.end()), alls.end());
1785 //      // 然后调用 alls.erase(unique(alls.begin(), alls.end()), alls.end()), 其
1786 中 unique 函数先将 alls 数组中相邻的重复元素移到数组末尾, 并返回指向去重后有效元素
1787 末尾的迭代器,
1788 //      // 接着 erase 函数依据该迭代器真正删除这些重复元素, 经过这两步操作, alls
1789 数组中就只保留了经过排序和去重后的离散化后的坐标值集合, 完成了离散化过程中的关键
1790 步骤, 实现了将分散的坐标值映射到相对紧凑、无重复的范围的目的。
1791
1792 //      for (auto item : add)
1793 //      {
1794 //          // 通过循环遍历 add 向量中的每个操作记录 item, 先调用 find 函数找到操作
1795 记录中原始坐标 item.first 对应的离散化后的坐标位置 x,
1796 //          // 然后依据这个离散化后的坐标位置 x, 将对应的增加值 item.second 累加到
1797 a[x] 中, 如此一来, a 数组就能准确记录下离散化后各个坐标位置上经过所有操作后的累计
1798 数值情况, 为后续计算前缀和做准备。
1799 //          int x = find(item.first);
1800 //          a[x] += item.second;
1801 //      }
1802
1803 //      // 预处理前缀和
1804 //      for (int i = 1; i <= static_cast<int>(alls.size()); i++) s[i] = s[i - 1] +
1805 a[i];
1806 //      // 从索引为 1 开始, 按照前缀和的计算原理, 通过循环遍历离散化后的坐标范围,
1807 对于每个位置 i, 将 s[i] 更新为前一个位置的前缀和 s[i - 1] 加上当前位置 a[i] 中的数值,
1808 //      // 以此计算并更新前缀和数组 s。经过这个预处理, 后续在计算区间和时就能利用
1809 前缀和的特性, 通过简单的减法运算 (如 s[r] - s[l - 1]) 快速得出结果, 避免了每次询问
1810 都重新遍历区间内所有元素进行求和的低效操作, 大大提高了区间求和的效率。
1811
1812 //      // 处理询问
1813 //      for (auto item : query)
1814 //      {
1815 //          int l = find(item.first), r = find(item.second);

```



```

1816 //          // 利用范围 for 循环遍历 query 向量中的每一个询问记录 item, 先分别调用
1817 find 函数找到询问区间左右端点 item.first 和 item.second 对应的离散化后的坐标位置 l
1818 和 r,
1819 //          // 由于之前已经预处理好了前缀和数组 s, 所以通过 s[r] - s[l - 1] 的方式,
1820 依据前缀和计算区间和的原理, 能够准确地计算并输出离散化后坐标区间 [l, r] 对应的区间
1821 和, 并且每次输出后通过 endl 进行换行, 使输出结果在终端显示时格式更加清晰规范, 完
1822 成对每次询问区间和的求解及输出。
1823 //          cout << s[r] - s[l - 1] << endl;
1824 //      }
1825
1826 //      return 0;
1827 // }
1828 // 合并区间 (合并所有有交集的区间)
1829
1830 // AcWing 803. 区间合并
1831 // 1、题目 (来源于 AcWing) :
1832 // 给定 n 个区间 [l, r], 要求合并所有有交集的区间。
1833 // 注意如果在端点处相交, 也算有交集。
1834 // 输出合并完成后的区间个数。
1835 // 例如: [1, 3] 和 [2, 6] 可以合并为一个区间 [1, 6]。
1836
1837 // 输入格式
1838 // 第一行包含整数 n。
1839 // 接下来 n 行, 每行包含两个整数 l 和 r。
1840
1841 // 输出格式
1842 // 共一行, 包含一个整数, 表示合并区间完成后的区间个数。
1843
1844 // 数据范围
1845 // 1 ≤ n ≤ 100000,
1846 // -109 ≤ l ≤ r ≤ 109
1847
1848 // 输入样例 :
1849 // 5
1850 // 1 2
1851 // 2 4
1852 // 5 6
1853 // 7 8
1854 // 7 9
1855 // 输出样例 :
1856 // 3
1857
1858 // 2、基本思想 :
1859 // 按区间的左端点排序, 当前维护区间的下一个区间有三种情况 :

```



```

1860 // ①含于当前维护区间
1861 // ②左端点>=当前维护区间左端点但是与当前维护区间有交集
1862 // ③左端点>当前维护区间右端点
1863
1864 // 对于①②两种情况,将当前维护区间更新即可,对于③将当前维护区间换成下一个区间。
1865
1866 // 合并区间(合并所有有交集的区间)
1867
1868 #include <iostream>
1869 #include <algorithm>
1870 #include <vector>
1871 using namespace std;
1872
1873 // 定义一个类型别名 PII,它是一个由两个 int 类型组成的 pair,用于表示区间的左右
1874 端点
1875 typedef pair<int, int> PII;
1876
1877 // 定义常量 N,用于表示最大可能的区间数量
1878 const int N = 100010;
1879
1880 // 存储输入的区间数量
1881 int n;
1882 // 存储所有输入的区间,每个区间用一个 PII 表示
1883 vector<PII> segs;
1884
1885 // 合并区间的函数,传入一个存储区间的 vector 的引用,方便直接修改原 vector
1886 void merge(vector<PII> &segs)
1887 {
1888     // 用于存储合并后的区间
1889     vector<PII> res;
1890
1891     // 对输入的区间按照 左端点 进行排序
1892     // 这样可以保证后续合并操作的顺序性
1893     sort(segs.begin(), segs.end());
1894
1895     // 初始化当前合并区间的左端点和右端点为一个极小值
1896     // 这个极小值用于标记初始状态,确保第一个区间能正确处理
1897     int st = -2e9, ed = -2e9;
1898
1899     // 遍历所有输入的区间
1900     for (auto seg : segs)
1901     {
1902         // 如果当前合并区间的右端点小于当前遍历到的区间的左端点
1903         // 说明这两个区间没有交集

```



```

1904         if (ed < seg.first)
1905         {
1906             // 如果 st 不等于初始的极小值, 说明已经有一个有效的合并区间
1907             // 将这个合并区间加入到结果 vector 中
1908             if (st != -2e9) // 这里用 ed 判断和 st 判断是等效的
1909                 res.push_back({st, ed});
1910             // 更新当前合并区间的左端点为当前遍历到的区间的左端点
1911             st = seg.first;
1912             // 更新当前合并区间的右端点为当前遍历到的区间的右端点
1913             ed = seg.second;
1914         }
1915         else
1916         {
1917             // 如果两个区间有交集, 更新当前合并区间的右端点为两者右端点的最大值
1918             // 这样可以确保合并后的区间能覆盖所有有交集的部分
1919             ed = max(ed, seg.second);
1920         }
1921     }
1922
1923     // 处理最后一个合并区间(因为前面最后一步若是执行的是 else 或第一次执行 if (未
1924     // 运行 if(st != -2e9) 的情况), 就没有处理完最后一个合并区间)
1925     // 如果 st 不等于初始的极小值, 说明存在有效的合并区间, 将其加入结果 vector
1926     if (st != -2e9)
1927         res.push_back({st, ed});
1928
1929     // 将合并后的区间赋值给原 vector, 完成合并操作
1930     segs = res;
1931 }
1932
1933 int main()
1934 {
1935     // 从标准输入读取区间的数量
1936     cin >> n;
1937
1938     // 循环读取 n 个区间的左右端点
1939     for (int i = 0; i < n; i++)
1940     {
1941         int l, r;
1942         cin >> l >> r;
1943         // 将读取到的区间加入到 segs vector 中
1944         segs.push_back({l, r});
1945     }
1946
1947     // 调用 merge 函数对区间进行合并

```



加油喵！QwQ！

```
1948     merge(segs);
1949
1950     // 输出合并后区间的数量
1951     cout << segs.size() << endl;
1952
1953     return 0;
1954 }
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
```



单链表

1991

1992

1993 // 链表

1994

1995 // 一. 单链表 (邻接表) : 存储图和树 (ps: 本小节讲的是单链表)

1996 // 个人理解: 节点 1 (索引为 0) -> 节点 2 ("索引为 1, ne[0] = 1", "e[0] = 一个值")

1997 -> 节点 3 ("索引为 2, ne[1] = 2", "e[1] = 一个值") -> 节点 4 ("索引为 3, ne[2] = 3",

1998 "e[2] = 一个值") -> 节点 5 ("索引为 4, ne[3] = 4", "e[3] = 一个值")

1999 //

2000 // 二. 双链表: 优化某些问题

2001

2002

2003 // 例题: 实现一个单链表, 链表初始为空, 支持三种操作:

2004 // 1. 向链表头插入一个数

2005 // 2. 删除第 k 个插入的数后面的数

2006 // 3. 在第 k 个插入的数后面后插入一个数

2007 // 现在要对该链表进行 M 次操作, 进行完所有操作后, 从头到尾输出整个链表。

2008 // 注意: 题目中第 k 个插入的元素并不是指的是当前链表的第 k 个数。例如操作过程中

2009 一共插入了 n 个数, 则按照插入的时间顺序, 这 n 个数依次为: 第 1 个插入的数, 第 2 个插

2010 入的数, 第 3 个插入的数...

2011 //

2012 // 输入格式:

2013 // 第一行包含整数 M, 表示操作次数。

2014 // 接下来 M 行每行包含一个操作命令, 操作命令可能为以下几种:

2015 // (1) "H x", 表示向链表头插入一个数 x。

2016 // (2) "D k", 表示删除第 k 个数输入的数后面的数 (当 k 为 0 时, 表示删除

2017 头结点)。

2018 // (3) "I k x", 表示在第 k 个输入的数后面插入一个数 x (此操作中的 k 均

2019 大于 0)。

2020 //

2021 // 输出格式:

2022 // 共一行, 将整个链表从头到尾输出。

2023 //

2024 // 输入样例: (所有输入都合法)

2025 // 10

2026 // H 9

2027 // I 1 1

2028 // D 1

2029 // D 0

2030 // H 6

2031 // I 3 6

2032 // I 4 5



```

2033         // I 4 5
2034         // I 3 4
2035         // D 6
2036         //
2037         // 输出样例 :
2038         //      6 4 6 5
2039
2040     #include<iostream>
2041     using namespace std;
2042
2043     const int N = 100010;
2044
2045     // head 表示头结点的坐标
2046     // e[i] 表示点 i 的值
2047     // ne[i] 表示点 i 的 next 指针是多少
2048     // idx 存储当前已经用到了哪个点)(确保索引唯一)
2049     int head, e[N], ne[N], idx;
2050
2051     // 初始化 (此时 head -> 空 )
2052     void init()
2053     {
2054         head = -1;
2055         idx = 0;
2056     }
2057
2058     // 将 x 插入到头结点(链头)
2059     void add_to_head(int x)
2060     {
2061         e[idx] = x;
2062         ne[idx] = head;
2063         head = idx;
2064         idx++;
2065     }
2066
2067     // 将 x 插入到下标是 k 的点后面
2068     void add(int k, int x)
2069     {
2070         e[idx] = x;
2071         ne[idx] = ne[k];
2072         ne[k] = idx;
2073         idx++;
2074     }
2075
2076     // 将下标是 k 的点的后面的那一个点删掉 (删掉一个点)

```



加油喵! QwQ !

```
2077 // 个人理解      : 节点 1 (索引为 0) -> 节点 2 ("索引为 1, ne[0] = 1", "e[0] = 一
2078 个值") -> 节点 3 ("索引为 2, ne[1] = 2", "e[1] = 一个值") -> 节点 4 ("索引为 3,
2079 ne[2] = 3", "e[2] = 一个值") -> 节点 5 ("索引为 4, ne[3] = 4", "e[3] = 一个值")
2080 // 删除(节点 3)后: 节点 1 (索引为 0) -> 节点 2 ("索引为 3, ne[0] = 3", "e[0] = 一
2081 个值") -> (被删掉的节点 3) 节点 4 ("索引为 3,
2082 ne[2] = 3", "e[2] = 一个值") -> 节点 5 ("索引为 4, ne[3] = 4", "e[3] = 一个值")
2083 void remove(int k)
2084 {
2085     ne[k] = ne[ne[k]]; // 就是被删掉的点的前一个点指向这个点 (指的是被删掉的点的
2086 前一个点) 的下一个点的下一个点;
2087 }
2088
2089 int main()
2090 {
2091     int m;
2092     cin >> m;
2093
2094     init();
2095
2096     while(m--)
2097     {
2098         int k, x;
2099         char op;
2100         cin >> op;
2101         if(op == 'H')
2102         {
2103             cin >> x;
2104             add_to_head(x);
2105         }
2106         else if(op == 'D')
2107         {
2108             cin >> k;
2109             if(!k) head = ne[head]; // 判断是不是删除头结点
2110             remove(k - 1); // 之所以要传 k - 1, 是因为代码中节点索引是从 0 开始
2111 的, 而题目中的 k 是基于插入顺序计数的, 所以要减 1 来对应到实际的节点索引
2112         }
2113         else
2114         {
2115             cin >> k >> x;
2116             add(k - 1, x); // 之所以要传 k - 1, 是因为代码中节点索引是从 0 开始
2117 的, 而题目中的 k 是基于插入顺序计数的, 所以要减 1 来对应到实际的节点索引
2118         }
2119     }
2120 }
```



加油喵！ QwQ ！

```
2121     for(int i = head; i != -1; i = ne[i])
2122     {
2123         cout << e[i] << " ";
2124     }
2125     cout << endl;
2126     return 0;
2127 }
```

2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164



双链表

2165

2166

2167 // 链表

2168

2169 // 一. 单链表 (邻接表) : 存储图和树

2170 // 个人理解: 节点 1 (索引为 0) -> 节点 2 ("索引为 1, ne[0] = 1", "e[0] = 一个值")

2171 -> 节点 3 ("索引为 2, ne[1] = 2", "e[1] = 一个值") -> 节点 4 ("索引为 3, ne[2] = 3",

2172 "e[2] = 一个值") -> 节点 5 ("索引为 4, ne[3] = 4", "e[3] = 一个值")

2173 //

2174 // 二. 双链表: 优化某些问题 (ps: 本小节讲的是双链表)

2175 // 个人理解: 双链表有 2 个指针, 一个指向前, 一个指向后。

2176

2177 // 这里偷个懒不定义头结点了, 用下标为 0 的点表示头结点, 用下标为 1 的点表示尾结点

2178

2179 #include <iostream>

2180 using namespace std;

2181

2182 const int N = 100010; // 定义一个较大的数组长度, 可根据实际需求调整

2183

2184 // 存储节点的值

2185 int e[N];

2186 // 存储节点的前驱索引, l[i] 表示索引为 i 的节点的前驱节点索引

2187 int l[N];

2188 // 存储节点的后继索引, r[i] 表示索引为 i 的节点的后继节点索引

2189 int r[N];

2190 // 记录当前可用的节点索引, 相当于链表节点的个数

2191 int idx;

2192

2193 // 初始化双向链表

2194 void init()

2195 {

2196 // 0 表示左端点 (虚拟头节点), 1 表示右端点 (虚拟尾节点)

2197 r[0] = 1;

2198 l[1] = 0;

2199 idx = 2;

2200 }

2201

2202 // 在节点 k 的右边插入一个值为 x 的新节点

2203 void add_right(int k, int x) // 如果要改成在双链表的左边插入一个数的话, 就把 传

2204 入函数时 把传入函数的 k 改成 l[k] 就好了

2205 {

2206 e[idx] = x;



```

2207     r[idx] = r[k];
2208     l[idx] = k;
2209     // 下面这2不可以调换顺序
2210     l[r[k]] = idx; // 当前节点的右边节点的左边指向新建节点
2211     r[k] = idx; // 当前节点的右边指向新建节点
2212     idx++;
2213 }
2214
2215 // 在节点 k 的左边插入一个值为 x 的新节点
2216 void add_left(int k, int x)
2217 {
2218     // 先获取节点 k 的前驱节点索引
2219     int prev_k = l[k];
2220     // 在节点 k 的前驱节点 prev_k 的右边插入新节点 (等同于在节点 k 的左边插入)
2221     add_right(prev_k, x);
2222 }
2223
2224 // 在链表头部插入一个值为 x 的新节点 (等同于在虚拟头节点 0 的右边插入)
2225 void add_to_head(int x)
2226 {
2227     add_right(0, x);
2228 }
2229
2230 // 在链表尾部插入一个值为 x 的新节点 (等同于在虚拟尾节点 1 的左边插入, 即虚拟尾节
2231 点 1 的前驱节点右边插入)
2232 void add_to_tail(int x)
2233 {
2234     add_right(l[1], x);
2235     // 或
2236     // add_left(1, x);
2237 }
2238
2239 // 删除索引为 k 的节点
2240 void remove(int k)
2241 {
2242     r[l[k]] = r[k]; // 左边节点的右边指向当前节点的右边
2243     l[r[k]] = l[k]; // 右边节点的左边指向当前节点的左边
2244 }
2245
2246 // 专门用于删除虚拟头节点后的第一个普通节点, 使得头结点直接指向尾节点 (保留虚拟
2247 头、尾节点)
2248 void remove_whole()
2249 {
2250     while(r[0] != 1)

```



```

2251     {
2252         r[0] = r[r[0]];
2253         l[r[0]] = 0;
2254     }
2255     idx = 2;
2256 }
2257
2258 // 遍历并输出链表中的节点值 (不包含虚拟头节点和虚拟尾节点)
2259 void printList()
2260 {
2261     for (int i = r[0]; i != 1; i = r[i])
2262     {
2263         cout << e[i] << " ";
2264     }
2265     cout << endl;
2266 }
2267
2268 int main()
2269 {
2270     init();
2271     cout << r[0] << endl;
2272     cout << l[1] << endl;
2273
2274     add_to_head(5);
2275     add_to_tail(10);
2276     add_right(2, 8); // 在索引为 2 的节点 (这里是值为 5 的节点) 右边插入值为 8 的节
2277 点
2278     add_left(2, 6); // 在索引为 2 的节点 (值为 5 的节点) 左边插入值为 6 的节点
2279     printList();
2280
2281     remove(2); // 删除索引为 2 的节点 (即值为 8 的节点)
2282
2283     cout << "1. After deleting: ";
2284     printList();
2285
2286     remove_whole();
2287
2288     cout << "2. After deleting: ";
2289     printList();
2290
2291     cout << r[0] << endl;
2292     cout << l[1] << endl;
2293
2294     return 0;

```



加油喵！ QwQ ！

2295 }
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338



栈

2339

2340

2341 // 栈 (先进后出)

2342

2343 // // ***** 栈

2344 // int stk[N], tt;

2345 // // 插入

2346 // stk[++tt] = x;

2347 // // 弹出

2348 // tt--;

2349 // // 判断栈是否为空

2350 // if (tt > 0) // 不空

2351 // else empty;

2352 // // 栈顶

2353 // stk[tt];

2354 // // *****

2355

2356 #include <iostream>

2357 using namespace std;

2358 const int N = 100010;

2359

2360 int stk[N];

2361 int tt = 0;

2362

2363 // 插入到栈顶

2364 void push(int x)

2365 {

2366 stk[++tt] = x;

2367 }

2368

2369 // 弹出栈顶元素

2370 void pop()

2371 {

2372 tt--;

2373 }

2374

2375 // 判断栈区是否为空

2376 bool is_Empty() // 这里不要命名为 is_empty 因为会和编译器内置的函数名字冲突

2377 {

2378 return tt == 0;

2379 }

2380



```

2381 void remove_Whole()
2382 {
2383     tt = 0;
2384 }
2385
2386 // print 栈顶
2387 void print_Top()
2388 {
2389     if(tt > 0)
2390     {
2391         cout << stk[tt - 1] << endl;
2392     }
2393     else
2394     {
2395         cout << "栈为空, 无栈顶元素可输出" << endl;
2396     }
2397 }
2398
2399 void print_stk()
2400 {
2401     cout << "( 栈顶 -> 栈底 ) : ";
2402     for(int i = tt - 1; i > 0; i--)
2403     {
2404         cout << stk[i] << " ";
2405     }
2406     cout << endl;
2407 }
2408
2409 int main()
2410 {
2411     push(1);
2412     push(2);
2413     push(3);
2414     cout << "弹出前的栈 : ";
2415     print_stk(); // 栈顶 -> 栈底
2416     cout << "弹出前的栈顶元素 : ";
2417     print_Top(); // 栈顶 -> 栈底
2418     if(is_Empty()) cout << "此时栈为空" << endl;
2419     else cout << "此时栈不为空" << endl;
2420     pop();
2421     cout << "弹出后的栈 : ";
2422     print_stk(); // 栈顶 -> 栈底
2423     remove_Whole();
2424     cout << "清空后的栈 : ";

```



加油喵！QwQ！

```
2425     print_stk(); // 栈顶 -> 栈底
2426     return 0;
2427 }
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
```



单调栈

```

2469
2470
2471 // 单调栈
2472
2473 // 例题：找到一个数列中每个数离这个数最近且比它小的数并返回这个数，若没有，则返
2474 回-1
2475 // 输入格式：第一行包含一个整数 N，表示数组长度
2476 //           第二行包含 N 个整数，表示整数数列
2477 // 输出格式：共一行，包含 N 个整数，其中第 i 个数表示第 i 个数左边第一个比它小的数，
2478 如果不存在则输出-1；
2479
2480 // 输入样例：5
2481 //           3 4 2 7 5
2482 // 输出样例：-1 3 -1 2 2
2483 // 个人理解：如果右边的数小于左边的数，那么左边的数就没有意义了（可以删去），最
2484 终留下来的数组是一个从左到右，从大到小的单调栈
2485
2486 // 本题中输入输出的压力较大，建议使用 scanf 和 printf。
2487 // 或
2488 // 加上
2489 // ios::sync_with_stdio(false); // 关闭 iostream 与 stdio 的同步，提高输入输出
2490 效率
2491 // cin.tie(0); // 解除 cin 与 cout 的绑定，进一步提高输入输出效率
2492 // cout.tie(0); // 解除 cout 与 cin 的绑定，进一步提高输入输出效率
2493 #include<iostream>
2494 using namespace std;
2495
2496 const int N = 1000010;
2497
2498 int n;
2499 int stk[N], tt;
2500
2501 int main()
2502 {
2503     cin >> n;
2504     for(int i = 0; i < n; i++)
2505     {
2506         int x;
2507         cin >> x;
2508         while(tt && stk[tt] >= x) tt--;
2509         if(tt) cout << stk[tt] << " ";
2510         else cout << -1 << " ";
2511         stk[++tt] = x;

```



加油喵！ QwQ ！

```
2512     }  
2513     return 0;  
2514 }  
2515  
2516  
2517  
2518  
2519  
2520  
2521  
2522  
2523  
2524  
2525  
2526  
2527  
2528  
2529  
2530  
2531  
2532  
2533  
2534  
2535  
2536  
2537  
2538  
2539  
2540  
2541  
2542  
2543  
2544  
2545  
2546  
2547  
2548  
2549  
2550  
2551  
2552  
2553  
2554
```



队列

2555

2556

2557 // 队列 (先进先出)

2558

2559 #include<iostream>

2560 using namespace std;

2561

2562 const int N = 100010;

2563

2564 // ***** 队列

2565

2566 // 在队尾插入元素, 在队头弹出元素

2567 int q[N], hh, tt = -1;

2568

2569 // 插入

2570 q[++tt] = x;

2571

2572 // 弹出

2573 hh ++;

2574

2575 // 判断是否为空

2576 if(hh <= tt) not empty;

2577 else empty;

2578

2579 // 取出队头元素 (拿数组模拟的话, 不仅可以取出队头元素, 还可以取出队尾元素)

2580 q[hh];

2581 // 取出队尾元素

2582 q[tt];

2583

2584 // *****

2585

2586

2587

2588

2589

2590

2591

2592

2593

2594

2595

2596



单调队列

```

2597
2598
2599 // 单调队列
2600
2601 // 题目来源: AcWing 154. 滑动窗口
2602
2603 // 一、题目描述
2604 // 给定一个大小为  $n \leq 10^6$  的数组。
2605 // 有一个大小为  $k$  的滑动窗口, 它从数组的最左边移动到最右边。
2606 // 你只能在窗口中看到  $k$  个数字。
2607 // 每次滑动窗口向右移动一个位置。
2608
2609 // 以下是一个例子:
2610 // 该数组为 [1 3 -1 -3 5 3 6 7],  $k$  为 3。
2611
2612 //      窗口位置      最小值    最大值
2613 // [1 3 -1] -3 5 3 6 7    -1        3
2614 // 1 [3 -1 -3] 5 3 6 7    -3        3
2615 // 1 3 [-1 -3 5] 3 6 7    -3        5
2616 // 1 3 -1 [-3 5 3] 6 7    -3        5
2617 // 1 3 -1 -3 [5 3 6] 7     3        6
2618 // 1 3 -1 -3 5 [3 6 7]     3        7
2619 // 你的任务是确定滑动窗口位于每个位置时, 窗口中的最大值和最小值。
2620
2621 // 输入格式
2622
2623 // 输入包含两行。
2624 // 第一行包含两个整数  $n$  和  $k$ , 分别代表数组长度和滑动窗口的长度。
2625 // 第二行有  $n$  个整数, 代表数组的具体数值。
2626 // 同行数据之间用空格隔开。
2627
2628 // 输出格式
2629
2630 // 输出包含两个。
2631 // 第一行输出, 从左至右, 每个位置滑动窗口中的最小值。
2632 // 第二行输出, 从左至右, 每个位置滑动窗口中的最大值。
2633
2634 // 输入样例:
2635 // 8 3
2636 // 1 3 -1 -3 5 3 6 7
2637
2638 // 输出样例:
2639 // -1 -3 -3 -3 3 3
2640 // 3 3 5 5 6 7

```



```

2641
2642 #include<iostream>
2643 using namespace std;
2644
2645 const int N = 100010;
2646
2647 int n, k;
2648 int a[N], q[N];
2649
2650 int main()
2651 {
2652     scanf("%d%d", &n, &k);
2653     for(int i = 0; i < n; i++) scanf("%d", &a[i]);
2654
2655     // 最小值
2656     int hh = 0, tt = -1;
2657     for(int i = 0; i < n; i++)
2658     {
2659         // 判断队头是否已经滑出窗口
2660         if(hh <= tt && i - k + 1 > q[hh]) hh++;
2661         while(hh <= tt && a[q[tt]] >= a[i]) tt--;
2662         q[++tt] = i;
2663         if(i >= k - 1) printf("%d ", a[q[hh]]);
2664
2665     }
2666     puts("");
2667
2668     // 最大值
2669     hh = 0, tt = -1;
2670     for(int i = 0; i < n; i++)
2671     {
2672         // 判断队头是否已经滑出窗口
2673         if(hh <= tt && i - k + 1 > q[hh]) hh++;
2674         while(hh <= tt && a[q[tt]] <= a[i]) tt--;
2675         q[++tt] = i;
2676         if(i >= k - 1) printf("%d ", a[q[hh]]);
2677
2678     }
2679     puts("");
2680
2681     return 0;
2682 }
2683

```



KMP 算法

2684

2685

2686 // KMP 算法

2687

2688 // 题目来源: AcWing 831. KMP 字符串

2689

2690 // 一、题目描述

2691 // 给定一个模式串 S , 以及一个模板串 P , 所有字符串中只包含大小写英文字母以及阿
2692 拉伯数字。2693 // 模板串 P 在模式串 S 中多次作为子串出现。2694 // 求出模板串 P 在模式串 S 中所有出现的位置的起始下标。

2695

2696 // 输入格式

2697 // 第一行输入整数 N , 表示字符串 P 的长度。2698 // 第二行输入字符串 P 。2699 // 第三行输入整数 M , 表示字符串 S 的长度。2700 // 第四行输入字符串 S 。

2701

2702 // 输出格式

2703 // 共一行, 输出所有出现位置的起始下标 (下标从 0 开始计数), 整数之间用空格隔开。

2704

2705 // 数据范围

2706 // 1 N 10^5 2707 // 1 M 10^6

2708

2709 // 输入样例:

2710 // 3

2711 // aba

2712 // 5

2713 // ababa

2714

2715 // 输出样例:

2716 // 0 2

2717

2718 #include<iostream>

2719 using namespace std;

2720

2721 const int N = 100010;

2722

2723 int n, m;

2724 int ne[N]; // next 数组简写

2725 char s[N], p[N];



```

2726
2727 int main()
2728 {
2729     cin >> n >> (p + 1) >> m >> (s + 1); // 下标从 1 开始
2730
2731     // 求 next 的过程
2732     for(int i = 2, j = 0; i <= n; i++)
2733     {
2734         while(j && p[i] != p[j + 1]) j = ne[j];
2735         if(p[i] == p[j + 1]) j++;
2736         ne[i] = j;
2737     }
2738
2739     // KMP 匹配过程
2740     for(int i = 1, j = 0; i <= m; i++)
2741     {
2742         while(j && s[i] != p[j + 1]) j = ne[j];
2743         if(s[i] == p[j + 1]) j++;
2744         if(j == n)
2745         {
2746             // 匹配成功
2747             printf("%d ", i - n);
2748             j = ne[j];
2749         }
2750     }
2751     cout << endl;
2752
2753     // for(int i = 0 ;i <= 130; i++)
2754     // {
2755     //     cout << ne[i] << " ";
2756     // }
2757
2758     return 0;
2759 }
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769

```



Trie 树

2770

2771

2772 // Trie 树的作用:用来高效地存储和查找字符串集合的数据结构

2773 // Trie 树是一种树形结构,用于存储字符串集合,特别适用于存储大量具有公共前缀的
2774 字符串,能够高效地实现字符串的插入和查询操作。

2775

2776 // 一般来说存储的字符串中的字符种类一般不会太多,但是会重复出现

2777

2778 // 题目来源:AcWing 835. Trie 字符串统计

2779 // 一、题目描述

2780 // 维护一个字符串集合,支持两种操作:

2781

2782 // 1. I x 向集合中插入一个字符串 x;

2783 // 2. Q x 询问一个字符串在集合中出现了多少次。

2784 // 共有 N 个操作,输入的字符串总长度不超过 10^5 ,字符串仅包含小写英文字母。

2785

2786 // 输入格式

2787 // 第一行包含整数 N,表示操作数。

2788

2789 // 接下来 N 行,每行包含一个操作指令,指令为 I x 或 Q x 中的一种。

2790

2791 // 输出格式

2792 // 对于每个询问指令 Q x,都要输出一个整数作为结果,表示 x 在集合中出现的次数。

2793 // 每个结果占一行。

2794

2795 // 数据范围

2796 // 1 N 2×10^4

2797

2798 // 输入样例:

2799 // 5

2800 // I abc

2801 // Q abc

2802 // Q ab

2803 // I ab

2804 // Q ab

2805

2806 // 输出样例:

2807 // 1

2808 // 0

2809 // 1

2810

2811 // 二、Trie 字典树思想



2812 // 字典树是用来高效地存储和查找字符串集合的数据结构。使用要求：所存储的字符串要
2813 么都是小写、要么都是大写、要么都是数字，反正字符的种类不会很多。

2814

2815 // 例如：一棵字典树如图所示：存储 abcdef、abdef、abc、acef、bcd、bcd、bcff、
2816 cdaa 等字符如图所示，字符串存储的时候，要将每一个字符串结尾打上一个标记，如下图
2817 中的三角形。

2818

```
2819 //          _____root_____
2820 //          /           |           |
2821 //          a           b           c
2822 //          /  ¥       |           |
2823 //          b   c       c           d
2824 //          /  ¥   |   /  ¥   |
2825 //         a* d   e   d   f   a
2826 //         |   |   |   /  ¥   |
2827 //         d   e   f* f* c* f* a*
2828 //         |   |
2829 //         e   f*
2830 //         |
2831 //         f*
```

2832

```
2833 #include<iostream>
2834 using namespace std;
```

2835

```
2836 const int N = 100010;
```

2837

```
2838 int son[N][26], cnt[N], idx;
```

```
2839 // // 下标是 0 的点既是根节点，又是空节点
```

```
2840 // // son[] 存储树中每个节点的子节点
```

```
2841 // // cnt[] 存储以每个节点结尾的单词有多少个
```

```
2842 // // idx 用于为新节点分配编号，从 1 开始，因为根节点编号为 0。
```

```
2843 char str[N];
```

2844

```
2845 // 第一个操作：插入操作（存储）
```

```
2846 void insert(char str[])
```

```
2847 {
```

```
2848     int p = 0; // 从根节点 0 开始
```

```
2849     for(int i = 0; str[i]; i++) // 因为在 C++ 中，字符串的结尾是 ¥0，所以可以用
2850 str[i] 来判断
```

```
2851     {
```

```
2852         int u = str[i] - 'a'; // 这步的目的是把 小写字母 a~z 映射成 数字 0~25
```

```
2853         if(!son[p][u]) son[p][u] = ++ idx; // 如果这个节点不存在的话，就把它创建
2854 出来
```

```
2855         p = son[p][u];
```



```

2856     }
2857     cnt[p]++;
2858
2859 }
2860
2861 // 第二个操作：查询操作
2862 int query(char str[]) // 查询操作和插入操作是类似的
2863 {
2864     int p = 0; // 从根节点 0 开始
2865     for(int i = 0; str[i]; i++)
2866     {
2867         int u = str[i] - 'a';
2868         if(!son[p][u]) return 0; // 如果不存在的话，直接 return 0
2869         p = son[p][u];
2870     }
2871     return cnt[p];
2872 }
2873
2874 int main()
2875 {
2876     int n;
2877     scanf("%d", &n);
2878     while(n--)
2879     {
2880         char op[2];
2881         scanf("%s%s", op, str); // 先读入操作类型，再读入字符串
2882         if(op[0] == 'I') insert(str);
2883         else printf("%d\n", query(str));
2884     }
2885
2886     return 0;
2887 }
2888
2889 // Trie 树    模板题 AcWing 835. Trie 字符串统计
2890
2891 // int son[N][26], cnt[N], idx;
2892 // // 0 号点既是根节点，又是空节点
2893 // // son[] 存储树中每个节点的子节点
2894 // // cnt[] 存储以每个节点结尾的单词数量
2895
2896 // // 插入一个字符串
2897 // void insert(char *str)
2898 // {
2899 //     int p = 0;

```



加油喵! QwQ !

```
2900 // for (int i = 0; str[i]; i ++ )
2901 // {
2902 //     int u = str[i] - 'a';
2903 //     if (!son[p][u]) son[p][u] = ++ idx;
2904 //     p = son[p][u];
2905 // }
2906 // cnt[p] ++ ;
2907 // }
2908
2909 // // 查询字符串出现的次数
2910 // int query(char *str)
2911 // {
2912 //     int p = 0;
2913 //     for (int i = 0; str[i]; i ++ )
2914 //     {
2915 //         int u = str[i] - 'a';
2916 //         if (!son[p][u]) return 0;
2917 //         p = son[p][u];
2918 //     }
2919 //     return cnt[p];
2920 // }
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
```



查并集

2944

2945

2946 // 并查集

2947 // 1. 将 2 个集合合并

2948 // 2. 询问 2 个元素是否在一个集合当中

2949

2950 // 每个集合用树来表示, 树根的编号就是整个集合的编号。每个节点存储它的父节点

2951

2952 // 问题一: 如何判断树根?

2953 // 如果一个节点是树根, 那么其 $p[x] == x$, 于是等价于 $if(p[x] == x)$;2954 // 问题二: 如何求 x 的集合编号2955 // 就是从 x 一直朝上走, 一直走到根节点: $while(p[x] != x) x = p[x]$;

2956 // 问题三: 如何合并 2 个集合?

2957 // 直接将一个集合插到另一个集合上, 比如 $p[x]$ 是 x 的集合编号, $p[y]$ 是 y 的集合编号。那么直接 $p[x] = y$;

2959

2960 // 路径压缩优化

2961 // 我们在查找一个元素 x 所在集合的根结点时, 一边往上溯根, 一边将这些结点的父结点都改为最终的集合根结点。一言以蔽之: 儿子和父亲变成兄弟。

2963

2964 // 按秩合并优化

2965 // 按秩合并优化的效果不明显, 因此在写代码的时候从来不会去写按秩合并的代码, 只会写路径压缩, 因此这里不再介绍。

2966

2967 // 题目来源: AcWing 836. 合并集合

2969 // 一、题目描述

2970

2971 // 一共有 n 个数, 编号是 $1 \sim n$, 最开始每个数各自在一个集合中。2972 // 现在要进行 m 个操作, 操作共有两种:2973 // 1. $M \ a \ b$, 将编号为 a 和 b 的两个数所在的集合合并, 如果两个数已经在同一个集合中, 则忽略这个操作;2974 // 2. $Q \ a \ b$, 询问编号为 a 和 b 的两个数是否在同一个集合中;

2975

2976 // 输入格式

2977 // 第一行输入整数 n 和 m 。2979 // 接下来 m 行, 每行包含一个操作指令, 指令为 $M \ a \ b$ 或 $Q \ a \ b$ 中的一种。

2980

2981 // 输出格式

2982 // 对于每个询问指令 $Q \ a \ b$, 都要输出一个结果, 如果 a 和 b 在同一集合内, 则输出 Yes, 否则输出 No。

2983 // 每个结果占一行。

2984



```

2986 // 数据范围
2987 // 1 n, m 10^5
2988
2989 // 输入样例：
2990 // 4 5
2991 // M 1 2
2992 // M 3 4
2993 // Q 1 2
2994 // Q 1 3
2995 // Q 3 4
2996
2997 // 输出样例：
2998 // Yes
2999 // No
3000 // Yes
3001
3002 // 二、并查集
3003 // 并查集可以快速的进行以下操作（近似  $O(1)$ ）：
3004
3005 // 将两个集合合并
3006 // 询问两个元素是否在一个集合当中
3007 // 每一个集合用一个树的形式来维护，每一个集合的编号就是当前集合树根结点的编号。
3008 // 对于每一个结点  $x$ ，都要记录其父结点  $p[x]$ 。
3009
3010 // 问题 1：如何判断树根？
3011 // if ( $p[x] == x$ )
3012 // 问题 2：如何求  $x$  的集合编号？
3013 // while( $p[x] != x$ )  $x = p[x]$ ;
3014 // 问题 3：如何合并两个集合？
3015 //  $p[x]$  是  $x$  的集合编号， $p[y]$  是  $y$  的集合编号， $p[x] = y$ ；
3016
3017 // 路径压缩优化
3018 // 我们在查找一个元素  $x$  所在集合的根结点时，一边往上溯根，一边将这些结点的父结点
3019 // 都改为最终的集合根结点。一言以蔽之：儿子和父亲变成兄弟。
3020
3021 // 按秩合并优化
3022 // 按秩合并优化的效果不明显，因此在写代码的时候从来不会去写按秩合并的代码，只会
3023 // 写路径压缩，因此这里不再介绍。
3024
3025 // *如何具体的知道每个集合里面包含哪些元素呢？
3026
3027 // vector<int> group;
3028 // for (int i = 0; i < n; i++)
3029 // group[find(i)].push_back(i);

```




```

3030
3031 #include<iostream>
3032 using namespace std;
3033
3034 const int N = 100010;
3035
3036 int n, m;
3037 // 一共有 n 个数, 编号是 1 ~ n, 最开始每个数各自在一个集合中。
3038 // 现在要进行 m 个操作, 操作共有两种:
3039 // 1. M a b, 将编号为 a 和 b 的两个数所在的集合合并, 如果两个数已经在同一个集合
3040 中, 则忽略这个操作;
3041 // 2. Q a b, 询问编号为 a 和 b 的两个数是否在同一个集合中;
3042 int p[N]; // 存储每个点的祖宗节点
3043
3044 // 第一个操作: 发现函数 (最终会返回 x 的祖宗节点 + 路径压缩优化)
3045 int find(int x)
3046 {
3047     if(p[x] != x) p[x] = find(p[x]); // 如果 x 不是根节点的话, 我们就让它的父节点
3048     等于它的祖宗节点, 最后返回它的父节点就可以了
3049     return p[x];
3050 }
3051
3052 int main()
3053 {
3054     scanf("%d%d", &n, &m);
3055     for(int i = 1; i <= n; i++) p[i] = i; // 最开始每个数各自在一个集合中
3056
3057     while(m--)
3058     {
3059         char op[2]; // 这里用 op[2] 是因为利用了 C++ 的一个特性, 字符串 (字符数组)
3060         相较于单个字符, 也就是可以忽略掉空格和回车
3061         int a, b;
3062         scanf("%s%d%d", op, &a, &b);
3063
3064         if(op[0] == 'M') p[find(a)] = find(b); // 这句话的含义就是让 a 的祖宗节
3065         点等于 b 的父节点 (a 插到 b 上)
3066         else
3067         {
3068             if(find(a) == find(b)) puts("Yes");
3069             else puts("No");
3070         }
3071     }
3072     return 0;
3073 }

```



```

3074 // 并查集 模板题 AcWing 836. 合并集合, AcWing 837. 连通块中点的数量
3075 // (1) 朴素并查集:
3076
3077 // int p[N]; // 存储每个点的祖宗节点
3078
3079 // // 返回 x 的祖宗节点
3080 // int find(int x)
3081 // {
3082 //     if (p[x] != x) p[x] = find(p[x]);
3083 //     return p[x];
3084 // }
3085
3086 // // 初始化, 假定节点编号是 1~n
3087 // for (int i = 1; i <= n; i++) p[i] = i;
3088
3089 // // 合并 a 和 b 所在的两个集合:
3090 // p[find(a)] = find(b);
3091
3092 // (2) 维护 size 的并查集:
3093
3094 // int p[N], size[N];
3095 // // p[] 存储每个点的祖宗节点, size[] 只有祖宗节点有意义, 表示祖宗节点所在集
3096 // 合中的点的数量
3097
3098 // // 返回 x 的祖宗节点
3099 // int find(int x)
3100 // {
3101 //     if (p[x] != x) p[x] = find(p[x]);
3102 //     return p[x];
3103 // }
3104
3105 // // 初始化, 假定节点编号是 1~n
3106 // for (int i = 1; i <= n; i++)
3107 // {
3108 //     p[i] = i;
3109 //     size[i] = 1;
3110 // }
3111
3112 // // 合并 a 和 b 所在的两个集合:
3113 // size[find(b)] += size[find(a)];
3114 // p[find(a)] = find(b);
3115
3116 // (3) 维护到祖宗节点距离的并查集:
3117

```



```

3118 //      int p[N], d[N];
3119 //      //p[] 存储每个点的祖宗节点, d[x] 存储 x 到 p[x] 的距离
3120
3121 //      // 返回 x 的祖宗节点
3122 //      int find(int x)
3123 //      {
3124 //          if (p[x] != x)
3125 //          {
3126 //              int u = find(p[x]);
3127 //              d[x] += d[p[x]];
3128 //              p[x] = u;
3129 //          }
3130 //          return p[x];
3131 //      }
3132
3133 //      // 初始化, 假定节点编号是 1~n
3134 //      for (int i = 1; i <= n; i ++ )
3135 //      {
3136 //          p[i] = i;
3137 //          d[i] = 0;
3138 //      }
3139
3140 //      // 合并 a 和 b 所在的两个集合 :
3141 //      p[find(a)] = find(b);
3142 //      d[find(a)] = distance; // 根据具体问题, 初始化 find(a) 的偏移量
3143
3144
3145
3146
3147
3148
3149
3150
3151
3152
3153
3154
3155
3156
3157
3158
3159
3160
3161

```



查并集例题

3162

3163

3164 // AcWing 837. 连通块中点的数量

3165

3166 // 1、题目 (来源于 AcWing) :

3167 // 给定一个包含 n 个点 (编号为 $1 \sim n$) 的无向图, 初始时图中没有边。3168 // 现在要进行 m 个操作, 操作共有三种 :3169 // 1. “C a b ”, 在点 a 和点 b 之间连一条边, a 和 b 可能相等 ;3170 // 2. “Q1 a b ”, 询问点 a 和点 b 是否在同一个连通块中, a 和 b 可能相等 ;3171 // 3. “Q2 a ”, 询问点 a 所在连通块中点的数量 ;

3172

3173 // 输入格式

3174 // 第一行输入整数 n 和 m 。3175 // 接下来 m 行, 每行包含一个操作指令, 指令为 “C a b ”, “Q1 a b ” 或 “Q2 a ” 中的一种。

3176

3177 // 输出格式

3178 // 对于每个询问指令 “Q1 a b ”, 如果 a 和 b 在同一个连通块中, 则输出 “Yes”, 否则输出 “No”。3181 // 对于每个询问指令 “Q2 a ”, 输出一个整数表示点 a 所在连通块中点的数量

3182 // 每个结果占一行。

3183

3184 // 数据范围

3185 // $1 \leq n, m \leq 10^5$

3186

3187 // 输入样例 :

3188 // 5 5

3189 // C 1 2

3190 // Q1 1 2

3191 // Q2 1

3192 // C 2 5

3193 // Q2 5

3194

3195 // 输出样例 :

3196 // Yes

3197 // 2

3198 // 3

3199

3200 // 2、基本思想 :

3201 // 与 AcWing 836. 合并集合类似, 但是加上一个 `size_` 表示每个连通块中点的数量, 仅有根节点的 `size_` 存储了该块中点的个数。

3203



```

3204 // 3、步骤：
3205 // 与 AcWing 836. 合并集合类似，求每个连通块中点的数量时
3206 // size_[find(a)] += size_[find(b)]。
3207
3208 #include<iostream>
3209 using namespace std;
3210
3211 const int N = 100010;
3212
3213 int n, m;
3214 int p[N], size[N];
3215 // 怎么样让 size 有意义，那就是只看根节点的 size
3216 // 根节点的 size 怎么更新？
3217 // 比如把 a 的根节点连到 b 上，那么就让 size[b] += size[a];
3218
3219 int find(int x) // 返回 x 的祖宗节点 + 路径压缩优化
3220 {
3221     if(p[x] != x) p[x] = find(p[x]); // 并查集最关键的两行
3222     return p[x];
3223 }
3224
3225 int main()
3226 {
3227     scanf("%d%d", &n, &m);
3228
3229     for(int i = 1; i <= n; i++)
3230     {
3231         p[i] = i;
3232         size[i] = 1; // 一开始每个集合只有一个元素，所以 size = 1
3233     }
3234
3235     while(m--)
3236     {
3237         char op[2];
3238         int a, b;
3239         scanf("%s", op);
3240
3241         if(op[0] == 'C')
3242         {
3243             scanf("%d%d", &a, &b);
3244             // // 特别是带有 size 这种附加属性的问题，一定要注意防止重复合并
3245             if(find(a) == find(b)) continue;
3246             else
3247             {

```



加油喵! QwQ !

```
3248         size[find(b)] += size[find(a)]; // 这是用来维护 size 所添加的代
3249 码, 解释见 size[N] 数组定义处
3250         p[find(a)] = find(b);
3251     }
3252 }
3253 else if(op[1] == '1')
3254 {
3255     scanf("%d%d", &a, &a);
3256     if(find(a) == find(b)) puts("Yes");
3257     else puts("No");
3258 }
3259 else
3260 {
3261     scanf("%d", &a);
3262     printf("%d\n", size[find(a)]); // size 打印代码
3263 }
3264 }
3265
3266 return 0;
3267 }
3268
3269
3270
3271
3272
3273
3274
3275
3276
3277
3278
3279
3280
3281
3282
3283
3284
3285
3286
3287
3288
3289
3290
3291
```



堆

3292

3293

3294 // 堆

3295

3296 // 如何手写一个堆? (前三个操作也是 STL 中堆直接支持的操作)

3297 // 1. 插入一个数

3298 // 2. 求集合中的最小值

3299 // 3. 删除最小值

3300 // 4. 删除任意一个元素

3301 // 5. 修改任意一个元素

3302

3303 // 堆 是一棵二叉树, 是一颗完全二叉树 (除了最后一层节点之外, 上面所有节点都是满的,
3304 最后一层节点是从左到右排列)

3305 // (STL 里的堆就是优先队列)

3306 // 小根堆: 每个点都是小于等于左右儿子的, 所以最上面的根节点就是最小值

3307

3308 // 堆的存储: 利用一维数组来存

3309 // 例如: 这是一个一维数组

3310 // 1 2 3 4 5 6 7 8 9 10

3311 // ??左儿子是 $2x$, 右儿子是 $2x+1$??

3312 // 1 的左儿子是 2, 右儿子是 3;

3313 // 2 的左儿子是 4, 右儿子是 5;

3314 // 3 的左儿子是 6, 右儿子是 7;

3315 // 4 的左儿子是 8, 右儿子是 9;

3316 // 5 的左儿子是 10;

3317 //

3318 //

3319 //

3320 //

3321 //

3322 //

3323 //

3324

3325 // 堆有 2 个操作

3326 // 第一个操作是 up(往上调整), 第二个操作是 down(往下调整);

3327

3328 // 我们可以用这 2 个操作类来实现下面 这 5 个功能

3329

3330 // 如何手写一个堆? (前三个操作也是 STL 中堆直接支持的操作)

3331 // 1. 插入一个数

3332 // `heap[ ++ size ] = x + up(size);`

3333 // 用人话翻译一下: 先把这个数插在 数组末尾 (或者说这个堆的最下面), 然后把这



```
3334 个数往上调整
3335
3336 // 2. 求集合中的最小值
3337 //     最小值就是 heap[1];
3338 //     用人话翻译一下：这个数组的最小值就是根节点
3339
3340 // 3. 删除最小值
3341 //     heap[1] = heap[size]; size--; down(1);
3342 //     用人话翻译一下：先把最小值（也就是头结点）和数组末尾的数交换位置后删掉，再
3343 把然后把交换完位置后的这个末尾的数往下调整
3344
3345 // 4. 删除任意一个元素
3346 //     heap[k] = heap[size]; size--; down(k); up(k);
3347 //     用人话翻译一下：先把要删掉的这个数和数组末尾的数交换位置后删掉，再把然后把
3348 交换完位置后的这个末尾的数向下调整或向上调整（或者干脆就像这样子先下一遍再上一遍，
3349 反正只有一个可以执行）
3350
3351 // 5. 修改任意一个元素
3352 //     heap[k] = x; down(k); up(k);
3353 //     用人话翻译一下：先直接修改这个数，再把然后把修改完的这个数向下调整或向上调
3354 整（或者干脆就像这样子先下一遍再上一遍，反正只有一个可以执行）
3355
3356
3357
3358
3359
3360
3361
3362
3363
3364
3365
3366
3367
3368
3369
3370
3371
3372
3373
3374
3375
3376
3377
```



堆排序

3378

3379

3380 // 题目描述

3381 // 输入一个长度为 n 的整数数列，从小到大输出前 m 小的数。

3382

3383 // 输入格式

3384 // 第一行包含整数 n 和 m 。

3385

3386 // 第二行包含 n 个整数，表示整数数列。

3387

3388 // 输出格式

3389 // 共一行，包含 m 个整数，表示整数数列中前 m 小的数。

3390

3391 // 数据范围

3392 // $1 \leq m \leq n \leq 10^5$,3393 // $1 \leq \text{数列中元素} \leq 10^9$

3394 // 输入样例：

3395 // 5 3

3396 // 4 5 1 3 2

3397 // 输出样例：

3398 // 1 2 3

3399

3400 #include<iostream>

3401 #include<algorithm>

3402 using namespace std;

3403

3404 const int N = 100010;

3405

3406 int n, m;

3407 int h[N], size;// size 是储存我当前这个 h[] 数组中有多少元素

3408

3409 // 时间复杂度看上去是 $O(\log n)$ ，实际上是小于并且很接近 $O(n)$ 的

3410 void down(int u)

3411 {

3412 int t = u;

3413

3414 // 这两行的目的是使 t 成为左右子节点中的更小的那个

3415 // 如果左子节点存在且小于当前最小元素，更新 t 为左子节点。

3416 if(u * 2 <= size && h[u * 2] < h[t]) t = u * 2;

3417 // 果右子节点存在且小于当前最小元素，更新 t 为右子节点。

3418 if(u * 2 + 1 <= size && h[u * 2 + 1] < h[t]) t = u * 2 + 1;

3419



加油喵! QwQ !

```
3420     if(u != t) // 如果上面 2 行代码被执行了, 那么 u 就会不等于 t , 也就是说此时 t
3421 和 u 相比, u 更小, 应该交换位置
3422     {
3423         swap(h[u], h[t]);
3424
3425         down(t); // 递归调用 down 函数, 继续向下调整交换后的节点, 确保堆的性质。
3426     }
3427 }
3428
3429 void up(int u)
3430 {
3431     while(u / 2 && h[u / 2] > h[u]) // 当 u 节点的上面还有节点 同时 u 的父节点大于
3432 u 节点本身时, 交换 2 者的位置, u 也随之更新
3433     {
3434         swap(h[u / 2], h[u]);
3435         u /= 2;
3436     }
3437 }
3438
3439 int main()
3440 {
3441     scanf("%d%d", &n, &m);
3442     for(int i = 1; i <= n; i++) scanf("%d", &h[i]);
3443     size = n;
3444
3445     for(int i = n / 2; i; i--) down(i);
3446
3447     while(m--)
3448     {
3449         printf("%d ", h[1]);
3450
3451         h[1] = h[size];
3452         size--;
3453         down(1);
3454     }
3455     return 0;
3456 }
3457
3458
3459
3460
3461
3462
3463
```



堆的 5 个操作 (带映射)

3464

3465

3466 // 题目来源: AcWing 839. 模拟堆

3467

3468 // 一、题目描述

3469 // 维护一个集合, 初始时集合为空, 支持如下几种操作:

3470 // I x, 插入一个数 x;

3471 // PM, 输出当前集合中的最小值;

3472 // DM, 删除当前集合中的最小值 (数据保证此时的最小值唯一);

3473 // D k, 删除第 k 个插入的数;

3474 // C k x, 修改第 k 个插入的数, 将其变为 x;

3475 // 现在要进行 N 次操作, 对于所有第 2 个操作, 输出当前集合的最小值。

3476

3477 // 输入格式

3478 // 第一行包含整数 N NN。

3479

3480 // 接下来 N 行, 每行包含一个操作指令, 操作指令为 I x, PM, DM, D k 或 C k x 中的
3481 一种。

3482

3483 // 输出格式

3484 // 对于每个输出指令 PM, 输出一个结果, 表示当前集合中的最小值。

3485

3486 // 每个结果占一行。

3487

3488 // 数据范围

3489 // 1 N 10^5 3490 // -10^9 x 10^9

3491 // 数据保证合法。

3492

3493 // 输入样例:

3494 // 8

3495 // I -10

3496 // PM

3497 // I -10

3498 // D 1

3499 // C 2 8

3500 // I 6

3501 // PM

3502 // DM

3503

3504 // 输出样例:

3505 // -10



```

3506 // 6
3507
3508 // 二、算法思路
3509 // 本题的基本操作依然还是来自基础堆, 现在需要做的额外的工作是如何维护第 k 次插入
3510 的数与堆中结点头下标 i 的映射关系。
3511
3512 // 首先我们定义两个数组 hp[] 和 ph[], 我们需要明白这两个数组的关系和作用: hp 是
3513 heap to pointer 的缩写, hp[i] = k 表示堆数组中的下标 i 所指的结点是第 k 次插入的。
3514 // ph 是 pointer to heap 的缩写, ph[k] = i 表示第 k 次插入的元素在堆数组中对应的
3515 结点头下标 i。
3516 // 因此, hp 和 ph 是反函数。
3517
3518 // 为什么要使用 ph 和 hp 数组呢?
3519 // 原因在于删除第 k 次插入的元素时, 我们必须知道第 k 次插入的元素在堆数组中的下
3520 标 i, 因此使用 ph 数组可以方便查找。
3521
3522 // // 删除第 k 次插入的元素
3523 // if (op == "D")
3524 // {
3525 //     int k;
3526 //     cin >> k;
3527 //     k = ph[k]; // 找到第 k 次插入的元素在堆中的下标
3528 //     heap_swap(k, n); // 第 k 次插入的元素和堆尾元素交换
3529 //     n--; // 删除堆尾元素
3530 //     up(k), down(k);
3531 // }
3532
3533 // 插入时需要完成的操作:
3534
3535 // if (op == "I")
3536 // {
3537 //     int x;
3538 //     cin >> x;
3539 //     n++, m++; // n 代表堆中元素总数, m 代表插入的次数
3540 //     ph[m] = n, hp[n] = m;
3541 //     h[n] = x; // 在结尾处插入新元素
3542 //     up(n);
3543 // }
3544 // 这样看起来, 似乎有了 ph 数组就可以完成所有操作了, 但是为什么还要有一个 hp 数
3545 组呢?
3546 // 原因在于才简单的堆的交换操作中, 我们使用的是 swap(h[a], h[b]), 传入的参数 a,
3547 b 都是堆中的下标, 但是无法知道每个堆数组的下标所对应的是第几次插入, 因此需要引入
3548 hp, 便于查找。
3549 // 本质上来说 hp 的存在是为了实现交换堆中 a, b 下标元素时, 交换 ph[a] 和 ph[b]。

```



```

3550
3551 // // 所有堆内下标的交换都换成这个
3552 // void heap_swap(int a, int b)
3553 // {
3554 //     swap(ph[hp[a]], ph[hp[b]]);
3555 //     swap(hp[a], hp[b]);
3556 //     swap(h[a], h[b]);
3557 // }
3558 #include<iostream>
3559 #include<algorithm>
3560 #include<string.h>
3561 using namespace std;
3562 const int N = 100010;
3563
3564 int h[N], size;// size 是储存我当前这个 h[] 数组中有多少元素
3565 int ph[N], hp[N];
3566 // ph 数组是 pointer to heap 的缩写, 表示第 k 次插入的元素在堆数组中对应的节点下
3567 标;
3568 // hp 数组是 heap to pointer 的缩写, 表示堆数组中的下标 i 所指的节点是第 k 次插
3569 入的,
3570 // 它们是相互映射的关系, 用于辅助堆操作。
3571
3572 void heap_swap(int a, int b)
3573 {
3574     swap(ph[hp[a]], ph[hp[b]]);
3575     swap(hp[a], hp[b]);
3576     swap(h[a], h[b]);
3577 }
3578 // 时间复杂度看上去是  $O(\log n)$ , 实际上是小于并且很接近  $O(n)$  的
3579 void down(int u)
3580 {
3581     int t = u;
3582     // 这两行的目的是使 t 成为左右子节点中的更小的那个
3583     // 如果左子节点存在且小于当前最小元素, 更新 t 为左子节点。
3584     if(u * 2 <= size && h[u * 2] < h[t]) t = u * 2;
3585     // 果右子节点存在且小于当前最小元素, 更新 t 为右子节点。
3586     if(u * 2 + 1 <= size && h[u * 2 + 1] < h[t]) t = u * 2 + 1;
3587     if(u != t) // 如果上面 2 行代码被执行了, 那么 u 就会不等于 t, 也就是说此时 t
3588 和 u 相比, u 更小, 应该交换位置
3589     {
3590         heap_swap(u, t);
3591         down(t); // 递归调用 down 函数, 继续向下调整交换后的节点, 确保堆的性质。
3592     }
3593 }

```



```

3594
3595 void up(int u)
3596 {
3597     while(u / 2 && h[u / 2] > h[u]) // 当 u 节点的上面还有节点 同时 u 的父节点大于
3598     u 节点本身时, 交换 2 者的位置, u 也随之更新
3599     {
3600         heap_swap(u / 2, u);
3601         u /= 2;
3602     }
3603 }
3604
3605 int main()
3606 {
3607     int n, m = 0;
3608     scanf("%d", &n);
3609     while(n--)
3610     {
3611         char op[10];
3612         int k, x;
3613         scanf("%s", op);
3614         if(!strcmp(op, "I"))
3615         {
3616             scanf("%d", &x);
3617             size++;
3618             m++;
3619             ph[m] = size, hp[size] = m;
3620             h[size] = x;
3621             up(size);
3622         }
3623         else if(!strcmp(op, "PM")) printf("%d\n", h[1]);
3624         else if(!strcmp(op, "DM"))
3625         {
3626             heap_swap(1, size);
3627             size--;
3628             down(1);
3629         }
3630         else if(!strcmp(op, "D"))
3631         {
3632             scanf("%d", &k);
3633             k = ph[k];
3634             heap_swap(k, size);
3635             size--;
3636             down(k), up(k);
3637         }

```



加油喵！ QwQ ！

```
3638         else
3639         {
3640             scanf("%d%d", &k, &x);
3641             k = ph[k];
3642             h[k] = x;
3643             down(k), up(k);
3644         }
3645     }
3646     return 0;
3647 }
3648
3649
3650
3651
3652
3653
3654
3655
3656
3657
3658
3659
3660
3661
3662
3663
3664
3665
3666
3667
3668
3669
3670
3671
3672
3673
3674
3675
3676
3677
3678
3679
3680
```



哈希表-开放寻址法

3681

3682

3683 // acWing- 25.Hash(哈希)表

3684

3685 // 哈希表 (Hash 表、散列表)

3686 // 散列表 (Hash table, 也叫哈希表), 是根据关键码值(Key value)而直接进行访问
3687 的数据结构。也就是说, 它通过把关键码值映射到表中一个位置来访问记录, 以加快查找的
3688 速度。

3689

3690 // 哈希表

3691 // 1. 存储结构

3692 // (1). 开放寻址法

3693 // 开放寻址法处理冲突的方式是, 如果这个数据所得的哈希值的位置已经被
3694 用过了, 那就从这个位置开始往后找, 直到找到空位直接把数据放进去。要实现这样的操作
3695 就要把数组开到我们规定数据范围的两到三倍, 用来尽可能避免冲突, 这样的好处是可以只
3696 建立一个 ha 数组。

3697 // (2). 拉链法

3698 // 2. 字符串哈希方式

3699

3700 // 什么情况下需要用到哈希表?

3701 // 答: 将一堆非常庞大的数据映射到一个比较小的范围

3702

3703 // 哈希表的操作一般只有 插入 和 查找 2 种, 如果非要删除, 那么就创建一个 bool 数组
3704 标记一下该值被删除了就好了 (一般是不会真正去删除这个元素的)

3705

3706 // 一般来说我们所用到的求哈希值的办法就是直接对我们数组的最大的数据范围取模。

3707 // (有个小 tip 就是我们最好对大于数据范围 2 倍 ~ 3 倍 的最大的质数进行取模, 比如这
3708 道题里最大数据范围是 $1e5$, 那我知道大于 $2e5$ 的最大指数是 $2e5+3$, 因此我的常数 N 就设
3709 为 $1e5+3$ 。这样可以最大限度的减少冲突的发生。)

3710 // 求大于 $2e5$ 的最大质数

3711 // 附上求大于数据范围的最大质数的代码

3712 // #include<iostream>

3713 // using namespace std;

3714 // int main()

3715 // {

3716 // for(int i=100000*2;;i++)

3717 // {

3718 // int flag=1;

3719 // for(int j=2;j*j<=i;j++)

3720 // {

3721 // if(i%j==0){

3722 // flag=0;




```

3723 //          break;
3724 //      }
3725 //  }
3726 //      if(flag){
3727 //          cout<<i;
3728 //          break;
3729 //      }
3730 //  }
3731 //      return 0;
3732 // }
3733
3734
3735
3736 // 题目来源:AcWing 840. 模拟散列表
3737
3738 // 一、题目描述
3739 // 维护一个集合，支持如下几种操作：
3740 // I x，插入一个数 x；
3741 // Q x，询问数 x 是否在集合中出现过；
3742 // 现在要进行 N 次操作，对于每个询问操作输出对应的结果。
3743
3744 // 输入格式
3745 // 第一行包含整数 N，表示操作数量。
3746 // 接下来 N 行，每行包含一个操作指令，操作指令为 I x，Q x 中的一种。
3747
3748 // 输出格式
3749 // 对于每个询问指令 Q x，输出一个询问结果，如果 x 在集合中出现过，则输出 Yes，否
3750 // 则输出 No。
3751 // 每个结果占一行。
3752
3753 // 数据范围
3754 // 1 ≤ N ≤ 105
3755 // -109 ≤ x ≤ 109
3756
3757 // 输入样例：
3758 // 5
3759 // I 1
3760 // I 2
3761 // I 3
3762 // Q 2
3763 // Q 5
3764
3765 // 输出样例：
3766 // Yes

```



```

3767 // No
3768
3769 // 开放寻址法
3770 // 基本思路：
3771 //      在使用开放寻址法时，通常需要定义一个数据范围之外的数作为标记，表示哈希表
3772 //      中的空位置。
3773 //      这个标记通常被称为“哨兵值”。这个哨兵值应该是一个我们知道不会出现在输入
3774 //      数据中的值。
3775 //      这样，当我们在哈希表中看到这个哨兵值时，就可以确定这个位置是空的，没有存
3776 //      储任何元素。这主要是为了处理查找操作。
3777 #include<cstring>
3778 #include<iostream>
3779 using namespace std;
3780
3781 const int N = 200003; // 为了减少冲突，尽量取模于质数，我们通过计算可以得到大于
3782 // 这题数据范围 1e5 的最小质数是 1e5+3。
3783 const int null = 0x3f3f3f3f; // 定一个不在数据范围内的“哨兵值”
3784
3785 int h[N];
3786
3787 // 查询操作
3788 int find(int x) // 如果 x 在哈希表中的话，就返回 x 的位置，否则就返回 x 应该存储的
3789 // 位置
3790 {
3791     int k = (x % N + N) % N;
3792
3793     while(h[k] != null && h[k] != x)
3794     {
3795         k++;
3796         if(k == N) k = 0; // 如果最后一个“坑位”已经没了的话，就返回看第一个坑
3797 // 位
3798     }
3799
3800     return k;
3801 }
3802
3803 int main()
3804 {
3805     int n;
3806     scanf("%d", &n);
3807
3808     memset(h, 0x3f, sizeof h); // 将 h 数组初始化
3809     // memset 函数简单了解一下，暂时够用就行。
3810     // 两种方式都是用到了 memset 函数来进行数组的初始化

```



加油喵! QwQ !

```
3811 // 他是一个一般是快速的初始化数组的一个函数, 使用该函数时要引头文
3812 件 #include<cstring>
3813 // 它每次以 一个字节 为单位进行赋值, 因此才出现在开放寻址法里 null 应该是
3814 0x3f3f3f3f, 但是在 memset 函数里只写了 0x3f 的原因
3815
3816 while(n--)
3817 {
3818     char op[2]; // 老样子: 养成习惯, 读入一个字符时创建一个字符串来防止 (题
3819 目中) 该字符后面可能出现的空格
3820     int x;
3821     scanf("%s%d", op, &x);
3822
3823     if(*op == 'I')
3824     {
3825         int k = find(x);
3826         h[k] = x;
3827     }
3828     else
3829     {
3830         int k = find(x);
3831         if(h[k] != null) puts("Yes");
3832         else puts("No");
3833     }
3834
3835 }
3836
3837 return 0;
3838 }
3839
3840
3841
3842
3843
3844
3845
3846
3847
3848
3849
3850
3851
3852
3853
3854
```



哈希表-拉链法

3855

3856

3857 // acWing- 25.Hash(哈希)表

3858

3859 // 哈希表 (Hash 表、散列表)

3860 // 散列表 (Hash table, 也叫哈希表), 是根据关键码值(Key value)而直接进行访问
 3861 的数据结构。也就是说, 它通过把关键码值映射到表中一个位置来访问记录, 以加快查找的
 3862 速度。

3863

3864 // 哈希表

3865 // 1. 存储结构

3866 // (1). 开放寻址法

3867 // 开放寻址法处理冲突的方式是, 如果这个数据所得的哈希值的位置已经被
 3868 用过了, 那就从这个位置开始往后找, 直到找到空位直接把数据放进去。要实现这样的操作
 3869 就要把数组开到我们规定数据范围的两到三倍, 用来尽可能避免冲突, 这样的好处是可以只
 3870 建立一个 ha 数组。

3871 // (2). 拉链法

3872 // 2. 字符串哈希方式

3873

3874 // 什么情况下需要用到哈希表?

3875 // 答: 将一堆非常庞大的数据映射到一个比较小的范围

3876

3877 // 哈希表的操作一般只有 插入 和 查找 2 种, 如果非要删除, 那么就创建一个 bool 数组
 3878 标记一下该值被删除了就好了 (一般是不会真正去删除这个元素的)

3879

3880 // 一般来说我们所用到的求哈希值的办法就是直接对我们数组的最大的数据范围取模。

3881 // (有个小 tip 就是我们最好对大于数据范围 2 倍 ~ 3 倍 的最大的质数进行取模, 比如这
 3882 道题里最大数据范围是 $1e5$, 那我知道大于 $2e5$ 的最大指数是 $2e5+3$, 因此我的常数 N 就设
 3883 为 $1e5+3$ 。这样可以最大限度的减少冲突的发生。)

3884 // 求大于 $2e5$ 的最大质数

3885 // 附上求大于数据范围的最大质数的代码

3886 // #include<iostream>

3887 // using namespace std;

3888 // int main()

3889 // {

3890 // for(int i=100000*2;;i++)

3891 // {

3892 // int flag=1;

3893 // for(int j=2;j*j<=i;j++)

3894 // {

3895 // if(i%j==0){

3896 // flag=0;



```

3897 //          break;
3898 //      }
3899 //  }
3900 //      if(flag){
3901 //          cout<<i;
3902 //          break;
3903 //      }
3904 //  }
3905 //      return 0;
3906 // }
3907
3908
3909
3910 // 题目来源:AcWing 840. 模拟散列表
3911
3912 // 一、题目描述
3913 // 维护一个集合, 支持如下几种操作:
3914 // I x, 插入一个数 x;
3915 // Q x, 询问数 x 是否在集合中出现过;
3916 // 现在要进行 N 次操作, 对于每个询问操作输出对应的结果。
3917
3918 // 输入格式
3919 // 第一行包含整数 N, 表示操作数量。
3920 // 接下来 N 行, 每行包含一个操作指令, 操作指令为 I x, Q x 中的一种。
3921
3922 // 输出格式
3923 // 对于每个询问指令 Q x, 输出一个询问结果, 如果 x 在集合中出现过, 则输出 Yes, 否
3924 // 则输出 No。
3925 // 每个结果占一行。
3926
3927 // 数据范围
3928 // 1 ≤ N ≤ 105
3929 // -109 ≤ x ≤ 109
3930
3931 // 输入样例:
3932 // 5
3933 // I 1
3934 // I 2
3935 // I 3
3936 // Q 2
3937 // Q 5
3938
3939 // 输出样例:
3940 // Yes

```



```

3941 // No
3942
3943 // 开放寻址法
3944 // 基本思路：
3945 //      在使用开放寻址法时，通常需要定义一个数据范围之外的数作为标记，表示哈希表
3946 //      中的空位置。
3947 //      这个标记通常被称为“哨兵值”。这个哨兵值应该是一个我们知道不会出现在输入
3948 //      数据中的值。
3949 //      这样，当我们在哈希表中看到这个哨兵值时，就可以确定这个位置是空的，没有存
3950 //      储任何元素。这主要是为了处理查找操作。
3951 #include<cstring>
3952 #include<iostream>
3953 using namespace std;
3954
3955 const int N = 200003; // 为了减少冲突，尽量取模于质数，我们通过计算可以得到大于
3956 // 这题数据范围 1e5 的最小质数是 1e5+3。
3957 const int null = 0x3f3f3f3f; // 定一个不在数据范围内的“哨兵值”
3958
3959 int h[N];
3960
3961 // 查询操作
3962 int find(int x) // 如果 x 在哈希表中的话，就返回 x 的位置，否则就返回 x 应该存储的
3963 // 位置
3964 {
3965     int k = (x % N + N) % N;
3966
3967     while(h[k] != null && h[k] != x)
3968     {
3969         k++;
3970         if(k == N) k = 0; // 如果最后一个“坑位”已经没了的话，就返回看第一个坑
3971 // 位
3972     }
3973
3974     return k;
3975 }
3976
3977 int main()
3978 {
3979     int n;
3980     scanf("%d", &n);
3981
3982     memset(h, 0x3f, sizeof h); // 将 h 数组初始化
3983     // memset 函数简单了解一下，暂时够用就行。
3984     // 两种方式都是用到了 memset 函数来进行数组的初始化

```



加油喵! QwQ !

```
3985 // 他是一个一般是快速的初始化数组的一个函数, 使用该函数时要引头文
3986 件 #include<cstring>
3987 // 它每次以 一个字节 为单位进行赋值, 因此才出现在开放寻址法里 null 应该是
3988 0x3f3f3f3f, 但是在 memset 函数里只写了 0x3f 的原因
3989
3990 while(n--)
3991 {
3992     char op[2]; // 老样子: 养成习惯, 读入一个字符时创建一个字符串来防止 (题
3993 目中) 该字符后面可能出现的空格
3994     int x;
3995     scanf("%s%d", op, &x);
3996
3997     if(*op == 'I')
3998     {
3999         int k = find(x);
4000         h[k] = x;
4001     }
4002     else
4003     {
4004         int k = find(x);
4005         if(h[k] != null) puts("Yes");
4006         else puts("No");
4007     }
4008 }
4009 return 0;
4010 }
4011
4012
4013
4014
4015
4016
4017
4018
4019
4020
4021
4022
4023
4024
4025
4026
4027
4028
```



深度优先搜索（八皇后）

4029

4030

4031 // 深度优先搜索(DFS) && 广度优先搜索(BFS)

4032

4033 // 深度优先搜索

4034

4035 // 题目：八皇后问题

4036

4037 // n 皇后问题是指将 n 个皇后放在 $n \times n$ 的国际象棋棋盘上，使得皇后不能相互攻击到，
 4038 即任意两个皇后都不能处于同一行、同一列或同一斜线上。

4039 // 现在给定整数 n ，请你输出所有的满足条件的棋子摆法。

4040

4041 // 输入格式

4042 // 共一行，包含整数 n 。

4043

4044 // 输出格式

4045 // 每个解决方案占 n 行，每行输出一个长度为 n 的字符串，用来表示完整的棋盘状态。

4046 // 其中“.”表示某一个位置的方格状态为空，“Q”表示某一个位置的方格上摆着皇后。

4047 // 每个方案输出完成后，输出一个空行。

4048

4049 // 数据范围

4050 // $1 \leq n \leq 9$

4051

4052 // 输入样例：

4053 // 4

4054

4055 // 输出样例：

4056 // .Q..

4057 // ...Q

4058 // Q...

4059 // ..Q.

4060

4061 // ..Q.

4062 // Q...

4063 // ...Q

4064 // .Q..

4065

4066 // 第一种搜索方法（按行搜索）

4067

4068 #include<iostream>

4069 using namespace std;

4070




```
4071  const int N = 20;
4072
4073  int n;
4074  char g[N][N];
4075  bool col[N], dg[N], udg[N];
4076
4077  void dfs(int u)
4078  {
4079      if(u == n)
4080      {
4081          for (int i = 0; i < n; i++) puts(g[i]);
4082          puts("");
4083          return;
4084      }
4085      for(int i = 0; i < n; i++)
4086      {
4087          if(!col[i] && !dg[u + i] && !udg[n - u + i])
4088          {
4089              g[u][i] = 'Q';
4090              col[i] = dg[u + i] = udg[n - u + i] = true;
4091              dfs(u + 1);
4092              g[u][i] = '.';
4093              col[i] = dg[u + i] = udg[n - u + i] = false;
4094          }
4095      }
4096  }
4097
4098
4099  int main()
4100  {
4101      cin >> n;
4102      for(int i = 0; i < n; i++)
4103      {
4104          for(int j = 0; j < n; j++)
4105          {
4106              g[i][j] = '.';
4107          }
4108      }
4109
4110      dfs(0);
4111
4112      return 0;
4113  }
4114
```



```

4115 // 第二种搜索方法 (按格搜索)
4116
4117 // #include<iostream>
4118
4119 // using namespace std;
4120
4121 // const int N = 20;
4122
4123 // int n;
4124 // char g[N][N];
4125 // bool row[N], col[N], dg[N], udg[N];
4126
4127 // void dfs(int x, int y, int s) // x, y 分别表示当前处理的格子的行和列坐标。s 表
4128 示已经放置的皇后数量
4129 // {
4130 //     if(y == n) y = 0, x++;
4131
4132 //     if(x == n)
4133 //     {
4134 //         if(s == n)
4135 //         {
4136 //             for(int i = 0; i < n; i++) puts(g[i]);
4137 //             puts("");
4138 //         }
4139 //         return;
4140 //     }
4141
4142 //     // 不放皇后
4143 //     dfs(x, y + 1, s);
4144
4145 //     // 放皇后
4146 //     if(!row[x] && !col[y] && !dg[x + y] && !udg[x - y + n])
4147 //     {
4148 //         g[x][y] = 'Q';
4149 //         row[x] = col[y] = dg[x + y] = udg[x - y + n] = true;
4150 //         dfs(x, y + 1, s + 1);
4151 //         g[x][y] = '.';
4152 //         row[x] = col[y] = dg[x + y] = udg[x - y + n] = false;
4153 //     }
4154 // }
4155
4156 // int main()
4157 // {
4158 //     cin >> n;

```



```

4159 //     for(int i = 0; i < n; i++)
4160 //     {
4161 //         for(int j = 0; j < n; j++)
4162 //         {
4163 //             g[i][j] = '.';
4164 //         }
4165 //     }
4166
4167 //     dfs(0, 0, 0);
4168
4169 //     return 0;
4170 // }
4171
4172 // ai 注释版
4173
4174 //     第一种
4175
4176 // #include<iostream>
4177 // using namespace std;
4178
4179 // // 定义常量 N，用于表示棋盘的最大规模，这里假设棋盘最大为 20x20
4180 // const int N = 20;
4181
4182 // // n 表示棋盘的大小，即 n x n 的棋盘，同时也是皇后的数量
4183 // int n;
4184 // // g 是一个二维字符数组，用于存储棋盘的状态，'.' 表示空位，'Q' 表示皇后
4185 // char g[N][N];
4186 // // col 数组用于标记每一列是否已经放置了皇后，true 表示该列已有皇后
4187 // bool col[N];
4188 // // dg 数组用于标记正对角线（从左上到右下）是否已经放置了皇后
4189 // bool dg[N];
4190 // // udg 数组用于标记反对角线（从右上到左下）是否已经放置了皇后
4191 // bool udg[N];
4192
4193 // // 深度优先搜索函数，u 表示当前正在处理的行号
4194 // void dfs(int u)
4195 // {
4196 //     // 当 u 等于 n 时，说明已经成功处理完了所有的行，即找到了一种合法的皇后放
4197 //     置方案
4198 //     if(u == n)
4199 //     {
4200 //         // 遍历棋盘的每一行
4201 //         for (int i = 0; i < n; i++)
4202 //             // 输出当前行的棋盘状态

```



```

4203 //          puts(g[i]);
4204 //          // 输出一个空行, 用于分隔不同的方案
4205 //          puts("");
4206 //          // 回溯, 继续寻找其他可能的方案
4207 //          return;
4208 //      }
4209
4210 //      // 遍历当前行的每一列
4211 //      for(int i = 0; i < n; i++)
4212 //      {
4213 //          // 检查当前列、当前正对角线和当前反对角线是否都没有皇后
4214 //          if(!col[i] && !dg[u + i] && !udg[n - u + i])
4215 //          {
4216 //              // 如果都没有皇后, 则在当前位置放置一个皇后
4217 //              g[u][i] = 'Q';
4218 //              // 标记当前列已经有皇后
4219 //              col[i] = true;
4220 //              // 标记当前正对角线已经有皇后
4221 //              dg[u + i] = true;
4222 //              // 标记当前反对角线已经有皇后
4223 //              udg[n - u + i] = true;
4224 //              // 递归调用 dfs 函数, 处理下一行
4225 //              dfs(u + 1);
4226 //              // 回溯操作, 撤销当前位置的皇后放置
4227 //              g[u][i] = '.';
4228 //              // 撤销当前列的标记
4229 //              col[i] = false;
4230 //              // 撤销当前正对角线的标记
4231 //              dg[u + i] = false;
4232 //              // 撤销当前反对角线的标记
4233 //              udg[n - u + i] = false;
4234 //          }
4235 //      }
4236 // }
4237
4238 // int main()
4239 // {
4240 //     // 从标准输入读取棋盘的大小 n
4241 //     cin >> n;
4242 //     // 初始化棋盘, 将所有位置都置为 '.', 表示空位
4243 //     for(int i = 0; i < n; i++)
4244 //     {
4245 //         for(int j = 0; j < n; j++)
4246 //         {

```



```

4247 //          g[i][j] = '.';
4248 //      }
4249 //  }
4250 //  // 从第 0 行开始进行深度优先搜索
4251 //  dfs(0);
4252 //  // 程序正常结束
4253 //  return 0;
4254 // }
4255
4256 //      第二种
4257
4258 // #include<iostream>
4259 // using namespace std;
4260 // // 定义常量 N, 用于表示棋盘的最大规模, 这里假设棋盘最大为 20x20
4261 // const int N = 20;
4262
4263 // // n 表示棋盘的大小, 即 n x n 的棋盘, 同时也是皇后的数量
4264 // int n;
4265 // // g 是一个二维字符数组, 用于存储棋盘的状态, '.' 表示空位, 'Q' 表示皇后
4266 // char g[N][N];
4267 // // row 数组用于标记每一行是否已经放置了皇后, true 表示该行已有皇后
4268 // bool row[N];
4269 // // col 数组用于标记每一列是否已经放置了皇后, true 表示该列已有皇后
4270 // bool col[N];
4271 // // dg 数组用于标记正对角线 (从左上到右下) 是否已经放置了皇后
4272 // bool dg[N];
4273 // // udg 数组用于标记反对角线 (从右上到左下) 是否已经放置了皇后
4274 // bool udg[N];
4275
4276 // // 深度优先搜索函数, x 和 y 表示当前正在处理的格子的坐标, s 表示已经放置的皇后数量
4277 // void dfs(int x, int y, int s)
4278 // {
4279 //     // 如果当前列 y 等于 n, 说明已经处理完了当前行, 需要转到下一行的第 0 列
4280 //     if(y == n)
4281 //     {
4282 //         y = 0;
4283 //         x++;
4284 //     }
4285 //
4286 //     // 如果当前行 x 等于 n, 说明已经处理完了所有的格子
4287 //     if(x == n)
4288 //     {
4289 //         // 检查已经放置的皇后数量是否等于 n, 如果等于 n, 说明找到了一种合法的
4290 //

```



```

4291  方案
4292  //      if(s == n)
4293  //      {
4294  //          // 遍历棋盘的每一行
4295  //          for(int i = 0; i < n; i++)
4296  //              // 输出当前行的棋盘状态
4297  //              puts(g[i]);
4298  //          // 输出一个空行, 用于分隔不同的方案
4299  //          puts("");
4300  //      }
4301  //      // 回溯, 继续寻找其他可能的方案
4302  //      return;
4303  //  }
4304
4305  //      // 情况一: 当前格子不放置皇后, 直接处理下一个格子
4306  //      dfs(x, y + 1, s);
4307
4308  //      // 情况二: 尝试在当前格子放置皇后
4309  //      // 检查当前行、当前列、当前正对角线和当前反对角线是否都没有皇后
4310  //      if(!row[x] && !col[y] && !dg[x + y] && !udg[x - y + n])
4311  //      {
4312  //          // 如果都没有皇后, 则在当前位置放置一个皇后
4313  //          g[x][y] = 'Q';
4314  //          // 标记当前行已经有皇后
4315  //          row[x] = true;
4316  //          // 标记当前列已经有皇后
4317  //          col[y] = true;
4318  //          // 标记当前正对角线已经有皇后
4319  //          dg[x + y] = true;
4320  //          // 标记当前反对角线已经有皇后
4321  //          udg[x - y + n] = true;
4322  //          // 递归调用 dfs 函数, 处理下一个格子, 同时皇后数量加 1
4323  //          dfs(x, y + 1, s + 1);
4324  //          // 回溯操作, 撤销当前位置的皇后放置
4325  //          g[x][y] = '.';
4326  //          // 撤销当前行的标记
4327  //          row[x] = false;
4328  //          // 撤销当前列的标记
4329  //          col[y] = false;
4330  //          // 撤销当前正对角线的标记
4331  //          dg[x + y] = false;
4332  //          // 撤销当前反对角线的标记
4333  //          udg[x - y + n] = false;
4334  //      }

```



```
4335 // }
4336
4337 // int main()
4338 // {
4339 //     // 从标准输入读取棋盘的大小 n
4340 //     cin >> n;
4341 //     // 初始化棋盘, 将所有位置都置为 '.', 表示空位
4342 //     for(int i = 0; i < n; i++)
4343 //     {
4344 //         for(int j = 0; j < n; j++)
4345 //         {
4346 //             g[i][j] = '.';
4347 //         }
4348 //     }
4349 //     // 从第 0 行第 0 列开始进行深度优先搜索, 初始皇后数量为 0
4350 //     dfs(0, 0, 0);
4351 //     // 程序正常结束
4352 //     return 0;
4353 // }
4354
4355
4356
4357
4358
4359
4360
4361
4362
4363
4364
4365
4366
4367
4368
4369
4370
4371
4372
4373
4374
4375
4376
4377
4378
```



深度优先搜索（全排列）

4379

4380

4381 // 深度优先搜索(DFS) && 广度优先搜索(BFS)

4382

4383 // 深度优先搜索

4384

4385 // 题目：排列数字

4386

4387 // 题目描述

4388 // 给定一个整数 n ，将数字 $1 \sim n$ 排成一排，将会有很多种排列方法。

4389 // 现在，请你按照字典序将所有的排列方法输出。

4390

4391 // 输入格式

4392 // 共一行，包含一个整数 n 。

4393

4394 // 输出格式

4395 // 按字典序输出所有排列方案，每个方案占一行。

4396

4397 // 数据范围

4398 // $1 \leq n \leq 7$

4399

4400 // 输入样例

4401 // 3

4402

4403 // 输出样例

4404 // 1 2 3

4405 // 1 3 2

4406 // 2 1 3

4407 // 2 3 1

4408 // 3 1 2

4409 // 3 2 1

4410

4411 #include<iostream>

4412 using namespace std;

4413

4414 const int N = 10;

4415

4416 int n;

4417 int path[N];

4418 bool st[N];

4419

4420 void dfs(int u)



加油喵! QwQ !

```
4421 {
4422     if(u == n)
4423     {
4424         for(int i = 0; i < n; i++) printf("%d ", path[i]);
4425         puts("");
4426         return;
4427     }
4428
4429     for(int i = 1; i <= n; i++)
4430     {
4431         if(!st[i])
4432         {
4433             path[u] = i;
4434             st[i] = true;
4435             dfs(u + 1);
4436             path[u] = 0;
4437             st[i] = false;
4438         }
4439     }
4440 }
4441
4442 int main()
4443 {
4444     cin >> n;
4445
4446     dfs(0);
4447
4448     return 0;
4449 }
4450
4451
4452
4453
4454
4455
4456
4457
4458
4459
4460
4461
4462
4463
4464
```



广度优先搜索

4465

4466

4467 // 深度优先搜索(DFS) && 广度优先搜索(BFS)

4468

4469 // 广度优先搜索

4470

4471 // 深搜是没有模板的, 但是宽搜有

4472 // 不是所有的最短路问题都可以用 BFS 来做。只有所有边的权重都一样时才可以, 有专门的最短路算法

4474 // BFS (宽搜), 可以看成一种特殊的 DP

4475

4476 // acwing 844. 走迷宫

4477 // 给定一个 $n \times m$ 的二维整数数组, 用来表示一个迷宫, 数组中只包含 0 或 1, 其中 0 表示可以走的路, 1 表示不可通过的墙壁。

4479 // 最初, 有一个人位于左上角(1, 1)处, 已知该人每次可以向上、下、左、右任意一个方向移动一个位置。

4481 // 请问, 该人从左上角移动至右下角(n, m)处, 至少需要移动多少次。4482 // 数据保证(1, 1)处和(n, m)处的数字为 0, 且一定至少存在一条通路。

4483

4484 // 输入格式

4485 // 第一行包含两个整数 n 和 m 。4486 // 接下来 n 行, 每行包含 m 个整数 (0 或 1), 表示完整的二维数组迷宫。

4487

4488 // 输出格式

4489 // 输出一个整数, 表示从左上角移动至右下角的最少移动次数。

4490

4491 // 数据范围

4492 // 1 n, m 100

4493

4494 // 输入样例:

4495 // 5 5

4496 // 0 1 0 0 0

4497 // 0 1 0 1 0

4498 // 0 0 0 0 0

4499 // 0 1 1 1 0

4500 // 0 0 0 1 0

4501

4502 // 输出样例:

4503 // 8

4504

4505 #include<iostream>

4506 #include<algorithm>



```

4507 #include<cstring>
4508
4509 using namespace std;
4510
4511 typedef pair<int, int> PII;
4512
4513 const int N = 110;
4514
4515 int n, m;
4516 int g[N][N]; // g 数组存的是这个地图
4517 int d[N][N]; // d 数组存的是每个点到起点的距离
4518 PII q[N * N], Prev[N][N]; // Prev 数组是用来记录路径的
4519
4520 int bfs()
4521 {
4522     int hh = 0, tt = 0; // hh 是队头, tt 是队尾
4523     q[0] = {0, 0};
4524
4525     memset(d, -1, sizeof d); // 先将 d 数组初始化为-1
4526
4527     d[0][0] = 0; // 表示起点走过了 (起点到起点的距离为 0)
4528
4529     int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1}; // x 方向的向量和 y 方向的
4530     向量组成的上、右、下、左
4531
4532     while(hh <= tt)
4533     {
4534         auto t = q[hh ++];
4535
4536         for(int i = 0; i < 4; i++)
4537         {
4538             int x = t.first + dx[i]; // x 表示沿着此方向走会走到哪个点
4539             int y = t.second + dy[i]; // y 表示沿着此方向走会走到哪个点
4540
4541             if(x >= 0 && x < n && y >= 0 && y < m && g[x][y] == 0 && d[x][y] == -1)
4542             // 在边界内 并且是空地可以走 且之前没有走过
4543             {
4544                 d[x][y] = d[t.first][t.second] + 1;
4545                 Prev[x][y] = t; // 记录每个点是从哪个点走来的
4546                 q[++ tt] = {x, y};
4547             }
4548         }
4549     }
4550

```



加油喵! QwQ !

```
4551 // 这下面这几行是输出路径 (是从终点回溯到起点, 所以输出的路径是逆序的)
4552 int x = n - 1, y = m - 1;
4553 while(x || y)
4554 {
4555     cout << x << " " << y << endl;
4556     auto t = Prev[x][y];
4557     x = t.first, y = t.second;
4558 }
4559
4560 return d[n - 1][m - 1];
4561 }
4562
4563 int main()
4564 {
4565     cin >> n >> m;
4566
4567     for(int i = 0; i < n; i++)
4568         for(int j = 0; j < m; j++)
4569             cin >> g[i][j];
4570
4571     cout << bfs() << endl;
4572
4573     return 0;
4574 }
4575
4576
4577
4578
4579
4580
4581
4582
4583
4584
4585
4586
4587
4588
4589
4590
4591
4592
4593
4594
```



数和图的存储

4595

4596

4597 // 数和图是怎么存储的?

4598 // 树是一种特殊的图, 无环连通图

4599

4600 // 图分为两种, 有向图和无向图 (无向图可以看作是特殊的有向图, 所以我们只需要了解
4601 有向图就可以了)

4602

4603 // 有向图 ($a \rightarrow b$)

4604 // 1. 邻接矩阵 (就是一个 2 维数组, 适合储存稠密图, 比较浪费空间)

4605 // 2. 邻接链表 ()

4606

4607 // 此代码利用邻接表存储图的结构。邻接表是一种常用的图的存储方式, 适用于稀疏图 (边
4608 的数量远小于顶点数量的平方), 它可以高效地存储图中顶点之间的连接关系。

4609

4610 #include<iostream>

4611 #include&ltcstring>

4612 #include<algorithm>

4613

4614 using namespace std;

4615

4616 // 定义常量 N 为图中顶点的最大数量

4617 // 这里假设图最多有 100010 个顶点

4618 const int N = 100010;

4619 // 定义常量 M 为图中边的最大数量

4620 // 由于无向图的每条边会在邻接表中存储两次, 所以边的最大数量设为顶点最大数量的 2
4621 倍

4622 const int M = N * 2;

4623

4624 // h 数组是邻接表的头数组, h[i] 存储顶点 i 的邻接表的头指针

4625 // 初始时, 所有顶点的邻接表为空, 后续添加边时会更新该数组

4626 int h[N];

4627 // e 数组用于存储每条边的终点

4628 // 对于每一条边 (a, b), e 数组会记录边的终点 b

4629 int e[M];

4630 // ne 数组用于存储下一条边的索引

4631 // 通过 ne 数组可以将同一个顶点的所有邻接边串成一个链表

4632 int ne[M];

4633 // idx 是边的索引计数器, 用于记录当前可用的边的编号

4634 // 初始值为 0, 每添加一条新边, idx 就会自增 1

4635 int idx;

4636



```

4637 // 向图中添加一条从顶点 a 到顶点 b 的边的函数
4638 void add(int a, int b) {
4639     // 将边的终点 b 存储到 e 数组的当前 idx 位置
4640     e[idx] = b;
4641     // 将当前边的下一条边的索引设为顶点 a 当前邻接表的头指针
4642     // 也就是把新边插入到顶点 a 邻接表的头部
4643     ne[idx] = h[a];
4644     // 更新顶点 a 的邻接表的头指针为当前边的索引
4645     h[a] = idx;
4646     // 边的索引计数器 idx 自增 1, 为下一次添加边做准备
4647     idx++;
4648 }
4649
4650 // 打印图的邻接表的函数, 参数 n 表示图中顶点的数量
4651 void printGraph(int n) {
4652     // 遍历图中的每个顶点
4653     for (int i = 1; i <= n; i++) {
4654         // 输出当前顶点的编号
4655         cout << "顶点 " << i << " 的邻接顶点: ";
4656         // 从顶点 i 的邻接表的头指针开始遍历其邻接边
4657         for (int j = h[i]; j != -1; j = ne[j]) {
4658             // 输出当前边的终点, 即顶点 i 的一个邻接顶点
4659             cout << e[j] << " ";
4660         }
4661         // 换行, 为下一个顶点的邻接顶点输出做准备
4662         cout << endl;
4663     }
4664 }
4665
4666 int main() {
4667     // 使用 memset 函数将 h 数组的所有元素初始化为 -1
4668     // -1 表示该顶点的邻接表为空, 没有邻接边
4669     memset(h, -1, sizeof h);
4670
4671     // 向图中添加边
4672     // 添加从顶点 1 到顶点 2 的边
4673     add(1, 2);
4674     // 添加从顶点 1 到顶点 3 的边
4675     add(1, 3);
4676     // 添加从顶点 2 到顶点 3 的边
4677     add(2, 3);
4678
4679     // 调用 printGraph 函数打印图的邻接表
4680     // 这里假设图中有 3 个顶点

```



```

4681     printGraph(3);
4682
4683     // 程序正常结束, 返回 0
4684     return 0;
4685 }
4686
4687 // 树和图的遍历-宽度优先搜索
4688
4689 // AcWing 848. 有向图的拓扑序列 : 拓扑序列就是从前指向后的边的序列 (不是所有边都
4690 有拓扑序列的) (可以证明一个有向无环图是一定存在一个拓扑序列的, 所以有向无环图也
4691 被称为拓扑图)
4692
4693 // 题目描述
4694 // 给定一个  $n$  个点  $m$  条边的有向图, 点的编号是 1 到  $n$ , 图中可能存在重边和自环。
4695 // 请输出任意一个该有向图的拓扑序列, 如果拓扑序列不存在, 则输出 -1。
4696 // 若一个由图中所有点构成的序列  $A$  满足: 对于图中的每条边  $(x, y)$ ,  $x$  在  $A$  中都出现
4697 在  $y$  之前, 则称  $A$  是该图的一个拓扑序列。
4698
4699 // 输入格式
4700 // 第一行包含两个整数  $n$  和  $m$ 。
4701 // 接下来  $m$  行, 每行包含两个整数  $x$  和  $y$ , 表示存在一条从点  $x$  到点  $y$  的有向边  $(x, y)$ 。
4702
4703 // 输出格式
4704 // 共一行, 如果存在拓扑序列, 则输出任意一个合法的拓扑序列即可。
4705 // 否则输出 -1。
4706 // 数据范围
4707 //  $1 \leq n, m \leq 10^5$ 
4708 // 输入样例:
4709 // 3 3
4710 // 1 2
4711 // 2 3
4712 // 1 3
4713 // 输出样例:
4714 // 1 2 3
4715 // 思路: 将入度为 0 的点入列, 再删去该点指向的所有边, 再将入度为 0 的点入列, 直到
4716 所有点都入列, 若所有点都入列, 则该序列是一个拓扑序列, 否则该序列不存在拓扑序列。
4717 // 入度: 是指指向该点的边的数量 (所以所有入度为 0 的点都可以排在最前面的位置)
4718 // 出度: 是指从该点出发的边的数量
4719
4720 #include <iostream>
4721 #include <algorithm>
4722 #include <cstring>
4723 using namespace std;
4724 const int N = 100010;

```



```
4725
4726 int n, m;
4727 int h[N], e[N], ne[N], idx;
4728 int q[N], d[N];
4729
4730 void add(int a, int b)
4731 {
4732     e[idx] = b;
4733     ne[idx] = h[a];
4734     h[a] = idx;
4735     idx++;
4736 }
4737
4738 bool topsort()
4739 {
4740     int hh = 0, tt = -1;
4741     for (int i = 1; i <= n; i++)
4742         if (!d[i])
4743             q[++tt] = i;
4744     while (hh <= tt)
4745     {
4746         int t = q[hh++];
4747         for (int i = h[t]; i != -1; i = ne[i])
4748         {
4749             int j = e[i];
4750             d[j]--;
4751             if (d[j] == 0)
4752             {
4753                 q[++tt] = j;
4754             }
4755         }
4756     }
4757     return tt == n - 1;
4758 }
4759
4760 int main()
4761 {
4762     cin >> n >> m;
4763     memset(h, -1, sizeof h);
4764     for (int i = 0; i < m; i++)
4765     {
4766         int a, b;
4767         cin >> a >> b;
4768         add(a, b);
```



加油喵! QwQ !

```
4769         d[b]++;
4770     }
4771     if (topsort())
4772     {
4773         for (int i = 0; i < n; i++)
4774             cout << q[i] << ' ';
4775     }
4776     else
4777         puts("-1");
4778     return 0;
4779 }
4780
4781
4782
4783
4784
4785
4786
4787
4788
4789
4790
4791
4792
4793
4794
4795
4796
4797
4798
4799
4800
4801
4802
4803
4804
4805
4806
4807
4808
4809
4810
4811
4812
```



数和图的遍历（宽度优先遍历）

4813

4814

4815 // // 我很强，比你们都强

4816

4817 // 树和图的遍历-宽度优先搜索

4818

4819 // AcWing 847. 图中点的层次

4820 // 题目描述

4821 // 给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环。// 重边指的是重复的有
 4822 向/无向边，自环指的是自己指向自己

4823 // 所有边的长度都是 1，点的编号为 $1 \sim n$ 。// 所有边的长度都是 1，告诉我们可以用宽搜
 4824 来求最短路

4825 // 请你求出 1 号点到 n 号点的最短距离，如果从 1 号点无法走到 n 号点，输出 -1。

4826

4827 // 输入格式

4828 // 第一行包含两个整数 n 和 m 。

4829 // 接下来 m 行，每行包含两个整数 a 和 b ，表示存在一条从 a 走到 b 的长度为 1 的边。

4830

4831 // 输出格式

4832 // 输出一个整数，表示 1 号点到 n 号点的最短距离。

4833

4834 // 数据范围

4835 // $1 \leq n, m \leq 10^5$

4836

4837 // 样例

4838 // 输入样例：

4839 // 4 5

4840 // 1 2

4841 // 2 3

4842 // 3 4

4843 // 1 3

4844 // 1 4

4845 // 输出样例：

4846 // 1

4847

4848 #include<iostream>

4849 #include<cstring>

4850 #include<algorithm>

4851 using namespace std;

4852 const int N = 100010;

4853 int n, m;

4854 int h[N], e[N], ne[N], idx;



```
4855 int d[N], q[N];
4856
4857 void add(int a, int b)
4858 {
4859     e[idx] = b;
4860     ne[idx] = h[a];
4861     h[a] = idx;
4862     idx ++;
4863 }
4864
4865 int bfs()
4866 {
4867     int hh = 0, tt = 0;
4868     q[0] = 1;
4869     memset(d, -1, sizeof d);
4870     d[1] = 0;
4871
4872     while(hh <= tt)
4873     {
4874         int t = q[hh ++];
4875         for(int i = h[t]; i != -1; i = ne[i])
4876         {
4877             int j = e[i];
4878             if(d[j] == -1)
4879             {
4880                 d[j] = d[t] + 1;
4881                 q[++ tt] = j;
4882             }
4883         }
4884     }
4885     return d[n];
4886 }
4887
4888 int main()
4889 {
4890     cin >> n >> m;
4891
4892     memset(h, -1, sizeof h);
4893
4894     for(int i = 0; i < m; i ++)
4895     {
4896         int a, b;
4897         cin >> a >> b;
4898         add(a, b);
```



加油喵！ QwQ ！

```
4899     }  
4900     cout << bfs() << endl;  
4901  
4902     return 0;  
4903 }  
4904  
4905  
4906  
4907  
4908  
4909  
4910  
4911  
4912  
4913  
4914  
4915  
4916  
4917  
4918  
4919  
4920  
4921  
4922  
4923  
4924  
4925  
4926  
4927  
4928  
4929  
4930  
4931  
4932  
4933  
4934  
4935  
4936  
4937  
4938  
4939  
4940  
4941  
4942
```



数和图的遍历（深度优先遍历）

4943

4944

4945 // 放弃很容易，但，努力一定很酷

4946

4947 // 树和图的遍历-深度优先搜索

4948

4949 // AcWing 846. 树的重心

4950

4951 // 题目描述

4952 // 给定一颗树，树中包含 n 个结点（编号 $1 \sim n$ ）和 $n-1$ 条无向边。

4953 // 请你找到树的重心，并输出将重心删除后，剩余各个连通块中点数的最大值。

4954 // 重心定义：重心是指树中的一个结点，如果将这个点删除后，剩余各个连通块中点数的

4955 最大值最小，那么这个节点被称为树的重心。

4956

4957 // 输入格式

4958 // 第一行包含整数 n ，表示树的结点数。

4959

4960 // 接下来 $n-1$ 行，每行包含两个整数 a 和 b ，表示点 a 和点 b 之前存在一条边。

4961

4962 // 输出格式

4963 // 输出一个整数 m ，表示重心的所有的子树中最大的子树的结点数目。

4964

4965 // 数据范围

4966 // $1 \leq n \leq 10^5$

4967

4968 // 样例

4969

4970 // 输入样例

4971 // 9

4972 // 1 2

4973 // 1 7

4974 // 1 4

4975 // 2 8

4976 // 2 5

4977 // 4 3

4978 // 3 9

4979 // 4 6

4980

4981 // 输出样例：

4982 // 4

4983

4984 #include<iostream>



```

4985 #include<algorithm>
4986 #include<cstring>
4987 using namespace std;
4988
4989 const int N = 100010, M = N * 2;
4990
4991 int n;
4992 int h[N], e[M], ne[M], idx;
4993 bool st[N];
4994
4995 int ans = N;
4996
4997 void add(int a, int b)
4998 {
4999     e[idx] = b;
5000     ne[idx] = h[a];
5001     h[a] = idx;
5002     idx ++;
5003 }
5004
5005 // 以 u 为根的子树中点的数量
5006 int dfs(int u)
5007 {
5008     st[u] = true; // 标记一下, 已经被搜过了
5009
5010     int sum = 1, res = 0;
5011     for(int i = h[u]; i != -1; i = ne[i])
5012     {
5013         int j = e[i];
5014         if(!st[j])
5015         {
5016             int s = dfs(j);
5017             res = max(res, s);
5018             sum += s;
5019         }
5020     }
5021
5022     res = max(res, n - sum);
5023
5024     ans = min(ans, res);
5025
5026     return sum;
5027 }
5028

```



加油喵！ QwQ ！

```
5029 int main()
5030 {
5031     cin >> n;
5032
5033     memset(h, -1, sizeof h);
5034
5035     for(int i = 0; i < n - 1; i++)
5036     {
5037         int a, b;
5038         cin >> a >> b;
5039         add(a, b), add(b, a);
5040     }
5041
5042     dfs(1);
5043
5044     cout << ans << endl;
5045
5046     return 0;
5047 }
5048
5049
5050
5051
5052
5053
5054
5055
5056
5057
5058
5059
5060
5061
5062
5063
5064
5065
5066
5067
5068
5069
5070
5071
5072
```



最短路

5073

5074

5075 // 最短路

5076

5077 // 我们学编程不会让我们去数学证明, 我们只需要知道结论, 然后用代码去“实现”就好了, 实际上就比的是记忆能力。

5079

5080 // (在每种情况下, 都有各自的最优抉择)

5081 // 分类:

5082 // 1. 单源最短路 (求的是一个点到其他所有点的最短距离)

5083 // (1). 所有边的权重都是正数

5084 // (a). 朴素 Dijkstra 算法 $O(n^2)$

5085 // 比较适合 稠密图

5086 // (b). 堆优化版的 Dijkstra 算法 $O(m \log n)$

5087 // 比较适合 稀疏图

5088 // (2). 存在负权边

5089 // ** (a). Bellman-Ford 算法 $O(nm)$ // ps: 如果能求出来最短路, 那么这个图里是没有负权回路的 所以会限制循环的次数, 也就是最多走多少次

5091 // 比较适合 存在负权边 的图

5092 // (b). SPFA 算法 $O(m)$ 5093 // 一般 $O(m)$, 最坏 $O(nm)$

5094 // 2. 多源汇最短路 (求的是任意两点之间的最短距离)

5095 // (1). Floyd 算法 $O(n^3)$

5096

5097 // 我们在讲最短路算法的时候, 都是针对有向图的

5098 // 如果遇到无向图, 我们一般会把无向图转化成有向图来处理

5099 // 例如: 把 $a - b$ 无向边 转化成 $a \rightarrow b$ 和 $b \rightarrow a$ 两条有向边

5100

5101

5102 // Dijkstra-朴素 $O(n^2)$

5103

5104 // 1. 初始化距离数组, $dist[1] = 0, dist[i] = \text{inf}$;5105 // 2. for n 次循环 每次循环确定一个 $\min$ 加入 S 集合中, n 次之后就得出所有的最短距离5106 // 3. 将不在 S 中 $dist_{\min}$ 的点 $\rightarrow t$ 5107 // 4. $t \rightarrow S$ 加入最短路集合5108 // 5. 用 t 更新到其他点的距离

5109

5110 // Dijkstra-堆优化 $O(m \log m)$

5111

5112 // 1. 利用邻接表, 优先队列

5113 // 2. 在 `priority_queue[HTML_REMOVED]`, `greater[HTML_REMOVED]` > `heap`; 中将返回堆顶

5114 // 3. 利用堆顶来更新其他点, 并加入堆中类似宽搜



加油喵! QwQ !

```
5115
5116 //      Bellman_ford0(nm)
5117
5118 // 1. 注意连锁想象需要备份, struct Edge{int a,b,c} Edge[M];
5119 // 2. 初始化 dist, 松弛  $\text{dist}[x,b] = \min(\text{dist}[x,b], \text{backup}[x,a] + x.w)$ ;
5120 // 3. 松弛 k 次, 每次访问 m 条边
5121
5122 //      Spfa  $O(n) \sim O(nm)$ 
5123
5124 // 1. 利用队列优化仅加入修改过的地方
5125 // 2. for k 次
5126 // 3. for 所有边利用宽搜模型去优化 bellman_ford 算法
5127 // 4. 更新队列中当前点的所有出边
5128
5129 //      Floyd  $O(n^3)$ 
5130
5131 // 1. 初始化 d
5132 // 2. k, i, j 去更新 d
5133
5134
5135
5136
5137
5138
5139
5140
5141
5142
5143
5144
5145
5146
5147
5148
5149
5150
5151
5152
5153
5154
5155
5156
5157
5158
```



5159 最短路-朴素 Dijkstra 算法

```

5160
5161 // 乘舟侧畔千帆过，病树前头万木春
5162
5163 // 最短路-单源最短路-所有边的权重都是正数-朴素 Dijkstra 算法
5164
5165 // 我们学编程不会让我们去数学证明，我们只需要知道结论，然后用代码去“实现”就好
5166 // 了，实际上就比的是记忆能力。
5167
5168 // （在每种情况下，都有各自的最优抉择）
5169 // 分类：
5170 // 1. 单源最短路（求的是一个点到其他所有点的最短距离）
5171 //     (1). 所有边的权重都是正数
5172 //         *(a). 朴素 Dijkstra 算法       $O(n^2)$ 
5173 //             比较适合 稠密图
5174 //         (b). 堆优化版的 Dijkstra 算法  $O(m \log n)$ 
5175 //             比较适合 稀疏图
5176 //     (2). 存在负权边
5177 //         (a). Bellman-Ford 算法       $O(nm)$ 
5178 //             比较适合 存在负权边 的图
5179 //         (b). SPFA 算法               $O(m)$ 
5180 //             一般  $O(m)$ ，最坏  $O(nm)$ 
5181 // 2. 多源汇最短路（求的是任意两点之间的最短距离）
5182 //     (1). Floyd 算法                 $O(n^3)$ 
5183
5184 // 我们在讲最短路算法的时候，都是针对有向图的
5185 // 如果遇到无向图，我们一般会把无向图转化成有向图来处理
5186 // 例如：把  $a - b$  无向边 转化成  $a \rightarrow b$  和  $b \rightarrow a$  两条有向边
5187
5188 // 具体思路：
5189 // 1. 初始化距离数组（一开始只有一个点，距离为 0，其他点距离为正无穷）
5190 // 2. 迭代  $n$  次（每次迭代都会确定一个点的最短路距离，然后用这个点去更新其他点的距
5191 // 离）
5192 //     (1). 找到不在  $s$  中的，距离最近的点  $t$ 
5193 //     (2). 用  $t$  更新其他点的距离
5194 // 3. 输出距离数组（每个点到起点的最短距离）
5195
5196 // 题目来源：AcWing 849. Dijkstra 求最短路 I
5197 // 一、题目描述
5198 // 给定一个  $n$  个点  $m$  条边的有向图，图中可能存在重边和自环，所有边权均为正值。
5199 // 请你求出 1 号点到  $n$  号点的最短距离，如果无法从 1 号点走到  $n$  号点，则输出  $-1$ 。
5200

```



```

5201 // 输入格式
5202 // 第一行包含整数 n 和 m 。
5203 // 接下来 m 行每行包含三个整数 x, y, z, 表示存在一条从点 x 到点 y 的有向边, 边长为
5204 z。
5205
5206 // 输出格式
5207 // 输出一个整数, 表示 1 号点到 n 号点的最短距离。
5208 // 如果路径不存在, 则输出 -1 。
5209
5210 // 数据范围
5211 // 1 ≤ n ≤ 500
5212 // 1 ≤ m ≤ 105
5213 // 图中涉及边长均不超过 10000 。
5214
5215 // 输入样例 :
5216 // 3 3
5217 // 1 2 2
5218 // 2 3 1
5219 // 1 3 4
5220 // 输出样例 :
5221 // 3
5222
5223 #include <iostream>
5224 #include <algorithm>
5225 #include <cstring>
5226
5227 using namespace std;
5228
5229 const int N = 510; // 这是一个稠密图, 所以用邻接矩阵来存
5230
5231 int n, m; // n 个点, m 条边
5232 int g[N][N]; // 邻接矩阵
5233 int dist[N]; // 距离数组: dist[i] 表示 1 号点到 i 号点的最短距离
5234 bool st[N]; // st[i] 表示 i 号点是否已经确定了距离
5235
5236 int dijkstra()
5237 {
5238     // 为什么要用 0x3f,
5239     // 因为 memset 是针对每个字节进行操作的,
5240     // C++ 中 int 型变量所占的位数为 4 个字节, 即 32 位
5241     // 4 个字节均为 0x3f 时, 0x3f3f3f3f 的十进制是 1061109567, 也就是 109 级别的
5242     (和 0x7fffffff 一个数量级)
5243     memset(dist, 0x3f, sizeof dist); // 初始化距离数组, 把所有点的距离都设置为正
5244     无穷

```



```

5245     dist[1] = 0;                                // 把起点的距离设置为 0
5246
5247     for (int i = 0; i < n - 1; i++) // 迭代 n - 1 次
5248     {
5249         int t = -1; // 找到不在 s 中的, 距离最近的点 t
5250         for (int j = 1; j <= n; j++)
5251             // 下面这个 if 的意思就是: 在不在 s 中的点中 (距离还没有确定距离, 即
5252             st[j] 为 false 的第一个点), 找到距离最近的点 t
5253             if (!st[j] && (t == -1 || dist[t] > dist[j])) // 如果 j 不在 s 中, 并
5254             且 t 是第一个点, 或者 dist[t] > dist[j]
5255                 t = j;                                // 那么就把 j 赋值给 t
5256
5257         // // 可以选加的优化
5258         // if (t == n) break; // 如果 t == n, 说明 1 号点到 n 号点的最短距离已经确
5259         定了, 所以就可以退出循环
5260         st[t] = true; // 把 t 加入 s
5261         for (int j = 1; j <= n; j++)                // 用 t 更新其他点的距离
5262             dist[j] = min(dist[j], dist[t] + g[t][j]); // dist[j] = min(dist[j],
5263             dist[t] + g[t][j])
5264         // 为什么是 min 呢?
5265         // 因为我们是要找到 1 号点到 n 号点的最短距离, 所以我们要取最小值
5266     }
5267     if (dist[n] == 0x3f3f3f3f)
5268         return -1; // 如果 dist[n] == 0x3f3f3f3f, 说明 1 号点到 n 号点没有路径,
5269     返回-1
5270
5271     return dist[n]; // 返回 1 号点到 n 号点的最短距离
5272 }
5273
5274 int main()
5275 {
5276     scanf("%d%d", &n, &m);
5277     memset(g, 0x3f, sizeof g); // 初始化邻接矩阵, 把所有边的权重都设置为正无穷
5278     while (m--)
5279     {
5280         int a, b, c;
5281         scanf("%d%d%d", &a, &b, &c);
5282         g[a][b] = min(g[a][b], c); // 因为可能存在重边, 所以要取最小值
5283     }
5284     int t = dijkstra();
5285     printf("%d\n", t);
5286     return 0;
5287 }
5288

```



```
5289 // 如果想优化时间复杂度，可以使用堆优化
5290 // 用堆优化的方式有 2 种：
5291 // 1. 手写堆
5292 // 2. 使用 STL 中的 priority_queue（优先队列）
5293 // 因为优先队列比手写堆要简单，所以我们一般使用优先队列来优化
5294 // 具体优化方法见下一节
5295
5296
5297
5298
5299
5300
5301
5302
5303
5304
5305
5306
5307
5308
5309
5310
5311
5312
5313
5314
5315
5316
5317
5318
5319
5320
5321
5322
5323
5324
5325
5326
5327
5328
5329
5330
5331
5332
```



5333

最短路-堆优化版的Dijkstra 算法

5334

5335 // 最短路

5336

5337 // 我们学编程不会让我们去数学证明, 我们只需要知道结论, 然后用代码去“实现”就好了, 实际上就比的是记忆能力。

5339

5340 // (在每种情况下, 都有各自的最优抉择)

5341 // 分类:

5342 // 1. 单源最短路 (求的是一个点到其他所有点的最短距离)

5343 // (1). 所有边的权重都是正数

5344 // (a). 朴素Dijkstra 算法 $O(n^2)$

5345 // 比较适合 稠密图

5346 // (b). 堆优化版的Dijkstra 算法 $O(m \log n)$

5347 // 比较适合 稀疏图

5348 // (2). 存在负权边

5349 // (a). Bellman-Ford 算法 $O(nm)$

5350 // 比较适合 存在负权边 的图

5351 // (b). SPFA 算法 $O(m)$ 5352 // 一般 $O(m)$, 最坏 $O(nm)$

5353 // 2. 多源汇最短路 (求的是任意两点之间的最短距离)

5354 // (1). Floyd 算法 $O(n^3)$

5355

5356 // 我们在讲最短路算法的时候, 都是针对有向图的

5357 // 如果遇到无向图, 我们一般会把无向图转化成有向图来处理

5358 // 例如: 把 $a - b$ 无向边 转化成 $a \rightarrow b$ 和 $b \rightarrow a$ 两条有向边

5359

5360

5361 // 如果想优化时间复杂度, 可以使用堆优化

5362 // 用堆优化的方式有 2 种:

5363 // 1. 手写堆

5364 // 2. 使用 STL 中的 priority_queue (优先队列)

5365

5366 // 因为优先队列比手写堆要简单, 所以我们一般使用优先队列来优化

5367

5368 // 题目描述:

5369 // 给定一个 n 个点 m 条边的有向图, 图中可能存在重边和自环, 所有边权均为非负值。5370 // 请你求出 1 号点到 n 号点的最短距离, 如果无法从 1 号点走到 n 号点, 则输出 -1。

5371

5372 // 输入输出格式:

5373 // 输入

5374 // 第一行包含整数 n 和 m 。

加油喵! QwQ !

```
5375 // 接下来 m 行每行包含三个整数 x, y, z, 表示存在一条从点 x 到点 y 的有向边, 边长为
5376 z。
5377
5378 // 输出
5379 // 输出一个整数, 表示 1 号点到 n 号点的最短距离。
5380 // 如果路径不存在, 则输出 -1。
5381
5382 // 样例 :
5383 // 输入
5384 // 3 3
5385 // 1 2 2
5386 // 2 3 1
5387 // 1 3 4
5388
5389 // 输出
5390 // 3
5391
5392 #include <iostream>
5393 #include <algorithm>
5394 #include <cstring>
5395 #include <queue>
5396
5397 using namespace std;
5398
5399 typedef pair<int, int> PII; // 定义一个 pair 类型, 第一个元素是距离, 第二个元素是
5400 点的编号
5401
5402 const int N = 100010;
5403
5404 int n, m; // n 个点, m 条边
5405 int h[N], e[N], ne[N], w[N], idx; // h 是邻接表的头结点, e 是边的终点, ne 是下一
5406 条边的编号, w 是边的权重, idx 是边的编号
5407 int dist[N]; // 存储 1 号点到每个点的最短距离
5408 bool st[N]; // 存储每个点是否已经确定了距离
5409
5410 void add(int a, int b, int c) // 添加一条边 a->b, 权重为 c
5411 {
5412
5413     // 这样子想 : h[a] 是一个链表, 链表的头结点是 h[a], 头结点的下一个是起点结点是
5414     ne[idx], 即 ne[h[a]], 头结点的下一个的下一个是起点结点是 ne[ne[h[a]]], 以此类推
5415     // 其中 e[idx] 是起点对应的终点, w[idx] 是权重, 可以看成起点的 2 个内容元素。
5416     e[idx] = b;
5417     // cout << "e[" << idx << "]" = " << b << endl;
5418     w[idx] = c;
```



```

5419 // cout << "w[" << idx << "]" = " << c << endl;
5420 ne[idx] = h[a];
5421 // cout << "ne[" << idx << "]" = " << h[a] << endl;
5422 h[a] = idx;
5423 // cout << "h[" << a << "]" = " << idx << endl;
5424 idx++;
5425 // cout << "idx = " << idx << endl;
5426 }
5427
5428 int dijkstra()
5429 {
5430     memset(dist, 0x3f, sizeof dist);
5431     dist[1] = 0;
5432
5433     priority_queue<PII, vector<PII>, greater<PII>> heap; // 定义一个小根堆, 存储
5434 距离和点的编号
5435     heap.push({0, 1}); // 把距离为 0 的 1 号点加入堆中
5436
5437     while (heap.size()) // while 堆不为空
5438     {
5439         auto t = heap.top(); // 每次找到距离最小的点, 也就是堆顶元素
5440         heap.pop(); // 弹出堆顶元素
5441
5442         int ver = t.second, distance = t.first; // ver 是点的编号, distance 是距
5443 离
5444
5445         if (st[ver]) continue; // 如果这个点已经被确定了距离, 就跳过
5446         // 比如有 2 条路被记录到堆中, 一条是{4, 3}, {5, 3}, 前者是{ 距离为 4, 起
5447 点到点 3 }, 后者是{ 距离为 5, 起点到点 3 }
5448         // 由于小根堆的特性, {4, 3} 会被先取出, 这是一切正常运行, st[3] 会从 false
5449 标记成 true, 然后堆里剩下的那个{5, 3}就没有了意义, 等到{5, 4}时就会直接 continue
5450         st[ver] = true;
5451
5452         for (int i = h[ver]; i != -1; i = ne[i]) // 遍历 ver 的所有出边
5453         {
5454             int j = e[i]; // j 是 ver 的出边的终点
5455             if (dist[j] > distance + w[i]) // 如果 j 的距离大于 ver 的距离加上 ver
5456 到 j 的权重
5457             {
5458                 dist[j] = distance + w[i]; // 更新 j 的距离
5459                 heap.push({dist[j], j}); // 把 j 加入堆中
5460             }
5461         }
5462     }

```




```
5463
5464
5465     if (dist[n] == 0x3f3f3f3f) return -1; // 如果 dist[n] == 0x3f3f3f3f, 说明 1
5466     号点到 n 号点没有路径, 返回-1
5467
5468     return dist[n]; // 返回 1 号点到 n 号点的最短距离
5469 }
5470
5471 int main()
5472 {
5473     scanf("%d%d", &n, &m);
5474
5475     memset(h, -1, sizeof h); // 初始化邻接表
5476
5477     while (m--)
5478     {
5479         int a, b, c;
5480         scanf("%d%d%d", &a, &b, &c);
5481         add(a, b, c); // 添加一条边 a->b, 权重为 c
5482     }
5483
5484     int t = dijkstra();
5485
5486     printf("%d\n", t);
5487
5488     return 0;
5489 }
5490
5491
5492
5493
5494
5495
5496
5497
5498
5499
5500
5501
5502
5503
5504
5505
5506
```



5507 最短路-Bellman-Ford 算法

```

5508
5509 // 最短路
5510
5511 // 我们学编程不会让我们去数学证明, 我们只需要知道结论, 然后用代码去“实现”就好
5512 // 了, 实际上就比的是记忆能力。
5513
5514 // (在每种情况下, 都有各自的最优抉择)
5515 // 分类:
5516 // 1. 单源最短路 (求的是一个点到其他所有点的最短距离)
5517 //     (1). 所有边的权重都是正数
5518 //         (a). 朴素 Dijkstra 算法       $O(n^2)$ 
5519 //             比较适合 稠密图
5520 //         (b). 堆优化版的 Dijkstra 算法  $O(m \log n)$ 
5521 //             比较适合 稀疏图
5522 //     (2). 存在负权边
5523 //         *(a). Bellman-Ford 算法       $O(nm)$  // ps: 如果能求出来最短路, 那
5524 // 么这个图里是没有负权回路的 所以会限制循环的次数, 也就是最多走多少次
5525 //             比较适合 存在负权边 的图
5526 //         (b). SPFA 算法                 $O(m)$ 
5527 //             一般  $O(m)$ , 最坏  $O(nm)$ 
5528 // 2. 多源汇最短路 (求的是任意两点之间的最短距离)
5529 //     (1). Floyd 算法                   $O(n^3)$ 
5530
5531 // 一些名词:
5532 // dist[b] = min(dist[b], dist[a] + w); // 松弛操作
5533 // dist[b] <= dist[a] + w; // 三角不等式
5534
5535 // Bellman-Ford 算法
5536 // 模板:
5537 // int n, m;          // n 表示点数, m 表示边数
5538 // int dist[N];        // dist[x] 存储 1 到 x 的最短路距离
5539
5540 // struct Edge        // 边, a 表示出点, b 表示入点, w 表示边的权重
5541 // {
5542 //     int a, b, w;
5543 // } edges[M];
5544
5545 // // 求 1 到 n 的最短路距离, 如果无法从 1 走到 n, 则返回-1。
5546 // int bellman_ford()
5547 // {
5548 //     memset(dist, 0x3f, sizeof dist);

```



```

5549 //    dist[1] = 0;
5550
5551 //    // 如果第 n 次迭代仍然会松弛三角不等式, 就说明存在一条长度是 n+1 的最短路径,
5552 由抽屉原理, 路径中至少存在两个相同的点, 说明图中存在负权回路。
5553 //    for (int i = 0; i < n; i ++ )
5554 //    {
5555 //        for (int j = 0; j < m; j ++ )
5556 //        {
5557 //            int a = edges[j].a, b = edges[j].b, w = edges[j].w;
5558 //            if (dist[b] > dist[a] + w)
5559 //                dist[b] = dist[a] + w;
5560 //        }
5561 //    }
5562
5563 //    if (dist[n] > 0x3f3f3f3f / 2) return -1;
5564 //    return dist[n];
5565 // }
5566
5567 // 题目描述
5568 // 给定一个 n 个点 m 条边的有向图, 图中可能存在重边和自环, 边权可能为负数。
5569
5570 // 请你求出从 1 号点到 n 号点的最多经过 k 条边的最短距离, 如果无法从 1 号点走到
5571 n 号点, 输出 impossible。
5572 // 注意: 图中可能 存在负权回路 。
5573
5574 // 输入格式
5575 // 第一行包含三个整数 n, m, k。
5576 // 接下来 m 行, 每行包含三个整数 x, y, z, 表示存在一条从点 x 到点 y 的有向边, 边长
5577 为 z, 点的编号为 1~n。
5578
5579 // 输出格式
5580 // 输出一个整数, 表示从 1 号点到 n 号点的最多经过 k 条边的最短距离。
5581 // 如果不存在满足条件的路径, 则输出 impossible。
5582
5583 // 数据范围
5584 // 1 ≤ n, k ≤ 500, 1 ≤ m ≤ 10000, 1 ≤ x, y ≤ n
5585 // 任意边长的绝对值不超过 10000
5586
5587 // 输入样例:
5588 // 3 3 1
5589 // 1 2 1
5590 // 2 3 1
5591 // 1 3 3
5592

```



```

5593 // 输出样例 :
5594 // 3
5595
5596 #include<iostream>
5597 #include<iostream>
5598 #include<algorithm>
5599 #include<cstring>
5600 using namespace std;
5601
5602 const int N = 510, M = 10010;
5603
5604 int n, m, k;
5605 int dist[N], backup[N];
5606
5607 struct Edge
5608 {
5609     int a, b, w;
5610 } edges[N]; // 这里 edges[N] 是一个数组, 数组的元素是 Edge 类型的结构体
5611
5612 int bellman_ford()
5613 {
5614     memset(dist, 0x3f, sizeof dist); // 初始化 dist 数组
5615     dist[1] = 0; // 1 号点到 1 号点的距离为 0
5616
5617     for(int i = 0; i < k; i++)
5618     {
5619         memcpy(backup, dist, sizeof dist); // 备份 dist 数组
5620         // 为什么要备份 dist 数组?
5621         // 因为我们要更新 dist 数组, 但是我们要保证更新 dist 数组的时候, dist 数组
5622         中的值是上一次迭代的结果, 而不是本次迭代的结果
5623         for(int j = 0; j < m; j++)
5624         {
5625             int a = edges[j].a, b = edges[j].b, w = edges[j].w;
5626             dist[b] = min(dist[b], backup[a] + w); // 更新 dist 数组
5627         }
5628     }
5629
5630     if(dist[n] > 0x3f3f3f3f / 2) return -1; // 如果 dist[n] 大于(一个量级很大的数)
5631     0x3f3f3f3f / 2, 说明最短路不存在
5632     return dist[n]; // 否则, 返回 dist[n]
5633
5634 }
5635
5636 int main()

```



加油喵! QwQ !

```
5637 {
5638     scanf("%d%d%d", &n, &m, &k);
5639
5640     for(int i = 0; i < m; i++)
5641     {
5642         int a, b, w;
5643         scanf("%d%d%d", &a, &b, &w);
5644         edges[i] = {a, b, w};
5645     }
5646
5647     int t = bellman_ford();
5648
5649     if(t == -1) puts("impossible"); // 如果最短路不存在, 输出 impossible
5650     else printf("%d\n", t); // 否则, 输出最短距离
5651
5652     return 0;
5653 }
5654
5655
5656
5657
5658
5659
5660
5661
5662
5663
5664
5665
5666
5667
5668
5669
5670
5671
5672
5673
5674
5675
5676
5677
5678
5679
5680
```



最短路-SPFA 算法

5681

5682

5683 // 最短路

5684

5685 // 我们学编程不会让我们去数学证明, 我们只需要知道结论, 然后用代码去“实现”就好了, 实际上就比的是记忆能力。

5686

5687 // (在每种情况下, 都有各自的最优抉择)

5688 // 分类:

5689 // 1. 单源最短路 (求的是一个点到其他所有点的最短距离)

5690 // (1). 所有边的权重都是正数

5691 // (a). 朴素 Dijkstra 算法 $O(n^2)$

5692 // 比较适合 稠密图

5693 // (b). 堆优化版的 Dijkstra 算法 $O(m \log n)$

5694 // 比较适合 稀疏图

5695 // (2). 存在负权边

5696 // (a). Bellman-Ford 算法 $O(nm)$

5697 // 比较适合 存在负权边 的图

5698 // *(b). SPFA 算法 $O(m)$ // ps: 如果能求出来最短路, 那么

5699 这个图里是没有负权回路的

5700 // 一般 $O(m)$, 最坏 $O(nm)$

5701 // 2. 多源汇最短路 (求的是任意两点之间的最短距离)

5702 // (1). Floyd 算法 $O(n^3)$

5703

5704 // 一些名词:

5705 // $\text{dist}[b] = \min(\text{dist}[b], \text{dist}[a] + w)$; // 松弛操作5706 // $\text{dist}[b] \leq \text{dist}[a] + w$; // 三角不等式

5707

5708 // spfa 算法 (队列优化的 Bellman-Ford 算法) 模板题

5709 // AcWing 851. spfa 求最短路

5710 // 时间复杂度是 $O(m)$, 一般情况下比 Dijkstra 算法要快, 最坏情况下是 $O(nm)$ 。// 大部分情况下, 正权图也可以用 spfa 算法。

5711 // 注意: spfa 算法不能处理负权回路, 因为负权回路是没有最短路的。

5712 // 模板:

5713 // int n; // 总点数

5714 // int h[N], w[N], e[N], ne[N], idx; // 邻接表存储所有边

5715 // int dist[N]; // 存储每个点到 1 号点的最短距离

5716 // bool st[N]; // 存储每个点是否在队列中

5717

5718 // // 求 1 号点到 n 号点的最短路距离, 如果从 1 号点无法走到 n 号点则返回-1

5719 // int spfa()

5720 // {



```

5723 //    memset(dist, 0x3f, sizeof dist);
5724 //    dist[1] = 0;
5725
5726 //    queue<int> q;
5727 //    q.push(1);
5728 //    st[1] = true;
5729
5730 //    while (q.size())
5731 //    {
5732 //        auto t = q.front();
5733 //        q.pop();
5734
5735 //        st[t] = false;
5736
5737 //        for (int i = h[t]; i != -1; i = ne[i])
5738 //        {
5739 //            int j = e[i];
5740 //            if (dist[j] > dist[t] + w[i])
5741 //            {
5742 //                dist[j] = dist[t] + w[i];
5743 //                if (!st[j])    // 如果队列中已存在 j, 则不需要将 j 重复插入
5744 //                {
5745 //                    q.push(j);
5746 //                    st[j] = true;
5747 //                }
5748 //            }
5749 //        }
5750 //    }
5751
5752 //    if (dist[n] == 0x3f3f3f3f) return -1;
5753 //    return dist[n];
5754 // }
5755
5756 // 题目来源: AcWing 851. spfa 求最短路
5757
5758 // 一、题目描述
5759 // 给定一个 n 个点 m 条边的有向图, 图中可能存在重边和自环, 边权可能为负数。
5760 // 请你求出 1 号点到 n 号点的最短距离, 如果无法从 1 号点走到 n 号点, 则输出
5761 // impossible。
5762 // 数据保证不存在负权回路。
5763
5764 // 输入格式
5765 // 第一行包含整数 n 和 m。
5766 // 接下来 m 行每行包含三个整数 x, y, z, 表示存在一条从点 x 到点 y 的有向边, 边

```



```

5767 长为 z。
5768
5769 // 输出格式
5770 // 输出一个整数，表示 1 号点到 n 号点的最短距离。
5771 // 如果路径不存在，则输出 impossible。
5772
5773 // 数据范围
5774 // 1 ≤ n, m ≤ 10^5, 图中涉及边长绝对值均不超过 10000。
5775
5776 // 输入样例：
5777 // 3 3
5778 // 1 2 5
5779 // 2 3 -3
5780 // 1 3 4
5781
5782 // 输出样例：
5783 // 2
5784
5785 #include<iostream>
5786 #include<algorithm>
5787 #include<cstring>
5788 #include<queue>
5789
5790 using namespace std;
5791
5792 const int N = 100010;
5793
5794 int n, m;
5795 int h[N], e[N], ne[N], w[N], idx;
5796 int dist[N];
5797 bool st[N];
5798
5799 void add(int a, int b, int c)
5800 {
5801     w[idx] = c;
5802     e[idx] = b;
5803     ne[idx] = h[a];
5804     h[a] = idx;
5805     // 这样子想：h[a]是一个链表，链表的头结点是 h[a]，头结点的下一个是起点结点是
5806     ne[idx]，即 ne[h[a]]，头结点的下一个的下一个是起点结点是 ne[ne[h[a]]]，以此类推
5807     // 其中 e[idx]是起点对应的终点，w[idx]是权重，可以看成起点的 2 个内容元素。
5808     idx ++;
5809 }
5810

```




```

5811 int spfa()
5812 {
5813     memset(dist, 0x3f, sizeof dist);
5814     dist[1] = 0;
5815
5816     queue<int> q;
5817     q.push(1); // 把距离为 0 的 1 号点加入队列中
5818     st[1] = true; // 表示 1 号点在队列中
5819
5820     while(q.size())
5821     {
5822         int t = q.front();
5823         q.pop();
5824
5825         st[t] = false; // 表示 t 号点不在队列中
5826
5827         for(int i = h[t]; i != -1; i = ne[i]) // 更新 t 号点的所有出边
5828         {
5829             int j = e[i];
5830             if(dist[j] > dist[t] + w[i]) // 如果 j 号点的距离大于 t 号点的距离加上
5831 边权
5832             {
5833                 dist[j] = dist[t] + w[i]; // 更新 j 号点的距离
5834                 if(!st[j]) // 如果 j 号点不在队列中
5835                 {
5836                     q.push(j); // 把 j 号点加入队列中
5837                     st[j] = true; // 表示 j 号点在队列中
5838                 }
5839             }
5840         }
5841     }
5842     if(dist[n] == 0x3f3f3f3f) return -1;
5843     return dist[n];
5844 }
5845
5846 int main()
5847 {
5848     cin >> n >> m;
5849     memset(h, -1, sizeof h);
5850     while(m--)
5851     {
5852         int a, b, c;
5853         cin >> a >> b >> c;
5854         add(a, b, c);

```



加油喵! QwQ !

```
5855     }
5856     int t = spfa();
5857     if(t == -1) cout << "impossible" << endl;
5858     else cout << t << endl;
5859
5860     return 0;
5861 }
5862
5863
5864
5865
5866
5867
5868
5869
5870
5871
5872
5873
5874
5875
5876
5877
5878
5879
5880
5881
5882
5883
5884
5885
5886
5887
5888
5889
5890
5891
5892
5893
5894
5895
5896
5897
5898
```



5899

最短路-Floyd 算法

5900

5901 // 最短路

5902

5903 // 我们学编程不会让我们去数学证明, 我们只需要知道结论, 然后用代码去“实现”就好了, 实际上就比的是记忆能力。

5905

5906 // (在每种情况下, 都有各自的最优抉择)

5907 // 分类:

5908 // 1. 单源最短路 (求的是一个点到其他所有点的最短距离)

5909 // (1). 所有边的权重都是正数

5910 // (a). 朴素 Dijkstra 算法 $O(n^2)$

5911 // 比较适合 稠密图

5912 // (b). 堆优化版的 Dijkstra 算法 $O(m \log n)$

5913 // 比较适合 稀疏图

5914 // (2). 存在负权边

5915 // (a). Bellman-Ford 算法 $O(nm)$

5916 // 比较适合 存在负权边 的图

5917 // (b). SPFA 算法 $O(m)$ 5918 // 一般 $O(m)$, 最坏 $O(nm)$

5919 // 2. 多源汇最短路 (求的是任意两点之间的最短距离)

5920 // *(1). Floyd 算法 $O(n^3)$

5921

5922 // 题目来源: AcWing 854. Floyd 算法求最短路

5923 // 题目描述:

5924 // 给定一个 n 个点 m 条边的有向图, 图中可能存在重边和自环, 边权可能为负数。

5925 // 再给定 k 个询问, 每个询问包含两个整数 x 和 y , 表示查询从点 x 到点 y 的最短距离,

5926 // 如果路径不存在, 则输出 impossible。

5927 // 数据保证图中不存在负权回路。

5929

5930 // 输入输出格式:

5931

5932 // 输入

5933 // 第一行包含三个整数 n, m, k 。

5934 // 接下来 m 行, 每行包含三个整数 x, y, z , 表示存在一条从点 x 到点 y 的有向边, 边长为 z 。

5936 // 接下来 k 行, 每行包含两个整数 x, y , 表示询问点 x 到点 y 的最短距离。

5937

5938 // 输出

5939 // 共 k 行, 每行输出一个整数, 表示询问的结果, 若询问两点间不存在路径, 则输出 impossible。

5940



```

5941
5942 // 数据范围
5943 // 1   n   200
5944 // 1   k   n^2
5945 // 1   m   20000
5946 // 图中涉及边长绝对值均不超过 10000。
5947
5948 // 输入输出样例：
5949
5950 // 输入
5951 // 3 3 2
5952 // 1 2 1
5953 // 2 3 2
5954 // 1 3 1
5955 // 2 1
5956 // 1 3
5957
5958 // 输出
5959 // impossible
5960 // 1
5961
5962 #include<iostream>
5963 #include<algorithm>
5964 #include<cstring>
5965
5966 using namespace std;
5967
5968 const int N = 210, INF = 1e9;
5969
5970 int n, m, Q; // n 个点, m 条边, Q 个询问
5971 int d[N][N]; // d 是邻接矩阵, d[i][j] 表示从 i 到 j 的最短距离
5972
5973 void floyd()
5974 {
5975     for(int k = 1; k <= n; k++) // 枚举中间点
5976         for(int i = 1; i <= n; i++) // 枚举起点
5977             for(int j = 1; j <= n; j++) // 枚举终点
5978                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]); // 松弛操作
5979 }
5980
5981 int main()
5982 {
5983     scanf("%d%d%d", &n, &m, &Q);
5984

```



加油喵! QwQ !

```
5985     // 初始化邻接矩阵, 将所有点到自己的距离初始化为 0, 其他点到自己的距离初始化为无穷大
5986
5987     for(int i = 1; i <= n; i++)
5988         for(int j = 1; j <= n; j++)
5989             if(i == j) d[i][j] = 0;
5990             else d[i][j] = INF;
5991
5992     while(m--)
5993     {
5994         int a, b, w;
5995         scanf("%d%d%d", &a, &b, &w);
5996
5997         d[a][b] = min(d[a][b], w); // 有重边, 只保留最短的边
5998     }
5999
6000     floyd();
6001
6002     // 处理询问
6003     while(Q--)
6004     {
6005         int a, b;
6006         scanf("%d%d", &a, &b);
6007
6008         if(d[a][b] > INF / 2) puts("impossible"); // 如果距离大于 INF/2, 说明没有路径
6009         else printf("%d\n", d[a][b]);
6010     }
6011
6012
6013     return 0;
6014 }
6015
6016
6017
6018
6019
6020
6021
6022
6023
6024
6025
6026
6027
6028
```



6029

最小生成树-朴素 Prim 算法

6030

6031 // acWing- 32. 最小生成树 1-朴素 Prim 算法

6032

6033 // 最小生成树

6034 // 1. 普利姆算法(Prim)

6035 // ** (1). 朴素版 Prim 算法 $O(n^2)$ ----- 适合稠密图6036 // (2). 堆优化版 Prim 算法 $O(m \log n)$ ----- 适合稀疏图6037 // 2. ** 克鲁斯卡尔算法(Kruskal) $O(m \log m)$ ----- 适合稀疏图

6038

6039 // (稀疏图一般用 Kruskal, 因为相较于 堆优化版 Prim, 代码更简单, 思路更清晰)

6040

6041 // 题目来源: AcWing 858. Prim 算法求最小生成树

6042 // 一、题目描述

6043 // 给定一个 n 个点 m 条边的无向图, 图中可能存在重边和自环, 边权可能为负数。

6044 // 求最小生成树的树边权重之和, 如果最小生成树不存在则输出 impossible。

6045 // 给定一张边带权的无向图 $G=(V, E)$, 其中 V 表示图中点的集合, E 表示图中边的集合,
6046 $n=|V|$, $m=|E|$ 。6047 // 由 V 中的全部 n 个顶点和 E 中 $n-1$ 条边构成的无向连通子图被称为 G 的一棵生成
6048 树, 其中边的权值之和最小的生成树被称为无向图 G 的最小生成树。

6049

6050 // 输入格式

6051 // 第一行包含两个整数 n 和 m 。6052 // 接下来 m 行, 每行包含三个整数 u, v, w , 表示点 u 和点 v 之间存在一条权值为 w 的
6053 边。

6054

6055 // 输出格式

6056 // 共一行, 若存在最小生成树, 则输出一个整数, 表示最小生成树的树边权重之和, 如果
6057 最小生成树不存在则输出 impossible。

6058

6059 // 数据范围

6060 // $1 \leq n \leq 500, 1 \leq m \leq 10^5$

6061 // 图中涉及边的边权的绝对值均不超过 10000。

6062

6063 // 输入样例:

6064 // 4 5

6065 // 1 2 1

6066 // 1 3 2

6067 // 1 4 3

6068 // 2 3 2

6069 // 3 4 4

6070



```

6071 // 输出样例 :
6072 // 6
6073
6074 // 二、最小生成树 Prim 算法
6075 // 什么是最小生成树问题?
6076 // 假设地图中有很多城市, 现在需要在城市中修建一条公路, 使得各个城市可以联通 (类
6077 似于主线), 现在询问最小需要修多少距离的公路。
6078 // 凡是最小生成树的问题, 一般而言对应的图是无向图。而解决最小生成树问题通常使用
6079 的是 Prim 和 Kruskal 算法。
6080 // 一般而言, 对于稠密图的最小生成树问题我们使用 Prim 算法, 时间复杂度是  $O(n^2)$ ;
6081 // 对于稀疏图, 我们一般使用的是 Kruskal 算法, 时间复杂度是  $O(m\log m)$ 。
6082 // Prim 算法与朴素版的 Dijkstra 算法非常相像。
6083 // 伪代码如下:
6084 // 首先, 初始化距离数组 dist 为正无穷, 表示每个点到最小生成树的距离初始都是  $\infty$ 
6085 //   for (int i = 0; i < n; i++)
6086 //   {
6087 //       t ← 找到最小生成树之外, 距离最小生成树最近的点
6088 //       用 t 更新其他点到最小生成树的距离
6089 //       将 t 加入到最小生成树中去
6090 //   }
6091 // 注意:
6092 // 用 t 更新其他点到最小生成树的距离是指, 如果其他点到最小生成树有多条边, 那么只
6093 取最短的那一条。
6094 // 如果不存在任何一条边与最小生成树相连, 则目前距离仍为正无穷。
6095
6096 #include<iostream>
6097 #include<cstring>
6098 #include<algorithm>
6099
6100 using namespace std;
6101
6102 const int N = 510, INF = 0x3f3f3f3f;
6103
6104 int n, m;
6105 int g[N][N];
6106 int dist[N];
6107 bool st[N];
6108
6109 int prim()
6110 {
6111     memset(dist, 0x3f, sizeof dist);
6112
6113     int res = 0;
6114     for(int i = 0; i < n; i++)

```



```

6115     {
6116         int t = -1;
6117         for(int j = 1; j <= n; j++)
6118             if(!st[j] && (t == -1 || dist[t] > dist[j]))
6119                 t = j;
6120
6121         if(i && dist[t] == INF) return INF; // 如果存在一个点到最小生成树的距离
6122         是正无穷, 那么就说明这个图不是连通图, 即不存在最小生成树
6123         if(i) res += dist[t]; // 如果不是第一个点, 则将距离加入到最小生成树中去
6124
6125         for(int j = 1; j <= n; j++)
6126             dist[j] = min(dist[j], g[t][j]); // 用 t 更新其他点到最小生成树的距离
6127             // (与Dijkstra算法的区别是Dijkstra算法是用t更新其他点到起点的距离,
6128 即代码是 dist[j] = min(dist[j], dist[j] + g[t][j]);
6129             // 为什么要先累加再更新呢?
6130             // 因为Prim算法是用t更新其他点到最小生成树的距离, 所以需要先累加再更新
6131
6132         st[t] = true; // 将t加入到最小生成树中去
6133
6134     }
6135     return res;
6136 }
6137
6138 int main()
6139 {
6140     scanf("%d%d", &n, &m);
6141
6142     memset(g, 0x3f, sizeof g);
6143
6144     while(m--)
6145     {
6146         int a, b, c;
6147         scanf("%d%d%d", &a, &b, &c);
6148         g[a][b] = g[b][a] = min(g[a][b], c);
6149     }
6150
6151     int t = prim();
6152
6153     if(t == INF) puts("impossible");
6154     else printf("%d¥n", t);
6155
6156     return 0;
6157 }
6158

```



6159

最小生成树-堆优化 Prim 算法 (略)

6160

6161 // acWing- 32. 最小生成树 2-堆优化 Prim 算法

6162

6163 // 最小生成树

6164 // 1. 普利姆算法(Prim)

6165 // (1). 朴素版 Prim 算法 $O(n^2)$ ----- 适合稠密图6166 // (2). 堆优化版 Prim 算法 $O(m \log n)$ ----- 适合稀疏图6167 // 2. **克鲁斯卡尔算法(Kruskal) $O(m \log m)$ ----- 适合稀疏图

6168

6169 // (稀疏图一般用 Kruskal, 因为相较于 堆优化版 Prim, 代码更简单, 思路更清晰)

6170

6171

6172

6173

6174

6175

6176

6177

6178

6179

6180

6181

6182

6183

6184

6185

6186

6187

6188

6189

6190

6191

6192

6193

6194

6195

6196

6197

6198

6199

6200



6201

最小生成树-Kruskal 算法

6202

6203 // Kruskal 算法

6204 // 1. 将所有边按权重从小到大排序 $O(m \log m)$

6205 // 这一步是 Kruskal 算法的瓶颈, 决定了 Kruskal 算法的时间复杂度

6206 // 2. 枚举每条边 a, b, w $O(m)$ 6207 // 如果 a, b 不连通, 将这条边加入到集合中

6208 // 为什么呢?

6209 // 因为如果 a, b 连通, 那么将 a, b 连起来, 会形成一个环, 那么环中一定存在一条边的权重大于等于 a, b 之间的边的权重。

6211

6212

6213 // 题目来源: AcWing 859. Kruskal 算法求最小生成树

6214

6215 // 一、题目描述

6216 // 给定一个 n 个点 m 条边的无向图, 图中可能存在重边和自环, 边权可能为负数。

6217 // 求最小生成树的树边权重之和, 如果最小生成树不存在则输出 impossible。

6218 // 给定一张边带权的无向图 $G = (V, E)$ $G=(V, E)$ $G=(V, E)$, 其中 V 表示图中点的集合,
6219 E 表示图中边的集合, $n=|V|$, $m=|E|$ 。6220 // 由 V 中的全部 n 个顶点和 E 中 $n-1$ 条边构成的无向连通子图被称为 G 的一棵生成
6221 树, 其中边的权值之和最小的生成树被称为无向图 G 的最小生成树。

6222

6223 // 输入格式

6224 // 第一行包含两个整数 n 和 m 。6225 // 接下来 m 行, 每行包含三个整数 u, v, w , 表示点 u 和点 v 之间存在一条权值为 w 的
6226 边。

6227

6228 // 输出格式

6229 // 共一行, 若存在最小生成树, 则输出一个整数, 表示最小生成树的树边权重之和, 如果
6230 最小生成树不存在则输出 impossible。

6231

6232 // 数据范围

6233 // $1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$

6234 // 图中涉及边的边权的绝对值均不超过 1000。

6235

6236 // 输入样例:

6237 // 4 5

6238 // 1 2 1

6239 // 1 3 2

6240 // 1 4 3

6241 // 2 3 2

6242 // 3 4 4



```

6243
6244 // 输出样例 :
6245 // 6
6246
6247 // 二、最小生成树 Kruskal 算法
6248 // Kruskal 算法非常简单 :
6249 // 将所有边按权重从小到大排序 (这也是该算法时间复杂度瓶颈 :  $O(m\log m)$  )
6250 // 枚举每条边  $a \rightarrow b$  , 权重是  $c$  。如果当前  $a, b$  不在一个集合中, 则将该边加入集合中
6251 (并查集) 。
6252
6253 #include <iostream>
6254 #include <algorithm>
6255 #include <cstring>
6256
6257 using namespace std;
6258
6259 const int N = 200010;
6260
6261 int n, m;
6262 int p[N];
6263
6264 struct Edge
6265 {
6266     int a, b, w;
6267
6268     bool operator< (const Edge &W) const
6269     {
6270         return w < W.w; // 从小到大排序
6271     }
6272
6273 } edges[N];
6274
6275 int find(int x)
6276 {
6277     if(p[x] != x) p[x] = find(p[x]);
6278     return p[x];
6279 }
6280
6281 int main()
6282 {
6283     scanf("%d%d", &n, &m);
6284
6285     for(int i = 0; i < m; i++)
6286     {

```



```

6287         int a, b, w;
6288         scanf("%d%d%d", &a, &b, &w);
6289         edges[i] = {a, b, w};
6290     }
6291
6292     sort(edges, edges + m);
6293
6294     for(int i = 1; i <= n; i++)
6295     {
6296         p[i] = i;
6297     }
6298
6299     int res = 0, cnt = 0; // res 是最小生成树的树边权重之和, cnt 是最小生成树的
6300 边数
6301     // 因为最小生成树的边数是 n - 1, 所以当 cnt == n - 1 时, 最小生成树就已经构
6302 造完毕了。
6303
6304     // 枚举每条边
6305     for(int i = 0; i < m; i++)
6306     {
6307         int a = edges[i].a;
6308         int b = edges[i].b;
6309         int w = edges[i].w;
6310
6311         a = find(a); // 找到 a 的祖宗节点
6312         b = find(b); // 找到 b 的祖宗节点
6313
6314         if(a != b) // 如果 a, b 不在一个集合中
6315         {
6316             res += w;
6317             cnt++;
6318             p[a] = b; // 将 a 的祖宗节点指向 b 的祖宗节点
6319         }
6320     }
6321
6322     if(cnt < n - 1) puts("impossible"); // 如果加的边数小于 n-1 的话, 说明这个连
6323 通图是不连通的
6324     else printf("%d\n", res); // 否则输出最小生成树的树边权重之和
6325
6326     return 0;
6327 }
6328
6329
6330

```



二分图-染色法

6331

6332

6333 // 二分图

6334

6335 // 学算法 -> 遵循实用主义

6336

6337 // 二分图

6338 // 1. **染色法(本质上就是一个简单的 dfs) $O(n + m)$

6339 // 注: 可以用来判断一个图是否是二分图

6340 // 2. 匈牙利算法 一般是 $O(m)$, 最坏是 $O(nm)$

6341

6342 // 二分图的性质:

6343 // 1. 二分图不存在奇数环

6344

6345 // 染色法个人理解: 相邻的点被染色成不同的颜色, 若是存在矛盾, 就证明不是二分图

6346 // 1. 首先我们要明确一点: 二分图是一个图, 其中的点可以被分为两个集合, 使得所有的边都只出现在集合之间。

6348 // 2. 我们可以用一个数组来存储每个点的颜色, 用 0 表示未染色, 1 表示染成了红色, 2 表示染成了蓝色。

6350 // 3. 我们可以用一个 dfs 来遍历整个图, 对于每个点, 如果它没有被染色, 我们就将它染成红色, 然后将它的所有邻接点染成蓝色。

6352 // 4. 如果某个点已经被染色了, 我们就判断它的颜色是否和它的邻接点的颜色相同, 如果相同, 就说明这个图不是二分图。

6354 // 5. 如果某个点没有被染色, 我们就将它染成红色, 然后将它的所有邻接点染成蓝色。

6355 // 6. 如果某个点已经被染色了, 我们就判断它的颜色是否和它的邻接点的颜色相同, 如果相同, 就说明这个图不是二分图。

6356

6357 // ACwing 860 - 染色法判断二分图 (染色法)

6358 // 题目描述

6359 // 给定一个 n 个点 m 条边的无向图, 图中可能存在重边和自环。

6360 // 请你判断这个图是否是二分图。

6361

6362 // 输入格式

6363 // 第一行包含两个整数 n 和 m 。6364 // 接下来 m 行, 每行包含两个整数 u 和 v , 表示点 u 和点 v 之间存在一条边。

6365

6366 // 输出格式

6367 // 如果给定图是二分图, 则输出 "Yes", 否则输出 "No"。

6368

6369 // 数据范围

6370 // $1 \leq n, m \leq 10^5$

6371



```

6373 // 输入样例 :
6374 // 4 4
6375 // 1 3
6376 // 1 4
6377 // 2 3
6378 // 2 4
6379
6380 // 输出样例 :
6381 // Yes
6382
6383 // 题目大意 :
6384 // 输入 n m 表示 n 个顶点 m 条边, 可能有重边和自环。判断该图是不是一个二分图。
6385
6386 #include<iostream>
6387 #include<algorithm>
6388 #include<cstring>
6389
6390 using namespace std;
6391
6392 const int N = 100010, M = 200010; // 这个图是无向图, 边数 * 2
6393
6394 int n, m;
6395 int h[N], e[M], ne[M], idx;
6396 int color[N];
6397
6398 void add(int a, int b)
6399 {
6400     e[idx] = b;
6401     ne[idx] = h[a];
6402     h[a] = idx;
6403     idx++;
6404 }
6405
6406 bool dfs(int u, int c)
6407 {
6408     color[u] = c;
6409
6410     for(int i = h[u]; i != -1; i = ne[i])
6411     {
6412         int j = e[i];
6413         if(!color[j])
6414         {
6415             if(!dfs(j, 3 - c)) return false;
6416         }

```



```

6417         else if(color[j] == c) return false;
6418     }
6419
6420     return true;
6421 }
6422
6423 int main()
6424 {
6425     scanf("%d%d", &n, &m);
6426
6427     memset(h, -1, sizeof h);
6428
6429     while(m--)
6430     {
6431         int a, b;
6432         scanf("%d%d", &a, &b);
6433         add(a, b), add(b, a);
6434     }
6435
6436     bool flag = true;
6437     for(int i = 1; i <= n; i++)
6438     {
6439         if(color[i] == 0)
6440         {
6441             color[i] = 1;
6442             if(!dfs(i, 1))
6443             {
6444                 flag = false;
6445                 return 0;
6446             }
6447         }
6448     }
6449
6450     if(flag) puts("Yes");
6451     else puts("No");
6452
6453     return 0;
6454 }
6455
6456
6457
6458
6459
6460

```



二分图-匈牙利算法

6461

6462

6463 // 二分图

6464

6465 // 学算法 -> 遵循实用主义

6466

6467 // 二分图

6468 // 1. 染色法(本质上就是一个简单的 dfs) $O(n + m)$

6469 // 注: 可以用来判断一个图是否是二分图

6470 // **2. 匈牙利算法 一般是 $O(m)$, 最坏是 $O(nm)$

6471 // 导读: 什么是最大匹配?

6472 // 要了解匈牙利算法必须先理解下面的概念:

6473

6474 // 匹配: 在图论中, 一个「匹配」是一个边的集合, 其中任意两条边都没有公共顶点。

6475 // 最大匹配: 一个图所有匹配中, 所含匹配边数最多的匹配, 称为这个图的最大匹配。

6476

6477 // 下面是一些补充概念:

6478 // 完美匹配: 如果一个图的某个匹配中, 所有的顶点都是匹配点, 那么它就是一个完美匹配。

6480 // 交替路: 从一个未匹配点出发, 依次经过非匹配边、匹配边、非匹配边...形成的路径叫交替路。

6482 // 增广路: 从一个未匹配点出发, 走交替路, 如果途径另一个未匹配点(出发的点不算), 则这条交替路称为增广路 (augmenting path)。

6484

6485 // 匈牙利算法

6486 // 基本思路: 1. 给出一个二分图, 求这个二分图的最大匹配 (匹配指的是不存在两条边共用一个点)

6488 // 2. 可以理解为一个集合里面都是男生, 另一个集合里面都是女生, 问我们最多可以在不出现脚踏多条船的情况下, 结成多少对恋爱关系

6490 // 3. 具体的思路就是, 对于每个男生, 从前往后看, 先看下这个男生所有看上的姑娘, 如果这个姑娘单身, 那么就匹配了

6492 // 4. 成功之后继续下一个男生, 同样也是一样的流程

6493 // 5. 如果某一个男生看上的女生已经有了对象了, 那么就看看这个姑娘看上的对象是谁, 然后看一下这个对象能不能换一个女生, 如果可以换, 那就换

6495 // 6. 如果所有尝试都失败了, 才会放弃

6496 // 7. 如果枚举的是左边的点集, 那么图只需要存从左边指向右边的边, 因为整个过程只会用到从左边指向右边的点

6498 // 8. 为了防止重复搜一个点, 需要一个标记数组

6499

6500 // Acwing861. 二分图的最大匹配 (匈牙利算法)

6501 // 题目描述

6502 // 给定一个二分图, 其中左半部包含 n_1 个点 (编号 $1 - n_1$), 右半部包含 n_2 个点 (编号


```

6503 1 - n2) , 二分图共包含 m 条边。
6504 // 数据保证任意一条边的两个端点都不可能在同一部分中。
6505 // 请你求出二分图的最大匹配数。
6506 // 二分图的匹配：给定一个二分图 G，在 G 的一个子图 M 中，M 的边集{E}中的任意两条边
6507 都不依附于同一个顶点，则称 M 是一个匹配。
6508 // 二分图的最大匹配：所有匹配中包含边数最多的一组匹配被称为二分图的最大匹配，其
6509 边数即为最大匹配数。
6510
6511 // 输入格式
6512 // 第一行包含三个整数 n1、 n2 和 m。
6513 // 接下来 m 行，每行包含两个整数 u 和 v，表示左半部点集中的点 u 和右半部点集中的点 v
6514 之间存在一条边。
6515 // 输出格式
6516 // 输出一个整数，表示二分图的最大匹配数。
6517
6518 // 数据范围
6519 // 1 n1, n2 500
6520 // 1 u n1
6521 // 1 v n2
6522 // 1 m 10^5
6523 // 输入样例：
6524 // 2 2 4
6525 // 1 1
6526 // 1 2
6527 // 2 1
6528 // 2 2
6529 // 输出样例：
6530 // 2
6531
6532 #include<iostream>
6533 #include<algorithm>
6534 #include<cstring>
6535
6536 using namespace std;
6537 const int N = 510, M = 100010;
6538
6539 int n1, n2, m;
6540 int h[N], e[M], ne[M], idx;
6541 int match[N]; // 存储每个女生匹配的男生是谁
6542 bool st[N]; // 标记每个女生是否已经匹配过
6543
6544 void add(int a, int b)
6545 {
6546     e[idx] = b;

```



```

6547     ne[idx] = h[a];
6548     h[a] = idx;
6549     idx++;
6550 }
6551
6552 bool find(int x)
6553 {
6554     for(int i = h[x]; i != -1; i++)
6555     {
6556         int j = e[i]; // 记录这个男生看上的女生
6557         if(!st[j]) // 如果这个女生没有匹配过
6558         {
6559             st[j] = true;
6560             if(match[j] == 0 || find(match[j])) // 该女生还没有匹配上任何一个男
6561 生 或 该女生已经匹配了男生, 但是我们可以为这个男生找到下一个女生
6562             {
6563                 match[j] = x;
6564                 return true;
6565             }
6566         }
6567     }
6568     return false;
6569 }
6570
6571 int main()
6572 {
6573     scanf("%d%d%d", &n1, &n2, &m);
6574     memset(h, -1, sizeof h);
6575     while(m--)
6576     {
6577         int a, b;
6578         scanf("%d%d", &a, &b);
6579         add(a, b);
6580     }
6581     int res = 0;
6582     for(int i = 1; i <= n1; i++) // 依次枚举每个男生
6583     {
6584         memset(st, false, sizeof st); // 一开始这个男生对每个女生的关系都为 false,
6585 // 保证每个女生都只考虑一遍
6586         if(find(i)) res++;
6587     }
6588     printf("%d", res);
6589     return 0;
6590 }

```



6591

优化代码效率

```

6592 #define IOS ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
6593 // 需要在main函数中输入 IOS 来使用这行代码
6594 IOS

```

6595

快速判断素数

```

6596 #include<iostream>
6597 #include<string>
6598 #include<cmath>
6599 #include<vector>
6600 using namespace std;
6601
6602 vector<int>z_num;
6603
6604 bool isPrime(int num)//不是 ai
6605 {
6606     for(int i : z_num)
6607     {
6608         if(num == i) return true;
6609         else
6610         {
6611             if (num <= 3) return num - 1;
6612             // 不在 6 的倍数两侧的一定不是质数
6613             if (num % 6 != 1 && num % 6 != 5) return false;
6614
6615             for (int i = 5; i <= int(sqrt(num)); i += 6)
6616             {
6617                 if (num % i == 0 || num % (i + 2) == 0)
6618                 {
6619                     z_num.push_back(i);
6620                     return false;
6621                 }
6622             }
6623
6624             return true;
6625         }
6626     }
6627     return true;
6628 }

```



```
6629 void answer(bool is)
6630 {
6631     bool answer = isPrime(is);
6632     if(answer)
6633     {
6634         cout << "prime" << endl;
6635     }
6636     else
6637     {
6638         cout << "noprime" << endl;
6639     }
6640 }
6641
6642 int main()
6643 {
6644     int T;
6645     cin >> T;
6646     int num;
6647     while(T--)
6648     {
6649         cin >> num;
6650         bool is = isPrime(num);
6651         if(is)
6652         {
6653             cout << "isprime" << endl;
6654             cout << num << endl;
6655         }
6656         else
6657         {
6658             cout << "noprime" << endl;
6659             for(int i = 2; i <= num; i++)
6660             {
6661                 if(num % i == 0)
6662                 {
6663                     bool is_2 = isPrime(i);
6664                     if(is_2) cout << i << " ";
6665                 }
6666             }
6667             cout << endl;
6668         }
6669     }
6670     return 0;
6671 }
6672
```

